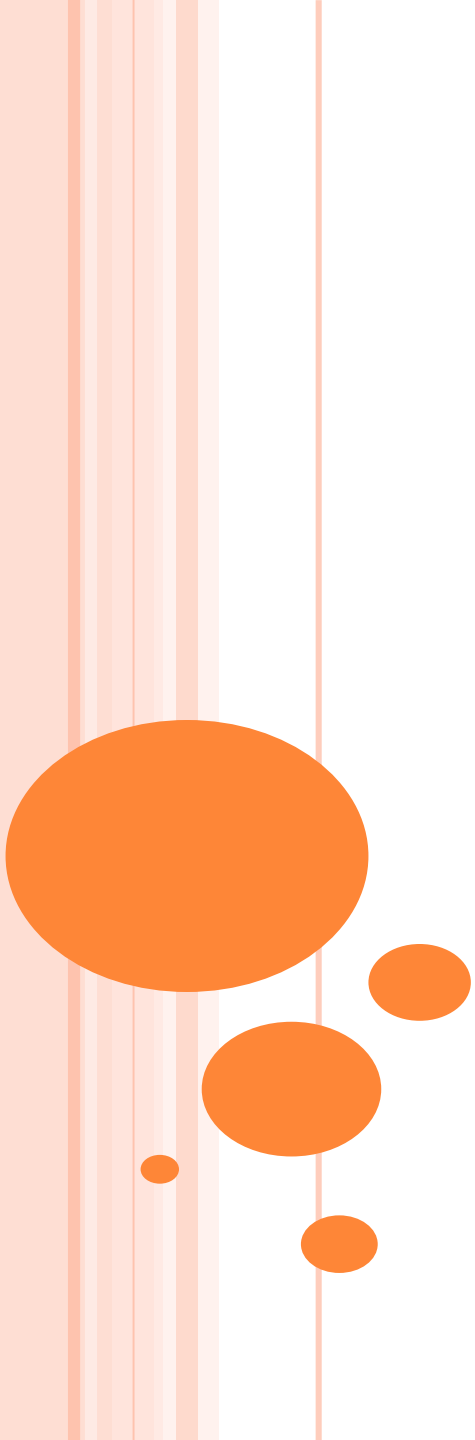




第六章

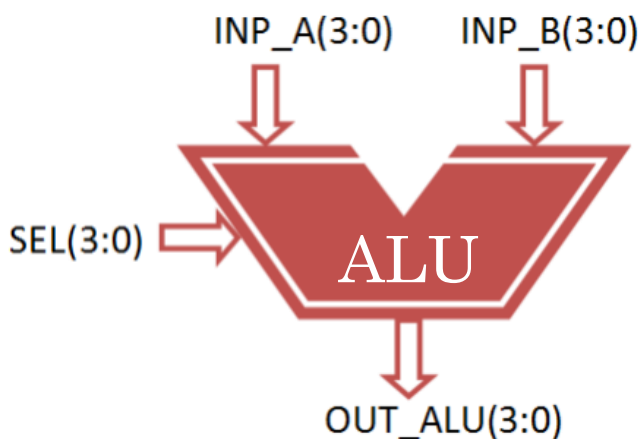
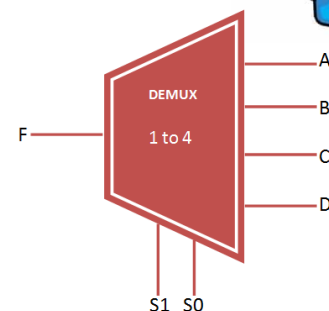
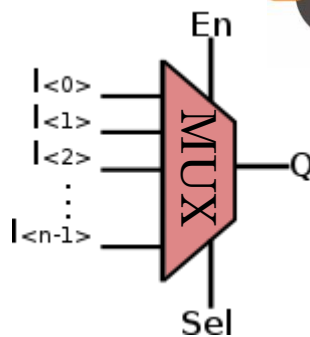
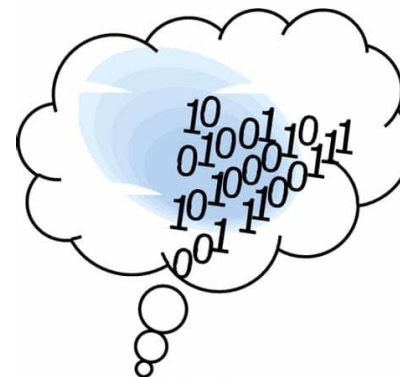
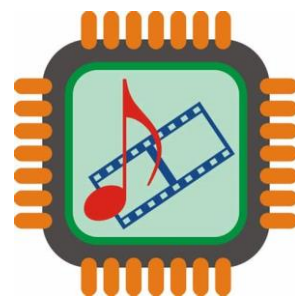
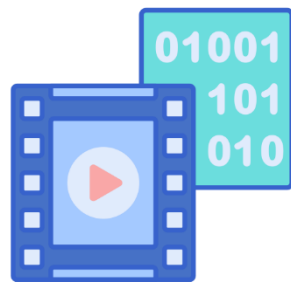
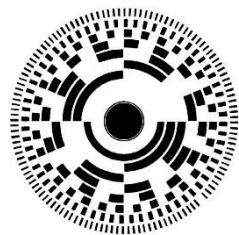
常用的组合逻辑功能器件

周玲

ling.zhou@seu.edu.cn

内容

- 6.1 编码器
- 6.2 译码器
- 6.3 数据选择器
- 6.4 数据分配器
- 6.5 数值比较器
- 6.6 奇偶校验位产生与校验电路
- 6.7 算术运算电路
- 6.8 算术逻辑单元ALU
- 本章小结



集成电路 (1)

Integrated Circuits

16

- SSI – Small-Scale Integration ($0-10^2$)---1960
- MSI – Medium-Scale Integration (10^2-10^3)---1967
- LSI – Large-Scale Integration (10^3-10^5)---1972
- VLSI – Very Large-Scale Integration (10^5-10^7)---1978
- ULSI – Ultra Large-Scale Integration ($\geq 10^7$)---1989
- GSI _ Giant Scale Integration ($\geq 10^9$)---2000

Some definition, < 20 gates

8-bit, 16-bit microprocessors

Pentium microprocessors (1993)

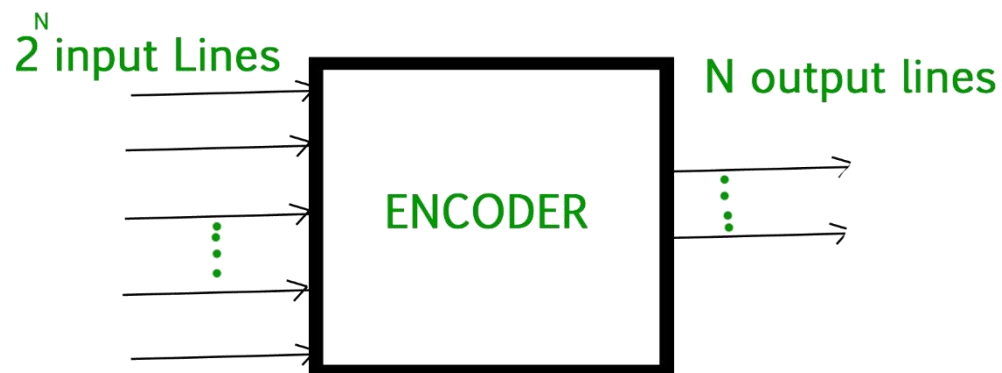


集成电路 (2)

Technology	Number of gates/transistors* per chip	Examples	Year
SSI	1 to 20	74XX series, 4xxx series	60's
MSI	100 to 1000	74XXX series, 45XX series	70's
LSI	1000 to 10,000/100 to 100,000*	8085,	80's
VLSI	10,000 to 100,000/1,000,000 *	CPLD, FPGA, advanced μ C, SoC	90's

- 有些组合逻辑电路是经常使用的，因此把这些组合逻辑电路集成**中小规模逻辑部件**，以方便使用。
- 常用的组合逻辑功能部件在本单元介绍。

编码器 (ENCODER)



- **编码**：在数字系统中，为了处理信号方便，将某个信号指定一组**代码**。
 - 在二值逻辑电路中，信号都是以**高、低电平**的形式给出。
- **编码功能**：**将输入**的信号**转换成**对应的**二进制数码**，用代码来代表相应的输入信号。
- **编码器**：具有编码功能的逻辑电路。
 - 具有**多个输入**和**多个输出**
 - 通常在一个时刻只有一个输入信号会被**转换**为二进制**代码**。

4线-2线编码器 (4-2 ENCODER)

- 对信号高电平有效的编码器
 - 即输入信号为高电平时进行编码
 - 多个输入信号中, 只能有一个信号为高电平。 I_0
- 设定一个输出端 F_{GS} (Group Select)
 - 有输入信号时, 表示编码器正常工作, $F_{GS} = 1$
 - 没有输入信号时, $F_{GS} = 0$

功能表	Inputs				Outputs	
	I_3	I_2	I_1	I_0	Y_1	Y_0
	0	0	0	1	0	0
	0	0	1	0	0	1
	0	1	0	0	1	0
	1	0	0	0	1	1

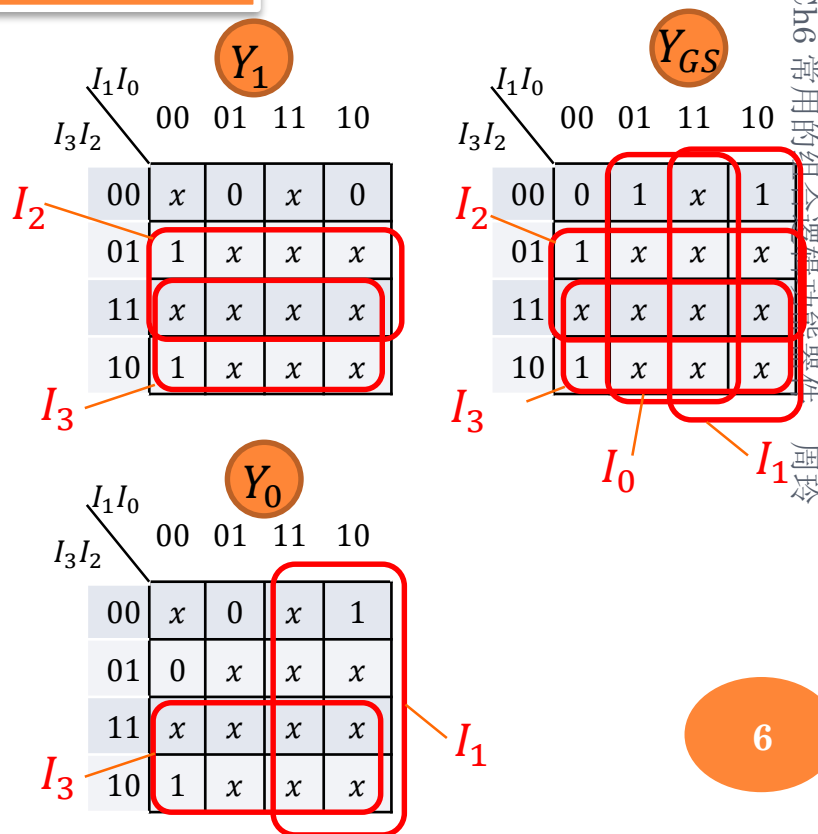
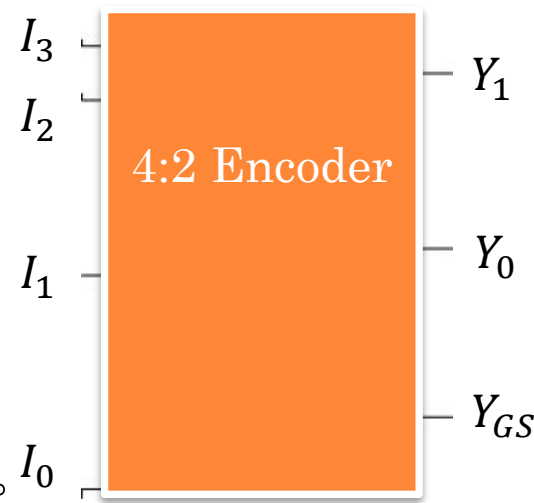
从左到右按
优先级
由高到低的
位置顺序排列

根据功能表画卡诺图
获得逻辑函数表达式

$$Y_1 = I_3 + I_2$$

$$Y_0 = I_3 + I_1$$

$$Y_{GS} = I_3 + I_2 + I_1 + I_0$$



优先编码器 (PRIORITY ENCODER)

问题：前面的4-2编码器只允许一个输入信号有效。

- 需要扩展应用，可以允许多个输入信号有效。
- 限制：**一个时刻只能对其中的一个输入信号编码。
- 优先编码器：**对输入信号设定优先级，在某一时刻只对优先级最高的输入信号编码
- 优先级：** $I_3 > I_2 > I_1 > I_0$

4-2优先编码器功能表

Inputs				Outputs		
I_3	I_2	I_1	I_0	Y_1	Y_0	Y_{GS}
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

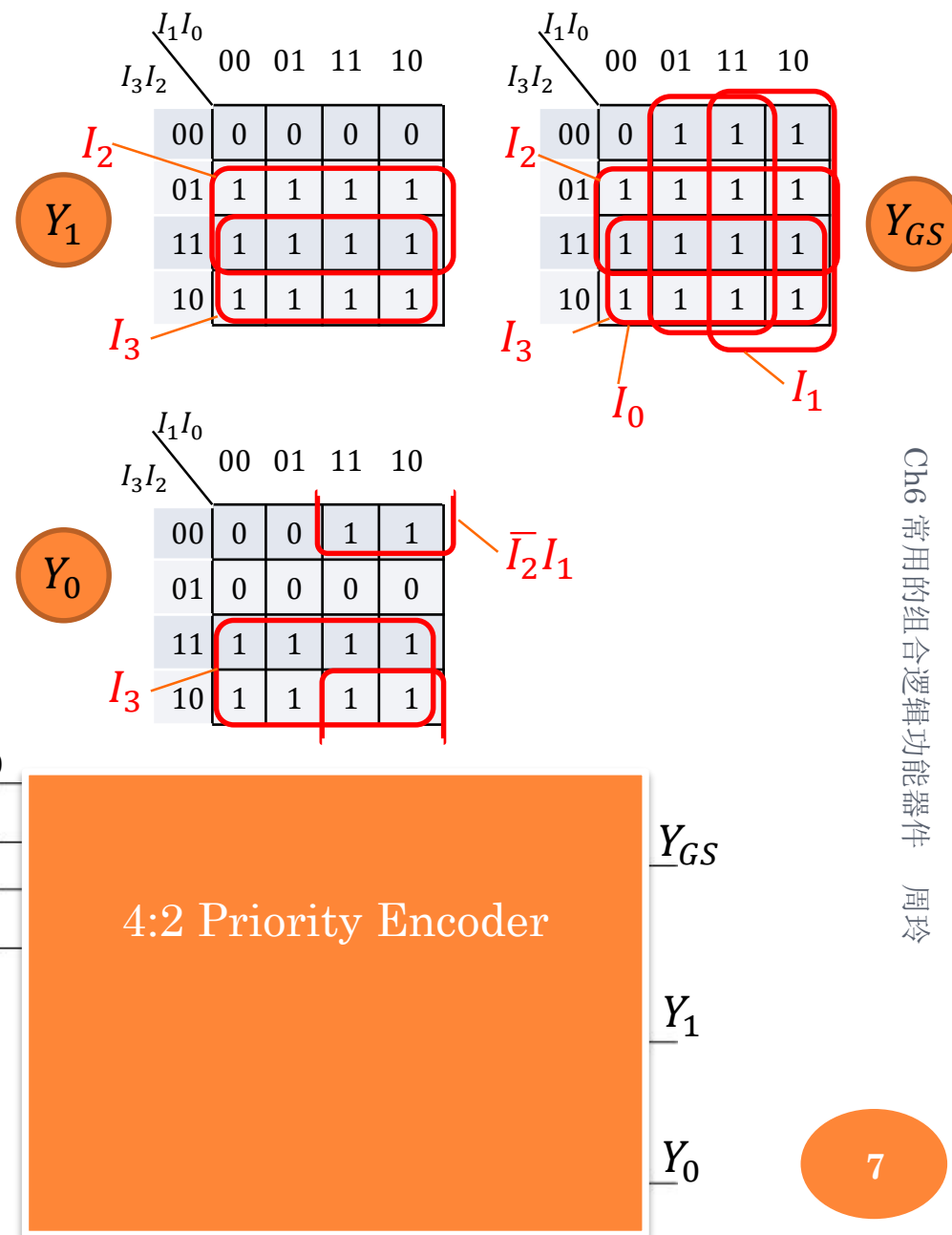
根据功能表画卡诺图
获得逻辑函数表达式

$$Y_1 = I_3 + \bar{I}_3 I_2 = I_3 + I_2$$

$$Y_0 = I_3 + \bar{I}_2 I_1$$

非最简式

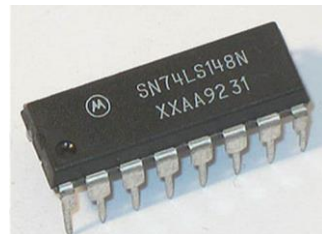
$$Y_{GS} = I_3 + I_2 + I_1 + I_0$$



集成编码器 (PRIORITY ENCODER)

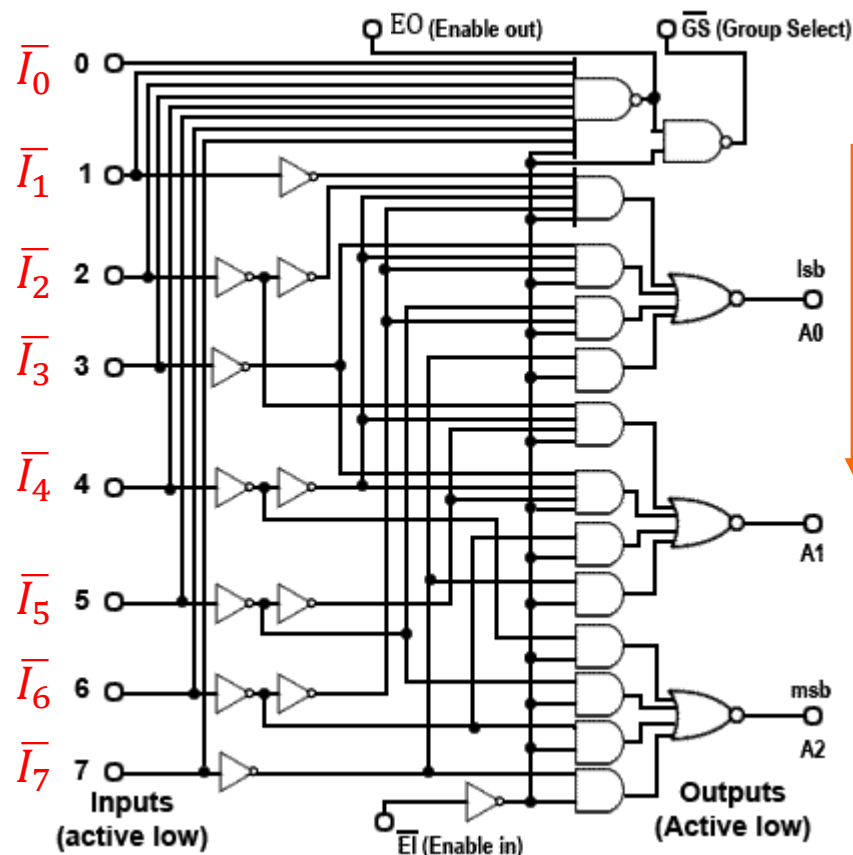
- 常用的集成优先编码器有

- 8线-3线优先编码器74148
- 10线-4线优先编码器74147



- 输入输出有效逻辑电平都是低电平 (Active low)
- 输入中，标号越大，优先级越高。
 - 优先级别 $\bar{I}_7 > \bar{I}_6 > \bar{I}_5 > \bar{I}_4 > \bar{I}_3 > \bar{I}_2 > \bar{I}_1 > \bar{I}_0$
 - 优先编码器允许同时有多个输入信号有效，但是仅仅会对优先级最高的信号进行编码。
- 在逻辑图中，输入使能端 ($\bar{EI} = Low$) 低电平有效，编码器才能工作；否则 $\bar{EI} = High$, 该编码器不工作。
- EO (Enable Output) 为选通输出端 (书上标为 Y_S)
 - $EO = 0$, 输出不代表编码
 - $EO = 1$, 三个输出是有效的编码信号
- $\overline{GS}$ 为扩展端 (书上标为 $\overline{Y_{EX}}$)，可以用来扩展编码器。

8线-3线优先编码器74148



1148

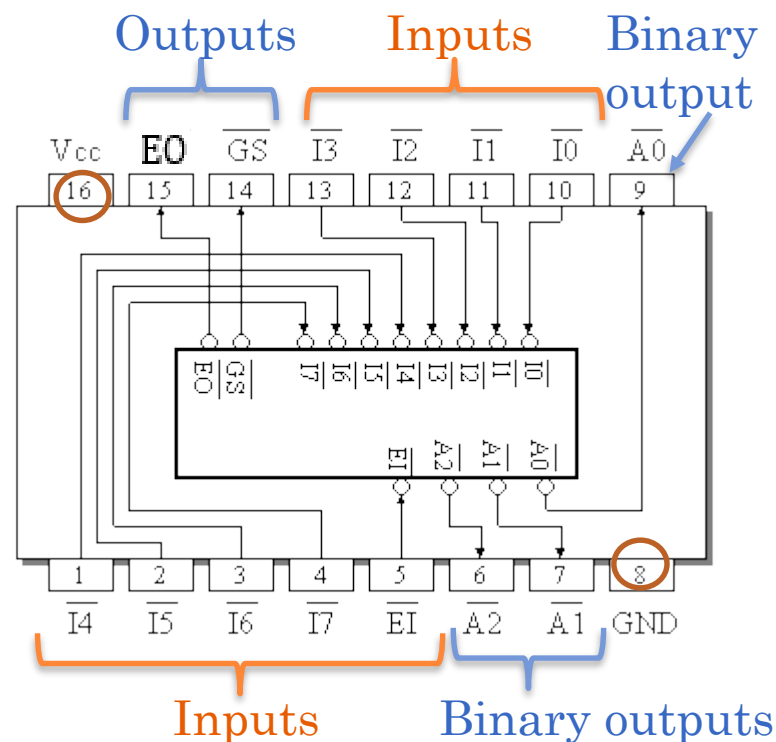
MSB

Inputs									Outputs				
$\overline{EI}$	$\overline{I}_0$	$\overline{I}_1$	$\overline{I}_2$	$\overline{I}_3$	$\overline{I}_4$	$\overline{I}_5$	$\overline{I}_6$	$\overline{I}_7$	$\overline{GS}$	$\overline{A}_0$	$\overline{A}_1$	$\overline{A}_2$	EO
H	X	X	X	X	X	X	X	X	H	H	H	H	H
L	H	H	H	H	H	H	H	H	H	H	H	H	L
L	X	X	X	X	X	X	X	L	L	L	L	L	H
L	X	X	X	X	X	X	L	H	L	H	L	L	H
L	X	X	X	X	X	L	H	H	L	L	H	L	H
L	X	X	X	X	L	H	H	H	L	H	H	L	H
L	X	X	X	L	H	H	H	H	L	L	L	H	H
L	X	X	L	H	H	H	H	H	L	H	L	H	H
L	X	L	H	H	H	H	H	H	L	L	H	H	H
L	L	H	H	H	H	H	H	H	L	H	H	H	H

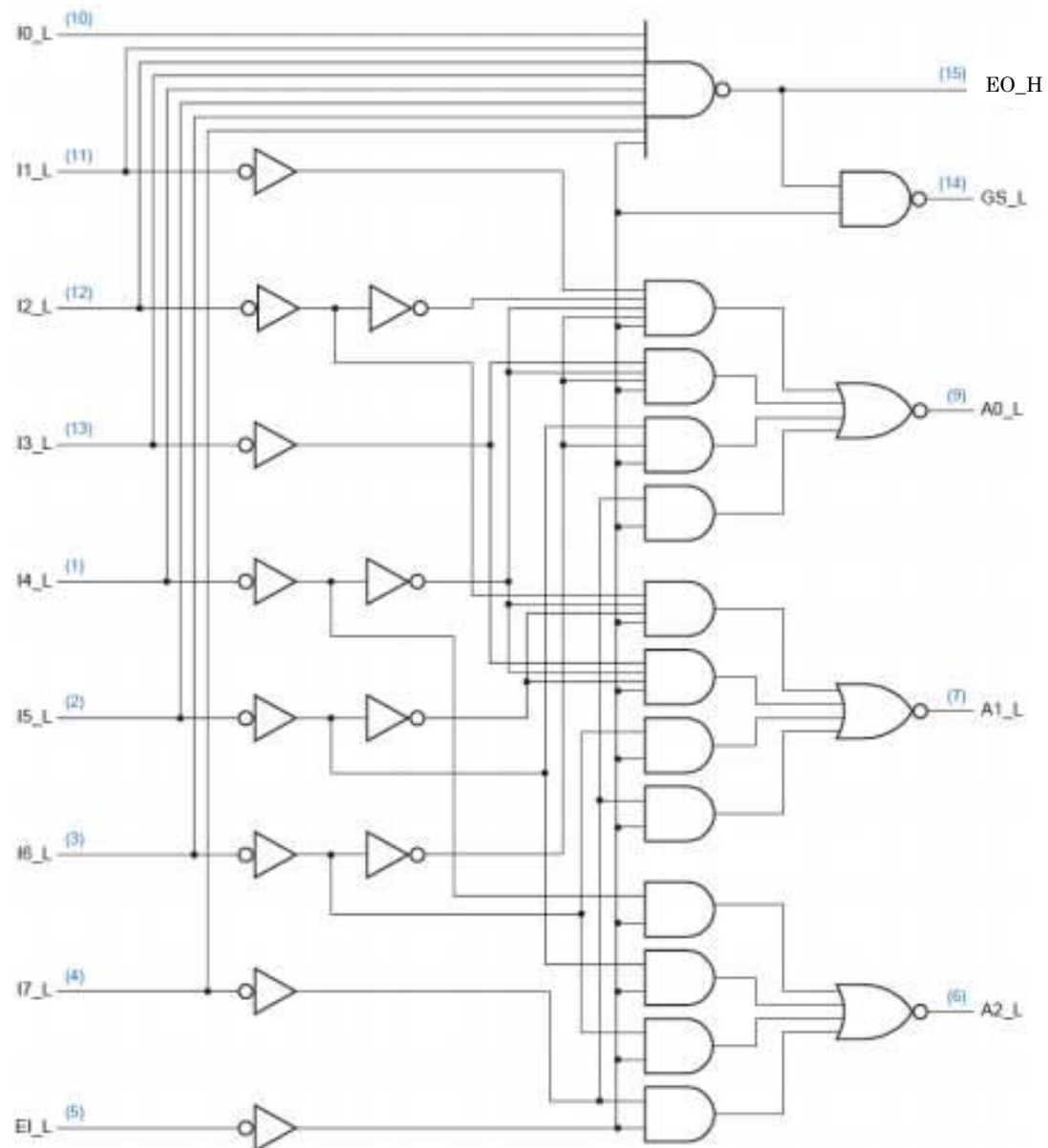
反码
形式

I_7 111
 I_6 110
 I_5 101
 I_4 100
 I_3 011
 I_2 010
 I_1 001
 I_0 000

$$\begin{aligned}\overline{A_2} &= \overline{\overline{I_4} + \overline{I_5} + \overline{I_6} + \overline{I_7}} \\ \overline{A_1} &= \overline{\overline{I_2} \overline{I_4} \overline{I_5} + \overline{I_3} \overline{I_4} \overline{I_5} + \overline{I_6} + \overline{I_7}} \\ \overline{A_0} &= \overline{\overline{I_1} \overline{I_2} \overline{I_4} \overline{I_6} + \overline{I_3} \overline{I_4} \overline{I_6} + \overline{I_5} \overline{I_6} + \overline{I_7}} \\ EO &= \overline{\overline{I_0} \overline{I_1} \overline{I_2} \overline{I_3} \overline{I_4} \overline{I_5} \overline{I_6} \overline{I_7}} \\ \overline{GS} &= \overline{EO \cdot \overline{EI}} = \overline{\overline{I_0} \overline{I_1} \overline{I_2} \overline{I_3} \overline{I_4} \overline{I_5} \overline{I_6} \overline{I_7}}\end{aligned}$$



8线-3线 优先编码器74148



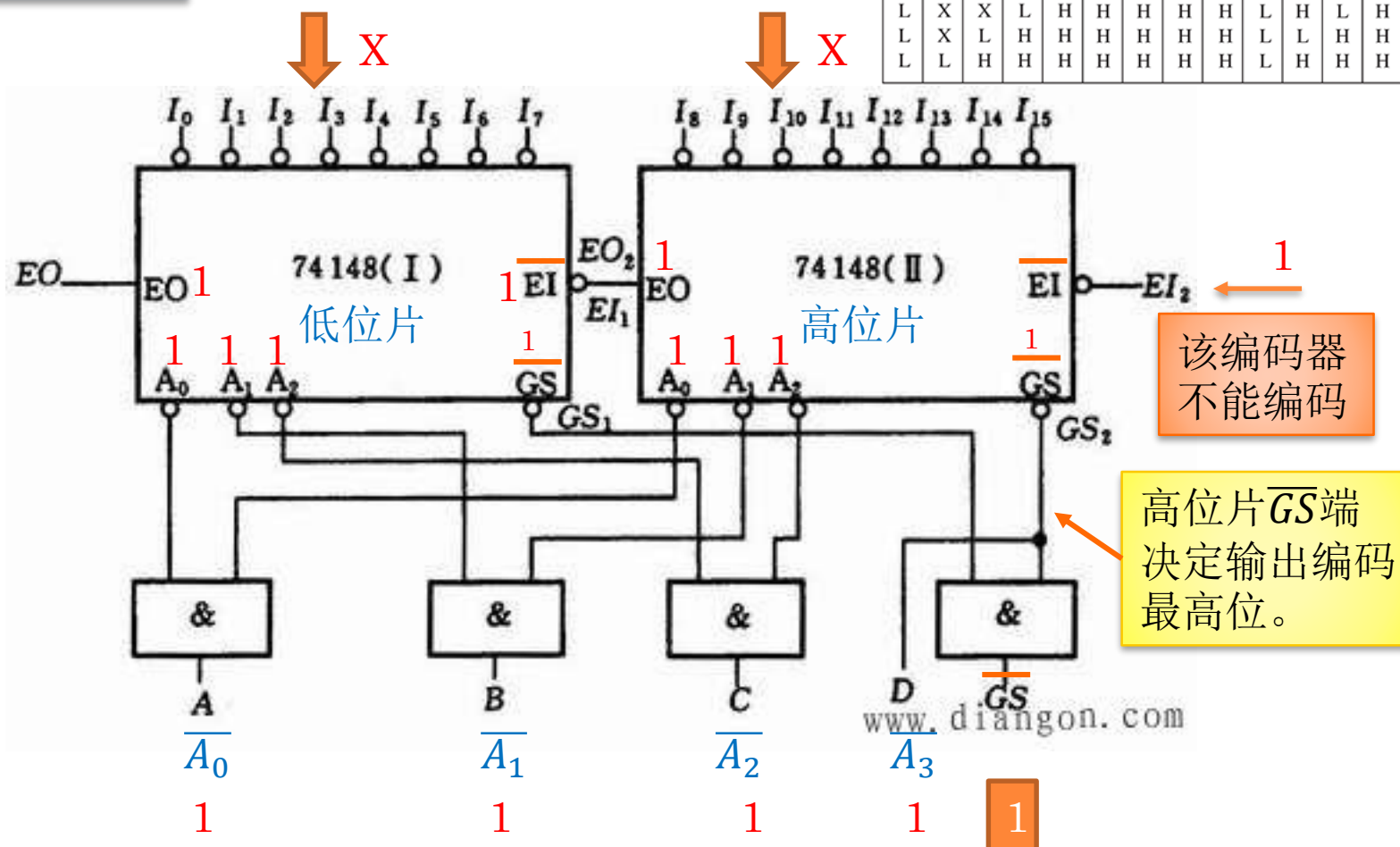
两片74148构成16线-4线优先编码器

Inputs										Outputs			
$\overline{EI}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{GS}$	$\overline{A_0}$	$\overline{A_1}$	$\overline{A_2}$	EO
H	X	X	X	X	X	X	X	X	H	H	H	H	H
L	H	H	H	H	H	H	H	L	H	L	L	L	L
L	X	X	X	X	X	X	X	L	L	H	L	L	H
L	X	X	X	X	X	L	H	H	L	L	H	L	H
L	X	X	X	L	H	H	H	H	L	L	L	H	H
L	X	X	L	H	H	H	H	H	L	H	L	H	H
L	X	L	H	H	H	H	H	H	L	H	L	H	H
L	L	H	H	H	H	H	H	H	L	L	H	H	H

输入端圆圈表示低电平输入有效
输出端圆圈表示低电平输出有效

注意连接方式：

- 1、高位片的 EO 端连接低位片的 $\overline{EI}$ 端；
- 2、两片的相应输出端通过与门相连输出对应编码位；
- 3、最高位 $\overline{A_3}$ 从高位片的 $\overline{GS}$ 直接引出。

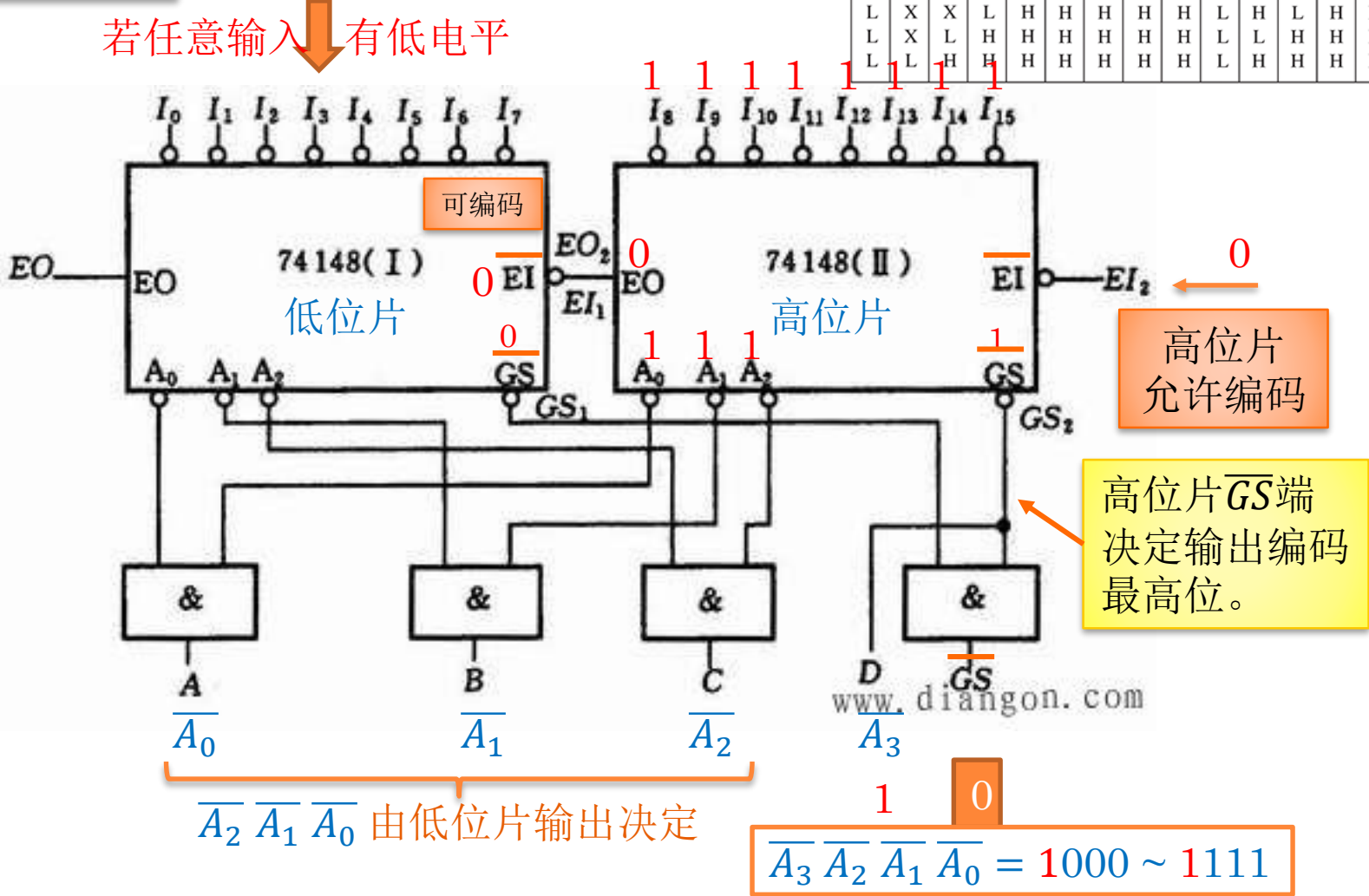


两片74148构成16线-4线优先编码器

Inputs										Outputs			
$\overline{EI}$	$\overline{I_0}$	$\overline{I_1}$	$\overline{I_2}$	$\overline{I_3}$	$\overline{I_4}$	$\overline{I_5}$	$\overline{I_6}$	$\overline{I_7}$	$\overline{GS}$	$\overline{A_0}$	$\overline{A_1}$	$\overline{A_2}$	EO
H	X	X	X	X	X	X	X	X	H	H	H	H	H
L	H	H	H	H	H	H	H	H	L	L	L	L	L
L	X	X	X	X	X	X	X	L	L	L	L	L	H
L	X	X	X	X	X	L	H	H	L	L	H	L	H
L	X	X	X	L	H	H	H	H	L	L	L	H	H
L	X	X	L	H	H	H	H	H	L	H	L	H	H
L	X	L	H	H	H	H	H	H	L	H	L	H	H
L	L	H	H	H	H	H	H	H	L	L	H	H	H

输入端圆圈表示低电平输入有效
输出端圆圈表示低电平输出有效

注意连接方式：
1、高位片的EO端连接低位片的EI端；
2、两片的相应输出端通过与门相连输出对应编码位；
3、最高位A₃从高位片的GS直接引出。

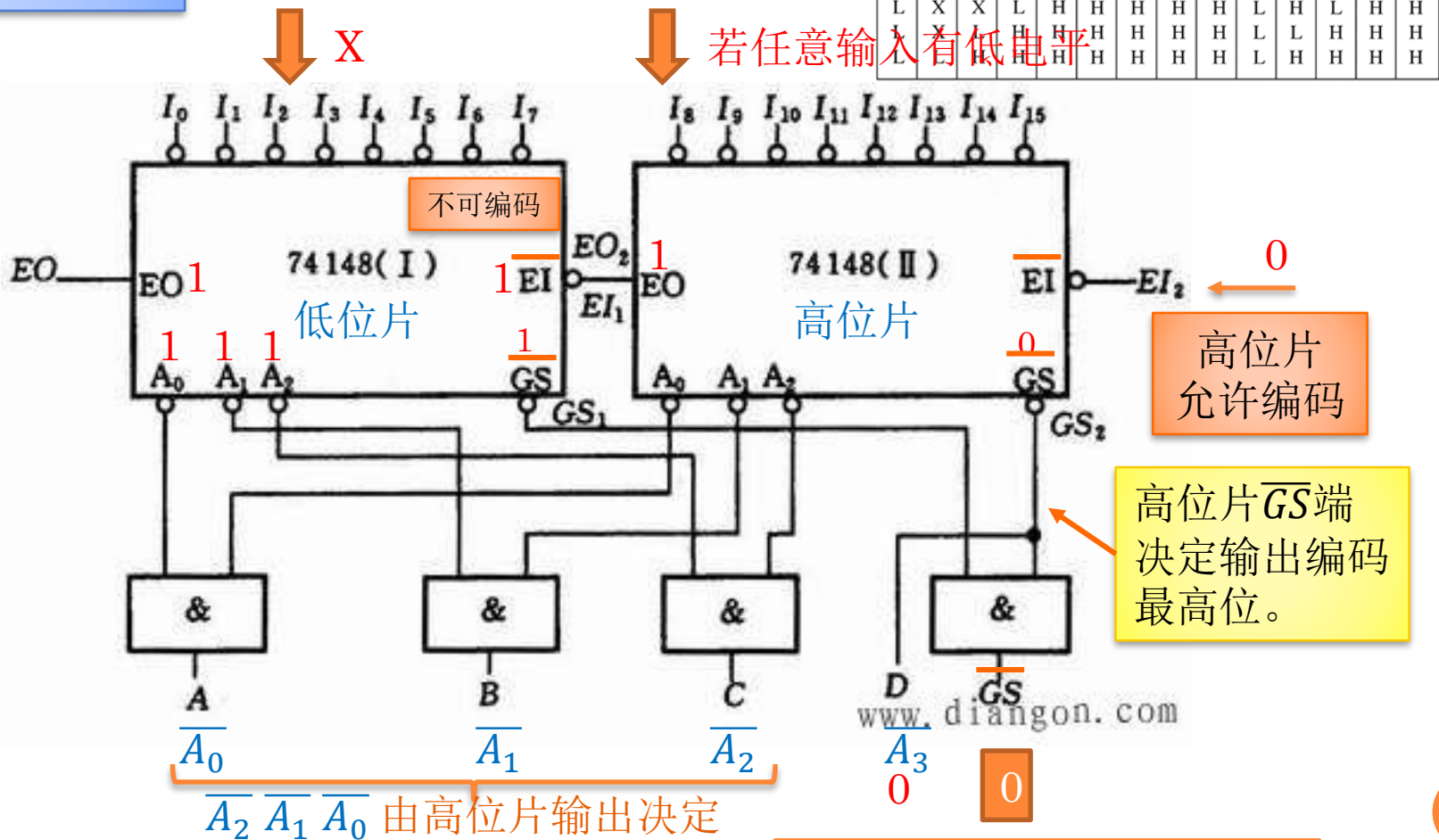


两片74148构成16线-4线优先编码器

输入端圆圈表示低电平输入有效
输出端圆圈表示低电平输出有效

注意连接方式:

- 1、高位片的 $\overline{EO}$ 端连接低位片的 $\overline{EI}$ 端；
- 2、两片的相应输出端通过与门相连输出对应编码位；
- 3、最高位 $\overline{A_3}$ 从高位片的 $\overline{GS}$ 直接引出。



至此整个电路实现了16-4优先编码

$$\overline{A_3} \overline{A_2} \overline{A_1} \overline{A_0} = 0000 \sim 0111$$

10线-4线优先编码器74147

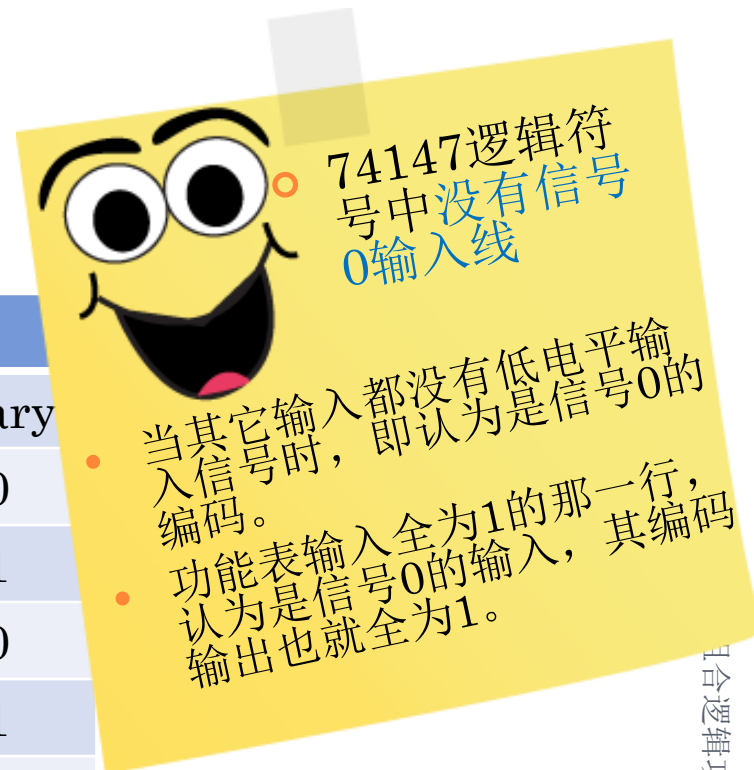
低电平有效

反码形式的
8421BCD码

十-二进制编码器

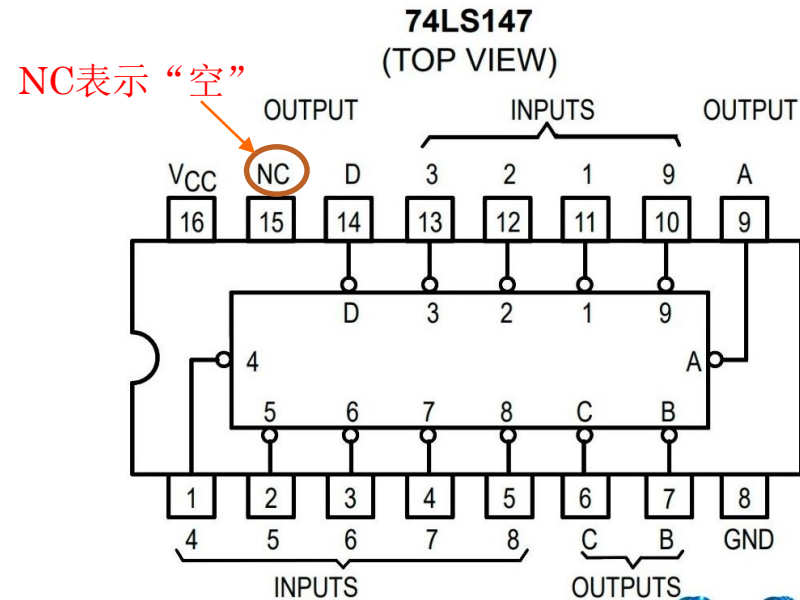
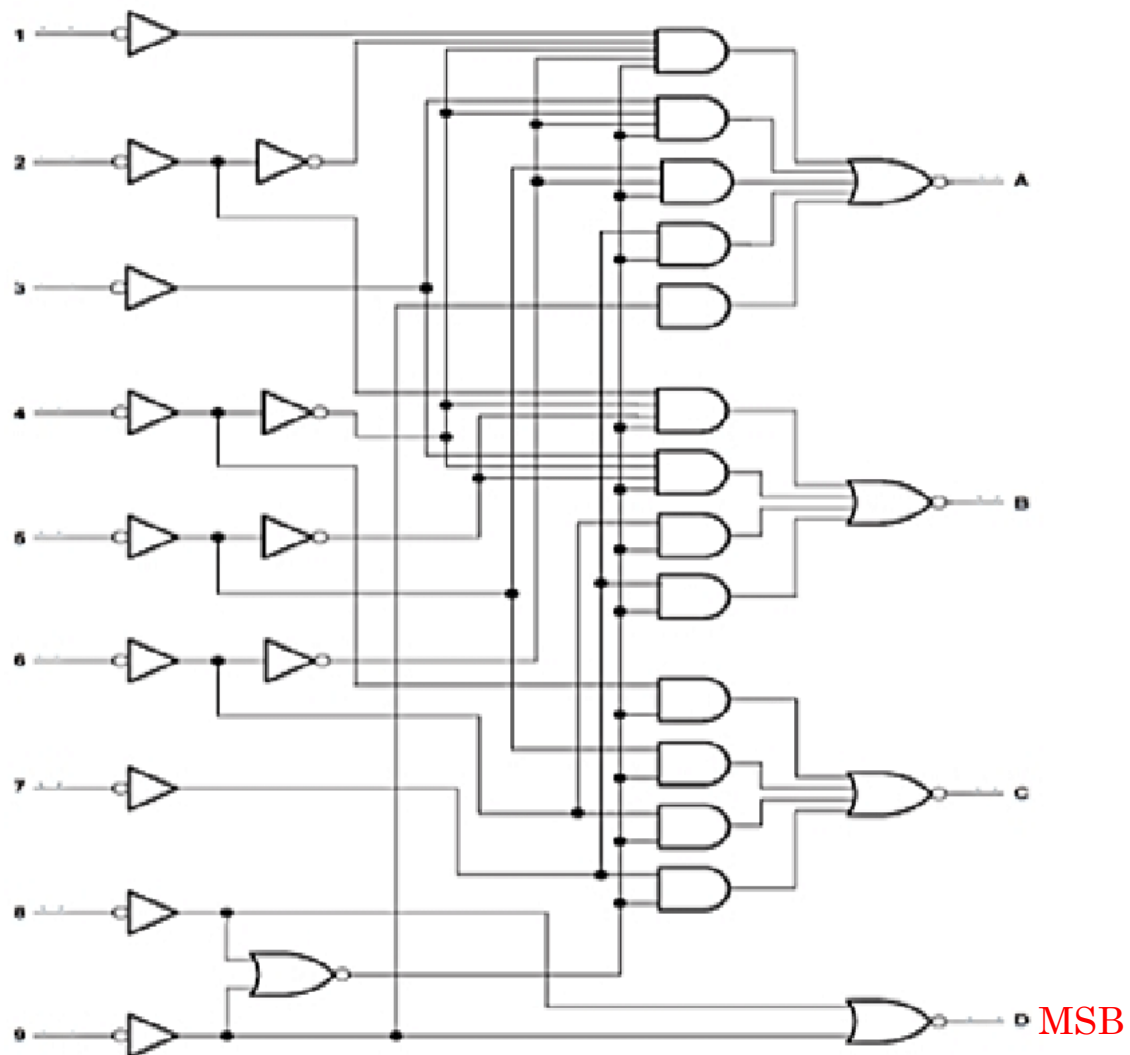
Inputs										Outputs				8421	
N	$\overline{I_9}$	$\overline{I_8}$	$\overline{I_7}$	$\overline{I_6}$	$\overline{I_5}$	$\overline{I_4}$	$\overline{I_3}$	$\overline{I_2}$	$\overline{I_1}$	$\overline{Y_3}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$	N	Binary
0	H	H	H	H	H	H	H	H	H	H	H	H	H	0	0000
1	H	H	H	H	H	H	H	H	L	H	H	H	L	1	0001
2	H	H	H	H	H	H	H	L	X	H	H	L	H	2	0010
3	H	H	H	H	H	H	L	X	X	H	H	L	L	3	0011
4	H	H	H	H	H	L	X	X	X	H	L	H	H	4	0100
5	H	H	H	H	L	X	X	X	X	H	L	H	L	5	0101
6	H	H	H	L	X	X	X	X	X	H	L	L	H	6	0110
7	H	H	L	X	X	X	X	X	X	H	L	L	L	7	0111
8	H	L	X	X	X	X	X	X	X	L	H	H	H	8	1000
9	L	X	X	X	X	X	X	X	X	L	H	H	L	9	1001

H = High logic level, L = Low logic level, X = Don't care

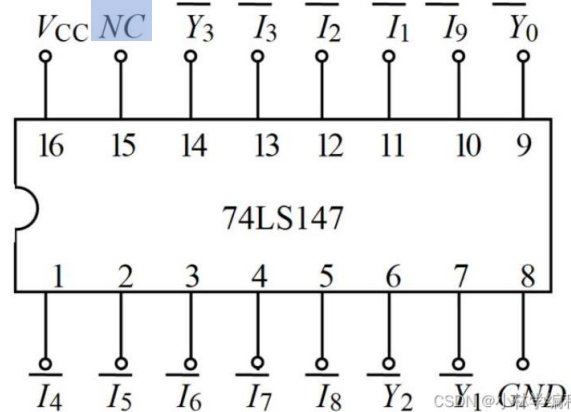


组合逻辑功能器件 周玲

LOGIC CIRCUIT OF 74147



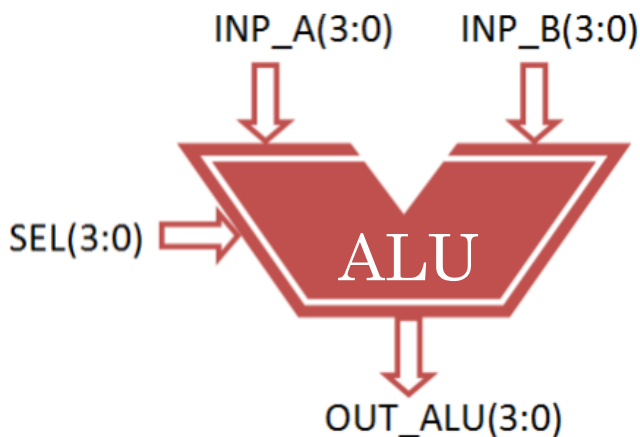
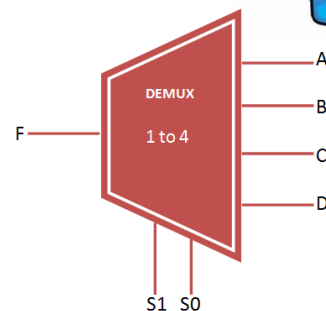
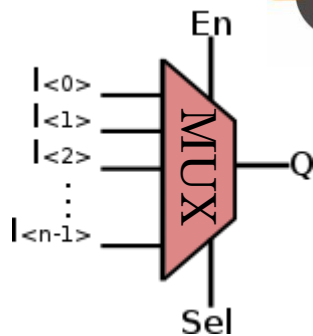
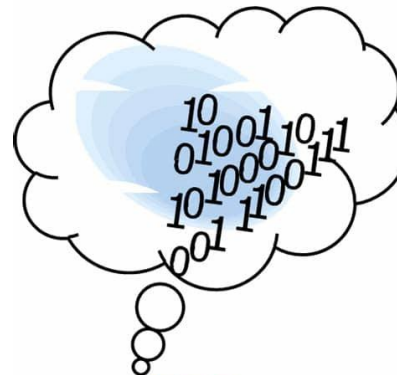
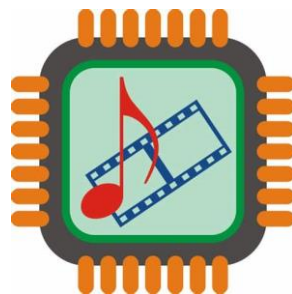
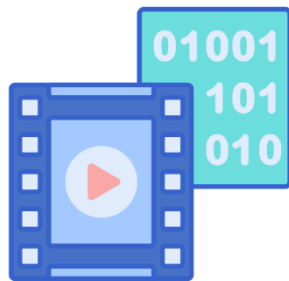
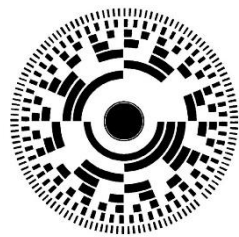
Not Connected /
No Connection



NC是NOT CONNECTED的缩写，即空脚。芯片中NC引脚没有任何用途，只是限于封装形式，该引脚必须存在。

内容

- 6.1 编码器
- 6.2 译码器
- 6.3 数据选择器
- 6.4 数据分配器
- 6.5 数值比较器
- 6.6 奇偶校验位产生与校验电路
- 6.7 算术运算电路
- 6.8 算术逻辑单元ALU
- 本章小结

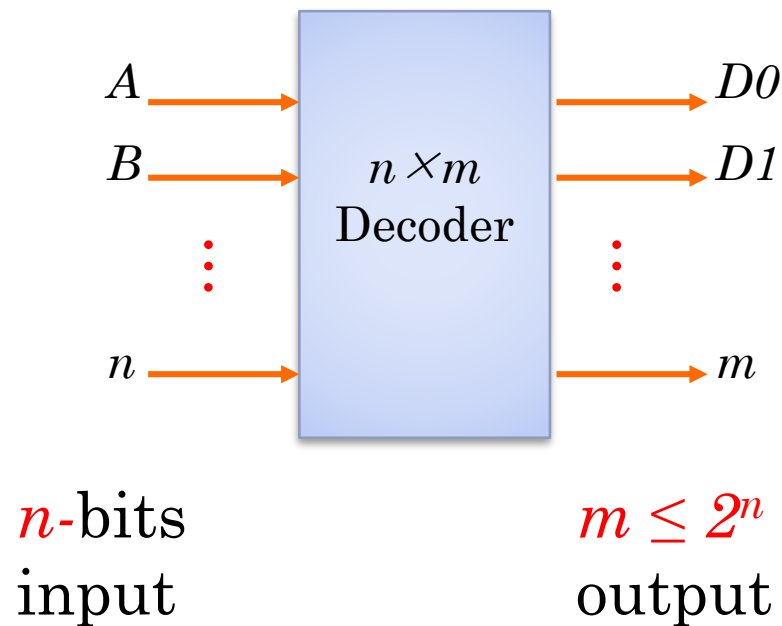


译码器 (DECODER)

- 译码是编码的逆过程
 - 将具有特定含义的二进制编码还原成它原来的信号。
- 译码器是一种多输入多输出的逻辑电路
- 两种类型：
 - 地址译码器：将一系列代码转换成与其一一对应的信号
 - 在计算机中，对存储单元地址译码，以便对存储单元进行读写操作
 - 代码变换器：将一种代码转换成另一种代码。
- 常用译码器：
 - 二进制译码器
 - 二-十进制译码器
 - 显示译码器

二进制译码器

- 将二进制代码转换为对应的输出信号
- 由于 n 位二进制码对 2^n 个信号，所以，这种译码器是有 n 个输入和 2^n 个输出的组合逻辑电路。
 - 对于一个输入代码，输出端有唯一的一个输出端输出有效电平，其余输出端输出无效电平。
- 常用的二进制译码器
 - 2线-4线译码器
 - 3线-8线译码器
 - 4线-16线译码器

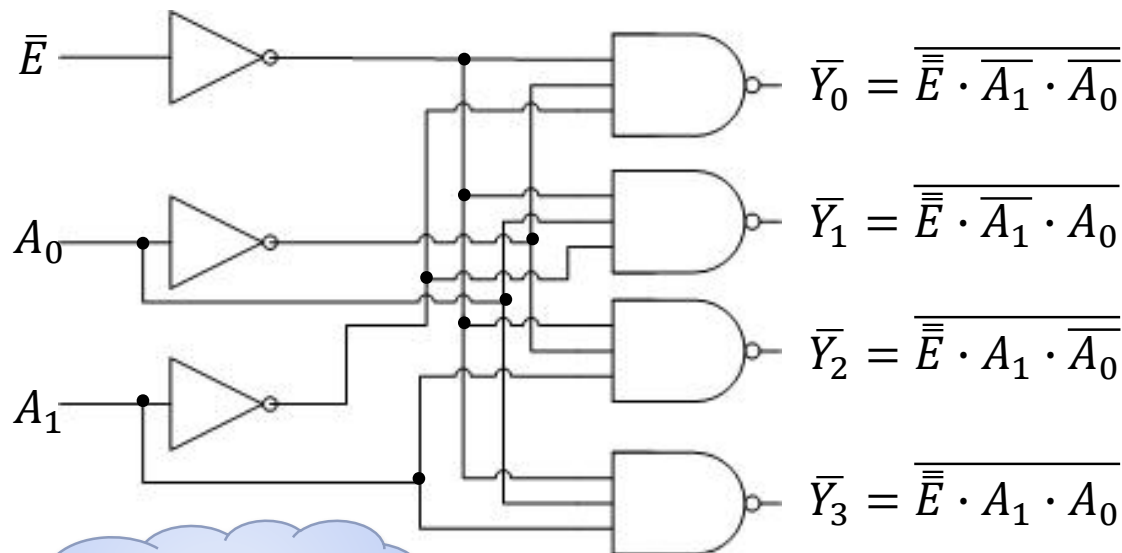
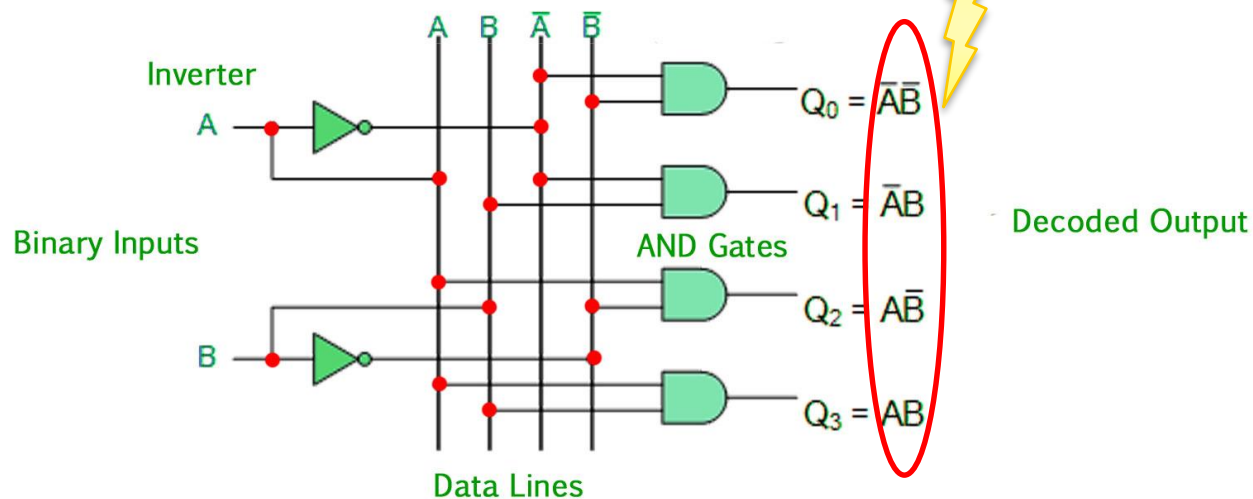


2线-4线译码器

该译码器也称为**最小项发生器**

Important

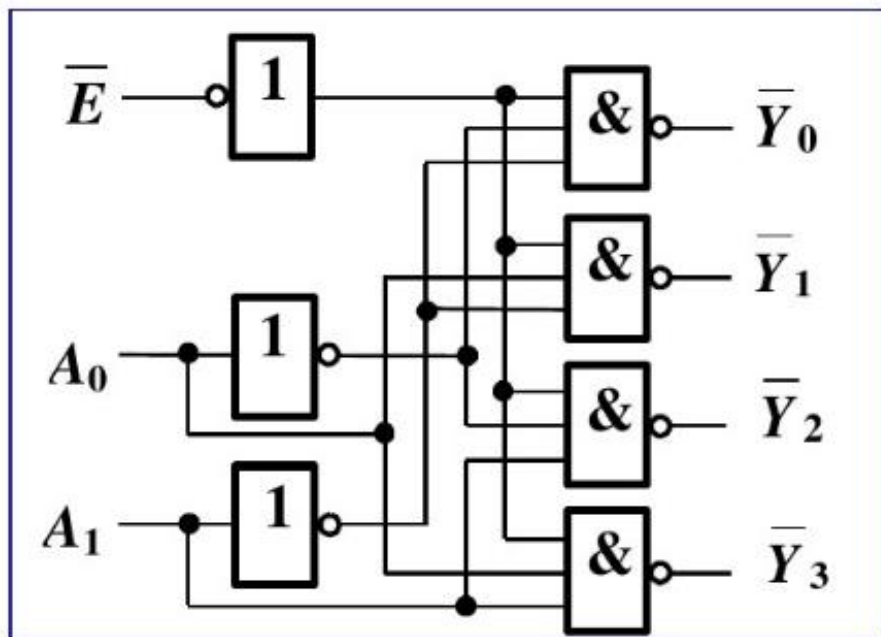
译码器产生所有2个输入变量的**最小项**的非。



$\bar{E} = 1$ 无效输入时，输出端为全1

Input			Output			
$\bar{E}$	A_1	A_0	$\bar{Y}_0$	$\bar{Y}_1$	$\bar{Y}_2$	$\bar{Y}_3$
1	x	x	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

2线 - 4线译码器的逻辑电路(分析)



$$\bar{Y}_0 = \bar{\bar{E}} \bar{A}_1 \bar{A}_0$$

$$\bar{Y}_2 = \bar{\bar{E}} A_1 \bar{A}_0$$

功能表

输入			输出			
$\bar{E}$	A_1	A_0	$\bar{Y}_0$	$\bar{Y}_1$	$\bar{Y}_2$	$\bar{Y}_3$
H	×	×	H	H	H	H
L	L	L	L	H	H	H
L	L	H	H	L	H	H
L	H	L	H	H	L	H
L	H	H	H	H	H	L

$$\bar{Y}_1 = \bar{\bar{E}} \bar{A}_1 A_0$$

$$\bar{Y}_3 = \bar{\bar{E}} A_1 A_0$$

$\bar{E} = 1$ 无效输入时，
输出端为全1

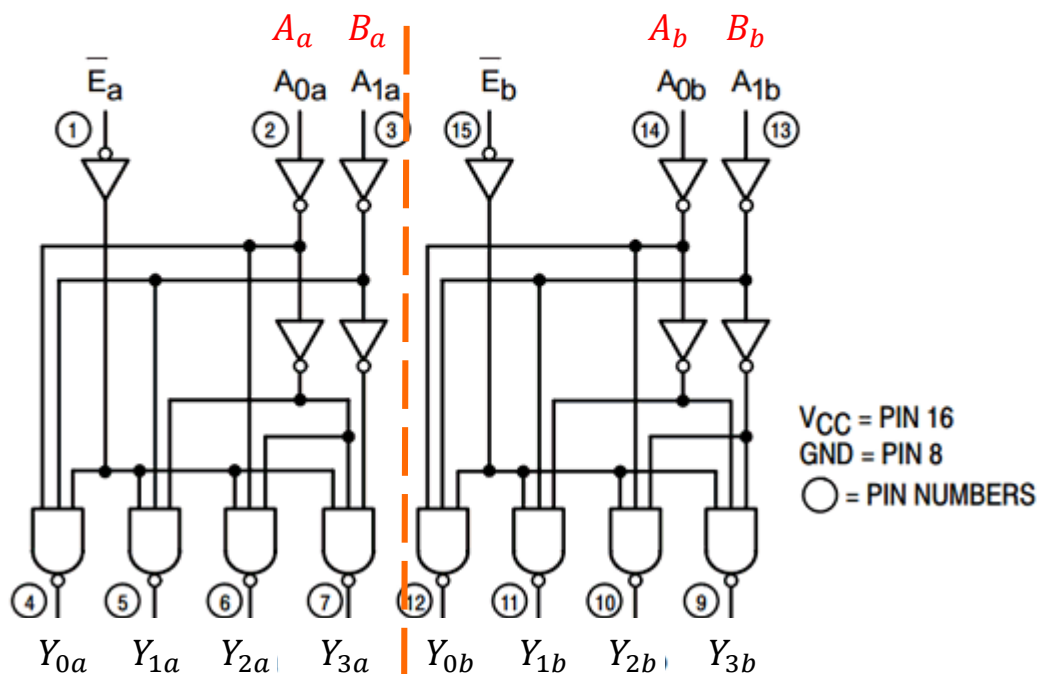
Important

译码器产生所有2个输入变量的最小项的非。

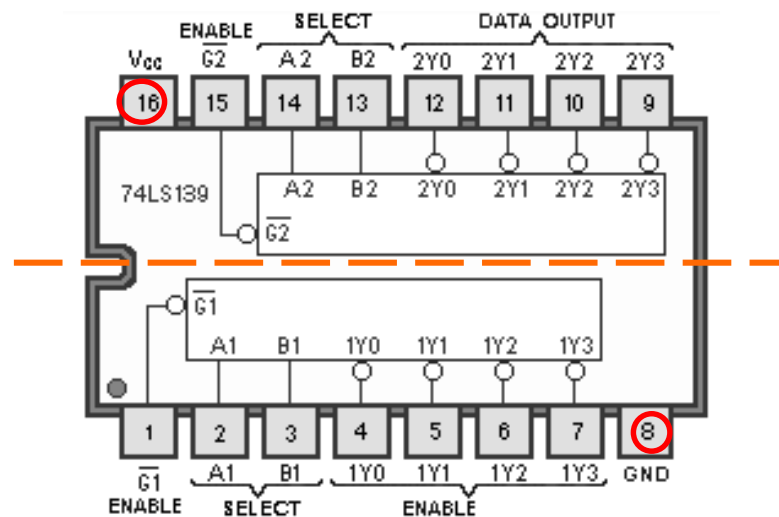
74139: 一个芯片中集成两个2-4线译码器

SN74LS139

LOGIC DIAGRAM



$\bar{G} = 1$ 无效输入时，输出端为全1

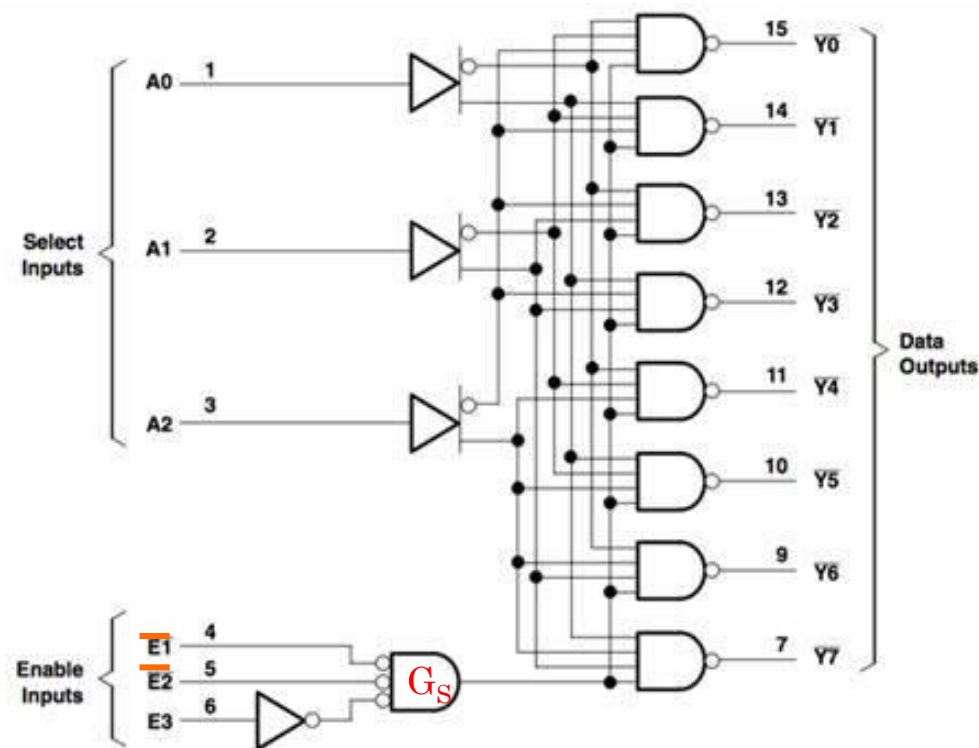


FUNCTION TABLE

INPUTS			OUTPUTS			
$\bar{G}$	SELECT					
	B	A	Y0	Y1	Y2	Y3
H	X	X	H	H	H	H
L	L	L	L	H	H	H
L	L	H	H	L	H	H
L	H	L	H	H	L	H
L	H	H	H	H	H	L

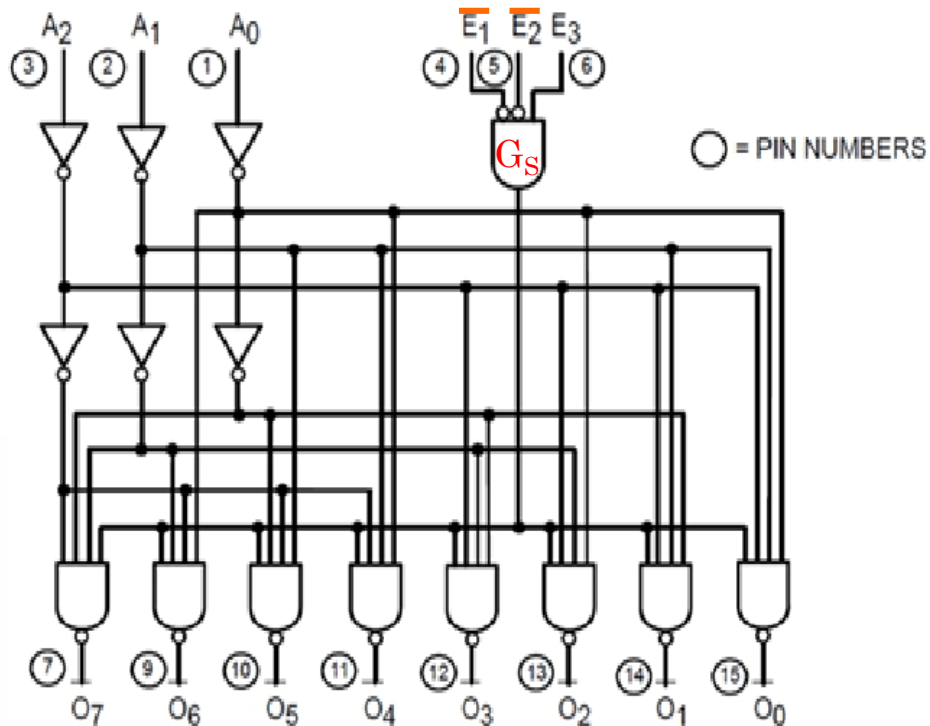
74138为3线-8线译码器

74138有3个附加的控制端E3、 $\overline{E1}$ 、 $\overline{E2}$ 。
当E3=1, $\overline{E1} = \overline{E2} = 0$ 时, G_S 输出为高电平,
译码器工作; 否则, 译码器禁止。



Important

译码器产生所有3个输入变量的最小项的非。



$$\begin{aligned}\overline{Y_0} &= \overline{A_2 \cdot A_1 \cdot A_0} \\ \overline{Y_1} &= \overline{A_2 \cdot A_1 \cdot A_0} \\ \overline{Y_2} &= \overline{A_2 \cdot A_1 \cdot A_0} \\ \overline{Y_3} &= \overline{A_2 \cdot A_1 \cdot A_0} \\ \overline{Y_4} &= \overline{A_2 \cdot A_1 \cdot A_0} \\ \overline{Y_5} &= \overline{A_2 \cdot A_1 \cdot A_0} \\ \overline{Y_6} &= \overline{A_2 \cdot A_1 \cdot A_0} \\ \overline{Y_7} &= \overline{A_2 \cdot A_1 \cdot A_0}\end{aligned}$$

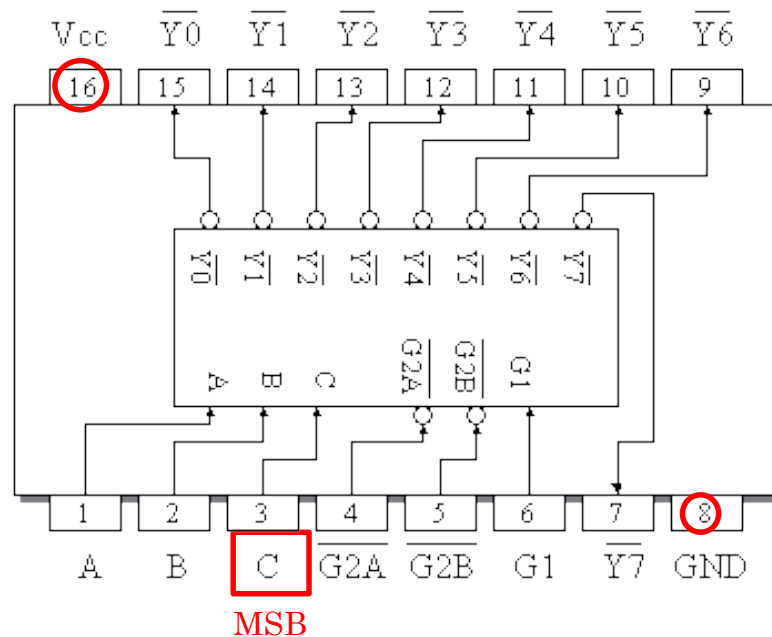
A_2	A_1	A_0
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

74138: 3线-8线译码器

- 根据74138或者74139这种最小项发生器的特点，可以用译码器设计相应的组合逻辑电路。

- 注意：译码器产生的是所有最小项的非。

$G1$, $\overline{G2A}$, $\overline{G2B}$ 为无效输入时，输出端为全1

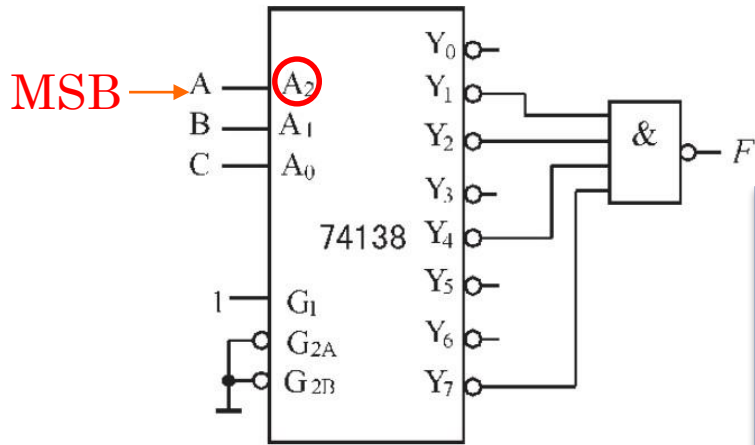


MSB

DECODER TABLE

Inputs						Outputs							
$G1$	$\overline{G2A}$	$\overline{G2B}$	$\overset{\text{MSB}}{\text{C}}$	B	A	$\overline{Y_7}$	$\overline{Y_6}$	$\overline{Y_5}$	$\overline{Y_4}$	$\overline{Y_3}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$
0	x	x	x	x	x	1	1	1	1	1	1	1	1
x	1	x	x	x	x	1	1	1	1	1	1	1	1
x	x	1	x	x	x	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	0	1	1
1	0	0	0	1	0	1	1	1	0	1	1	1	1
1	0	0	0	1	1	1	1	0	1	1	1	1	1
1	0	0	1	0	0	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1

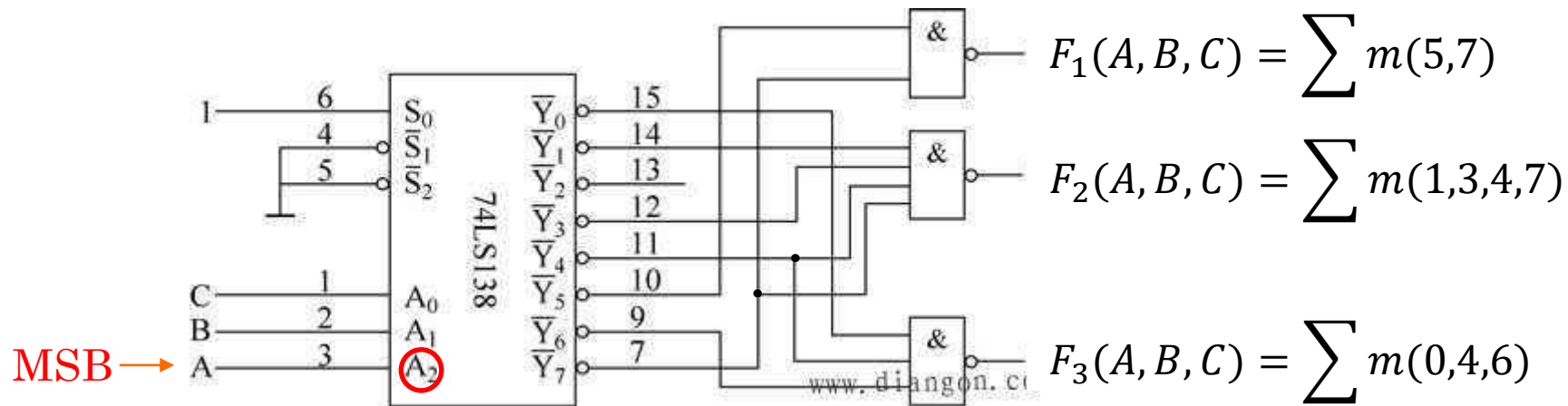
例：用74138实现函数 $F(A, B, C) = \sum m(1, 2, 4, 7)$



$$\begin{aligned} F(A, B, C) &= \overline{m_1} \cdot \overline{m_2} \cdot \overline{m_4} \cdot \overline{m_7} \\ &= m_1 + m_2 + m_4 + m_7 \\ &= \sum m(1, 2, 4, 7) \end{aligned}$$

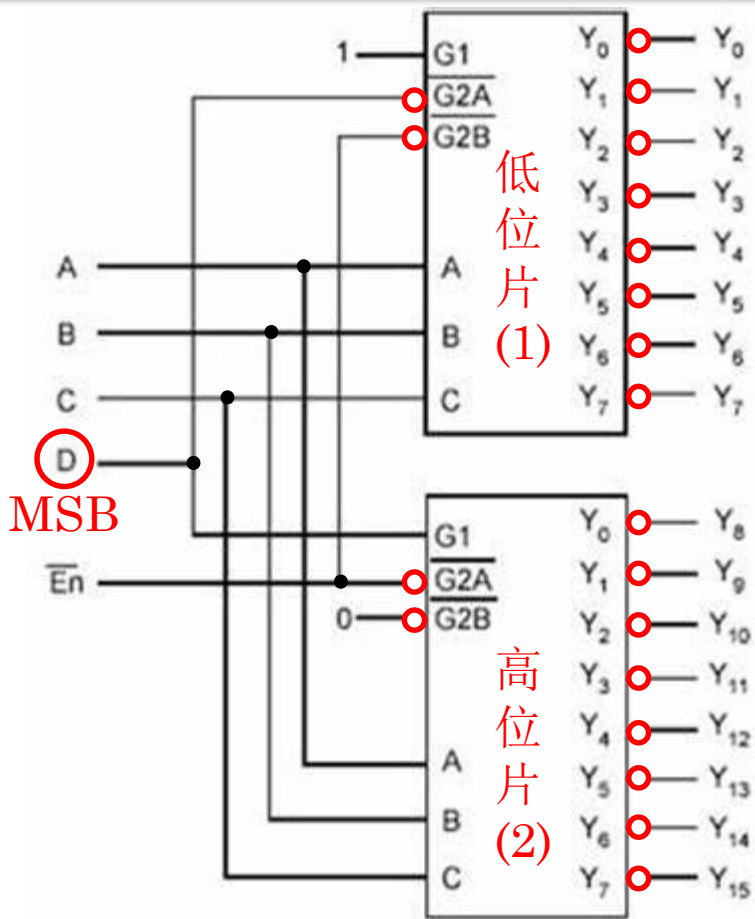
对于多输出的函数

- 由于译码器产生了所有的最小项的非
- 函数增加一个输出，只要再增加一个与非门就可以完成
- 采用译码器实现多输出函数非常方便。

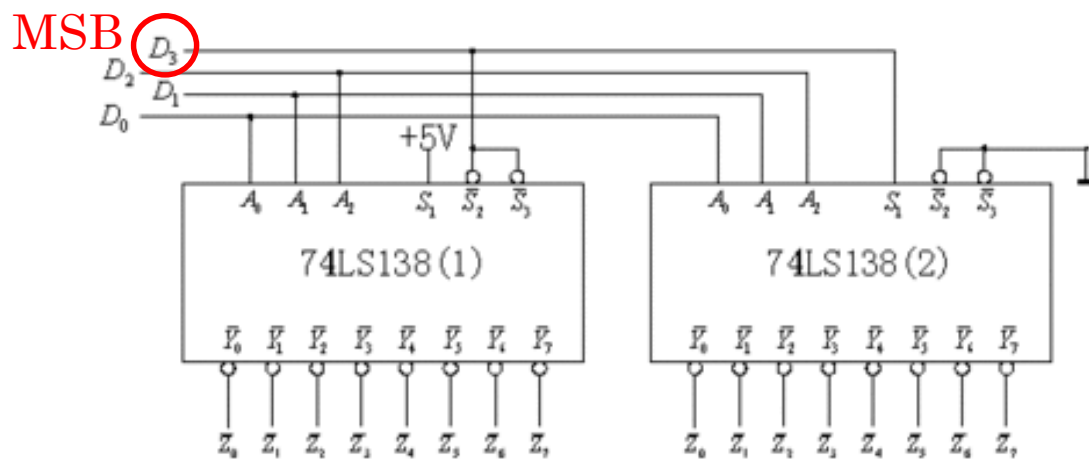
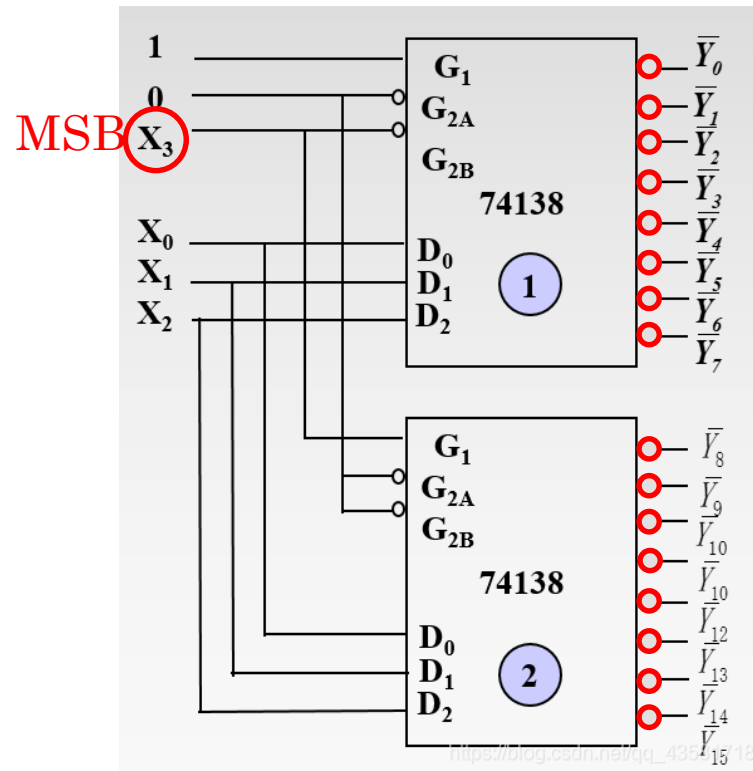


译码器容量的扩展

利用使能端，将多个译码器连接起来。
例如两片74138可以构成一个4线-16线译码器



	X_3	X_2	X_1	X_0
	D	C	B	A
低位片 1	0	0	0	0
	0	0	0	1
	0	0	1	0
	⋮	⋮	⋮	⋮
高位片 2	1	0	0	0
	1	0	0	1
	1	0	1	0
	⋮	⋮	⋮	⋮



译码器容量的扩展

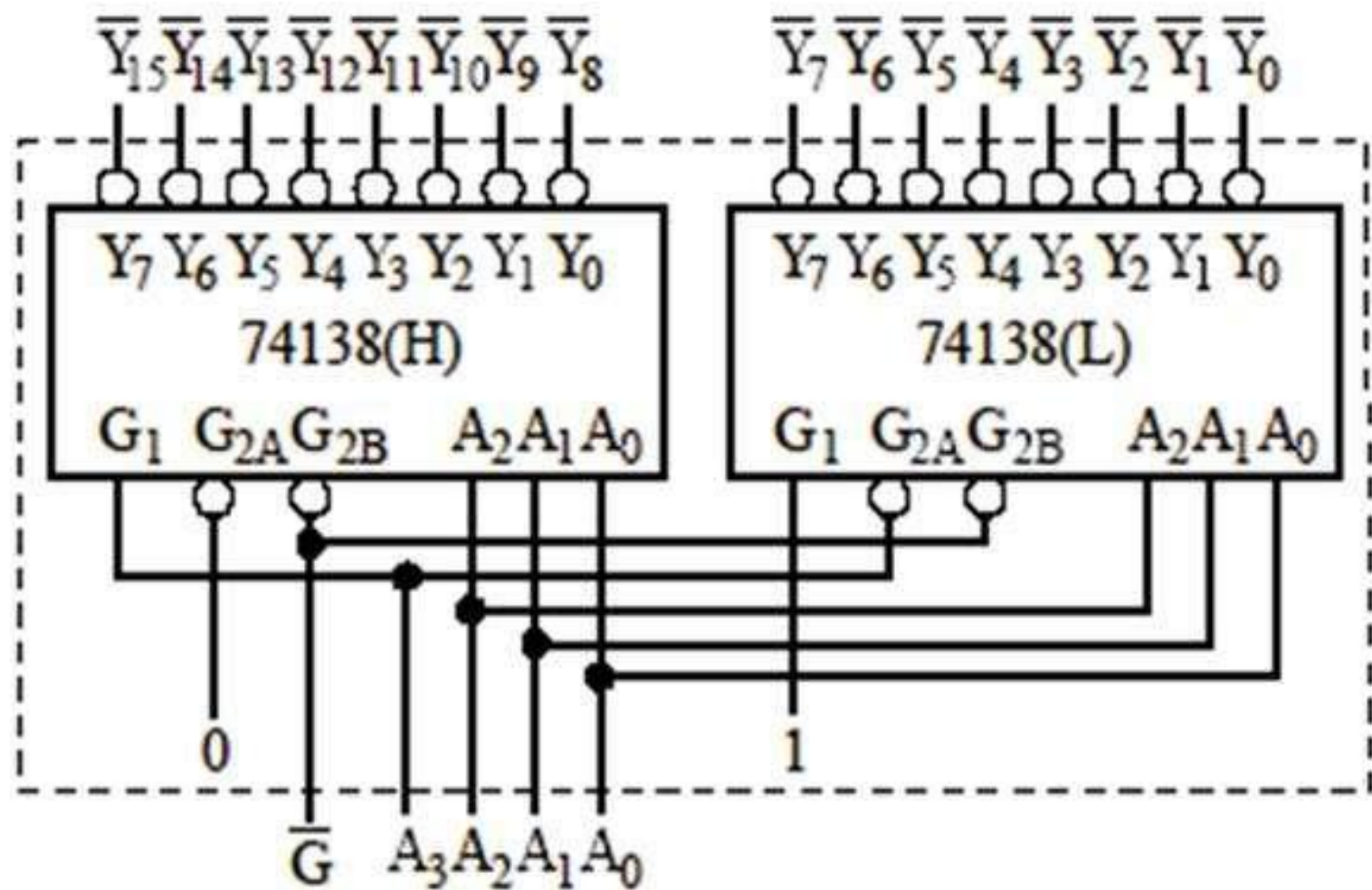
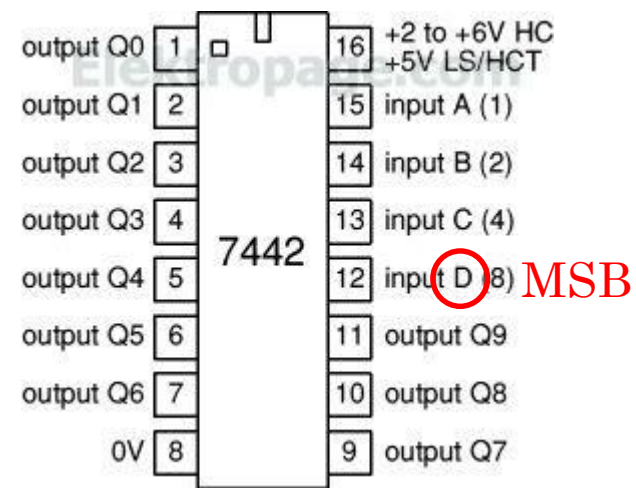
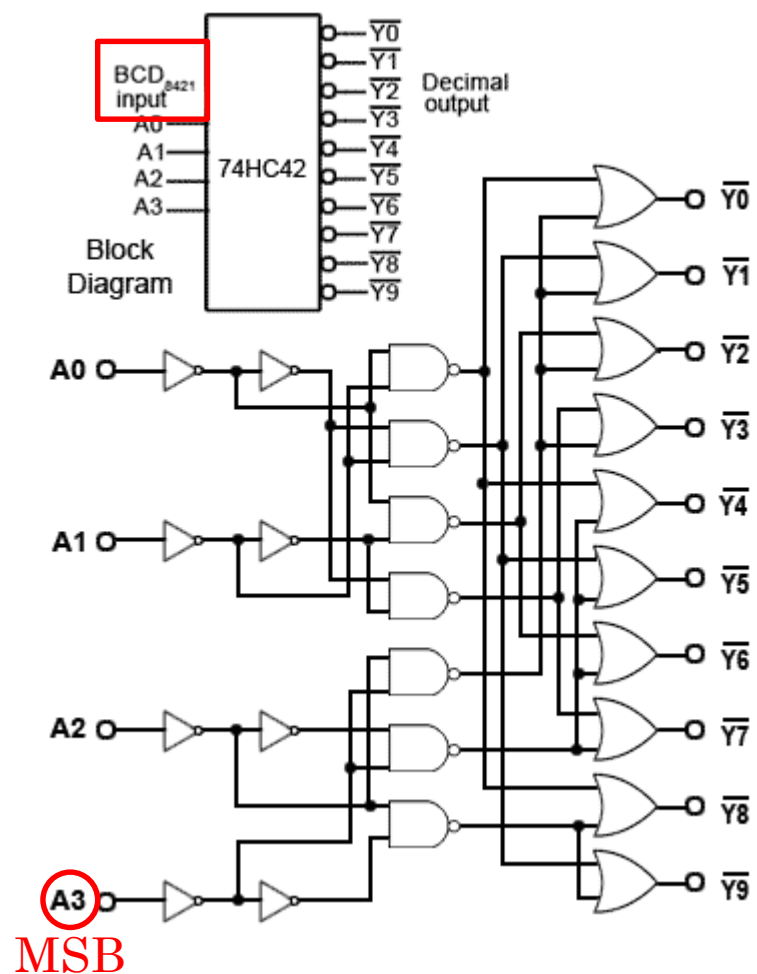
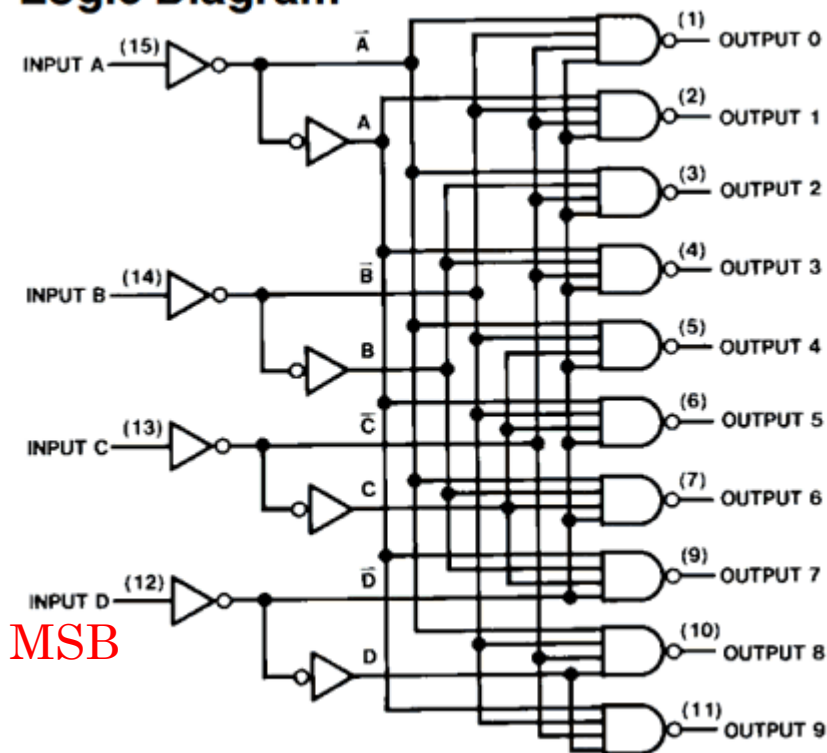


图 2—29 74138 构成的 4 线—16 线译码器

7442 二-十进制译码器



Logic Diagram



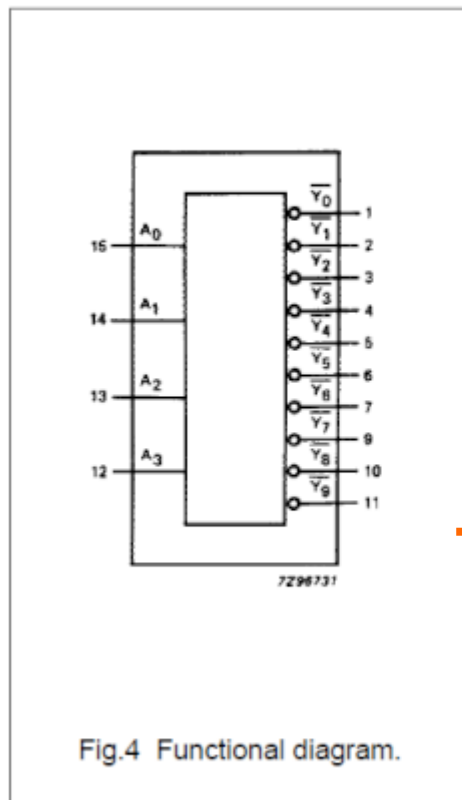
7442 TRUTH TABLE

input
8421BCD码

output
10个高低电平信号，
低电平有效

BCD to decimal decoder (1-of-10)

74HC/HCT42



FUNCTION TABLE

N	INPUTS				OUTPUTS									
	A ₃	A ₂	A ₁	A ₀	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇	Y ₈	Y ₉
0	L	L	L	L	L	H	H	H	H	H	H	H	H	H
1	L	L	L	H	H	L	H	H	H	H	H	H	H	H
2	L	L	H	L	H	H	L	H	H	H	H	H	H	H
3	L	L	H	H	H	H	H	L	H	H	H	H	H	H
4	L	H	L	L	H	H	H	H	L	H	H	H	H	H
5	L	H	L	H	H	H	H	H	H	L	H	H	H	H
6	L	H	H	L	H	H	H	H	H	H	L	H	H	H
7	L	H	H	H	H	H	H	H	H	H	H	L	H	H
8	H	L	L	L	H	H	H	H	H	H	H	H	L	H
9	H	L	L	H	H	H	H	H	H	H	H	H	H	L
	H	L	H	L	H	H	H	H	H	H	H	H	H	H
	H	L	H	H	H	H	H	H	H	H	H	H	H	H
	H	H	L	L	H	H	H	H	H	H	H	H	H	H
	H	H	L	H	H	H	H	H	H	H	H	H	H	H
	H	H	H	L	H	H	H	H	H	H	H	H	H	H
	H	H	H	H	H	H	H	H	H	H	H	H	H	H

伪码

Note

1. H = HIGH voltage level
L = LOW voltage level

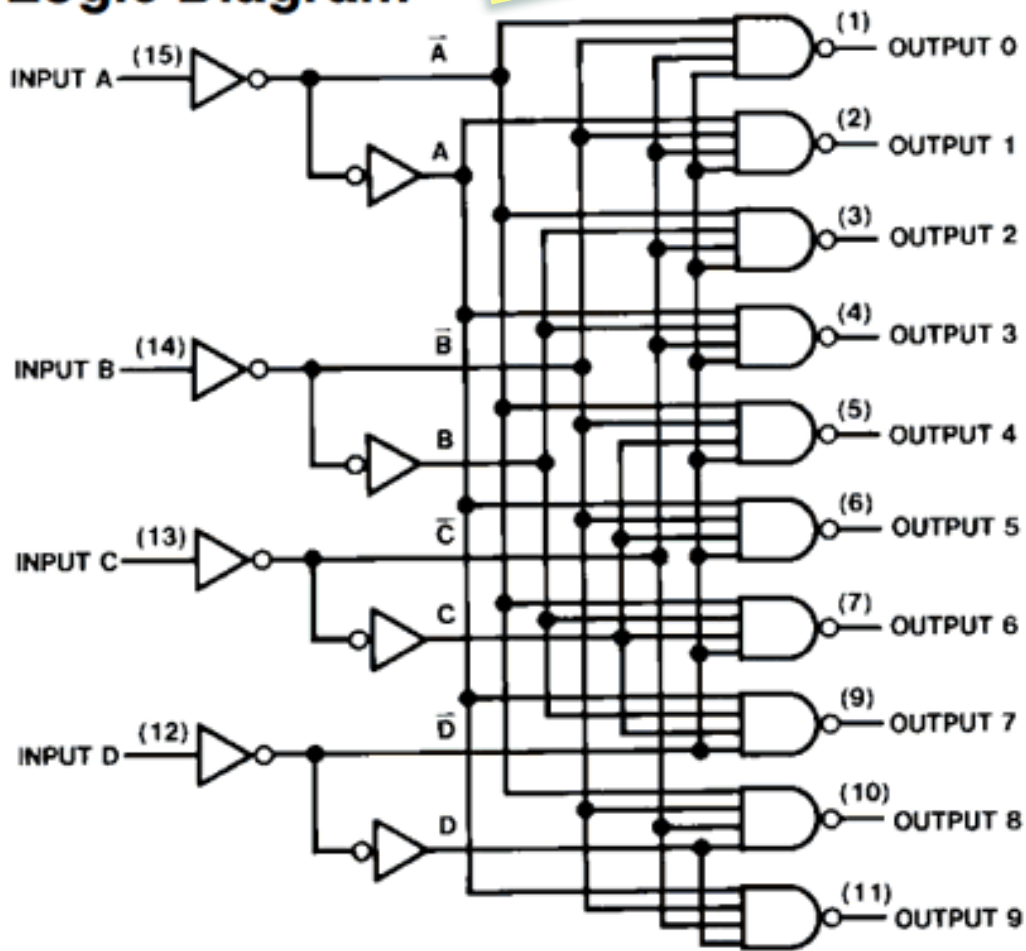
输入为伪码时，无对应低电平输出，输出端全为高电平。

7442

Important

译码器产生4个输入变量的最小项的非。

Logic Diagram



$$\bar{Y}_0 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

$$\bar{Y}_1 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

$$\bar{Y}_2 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

$$\bar{Y}_3 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

$$\bar{Y}_4 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

$$\bar{Y}_5 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

$$\bar{Y}_6 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

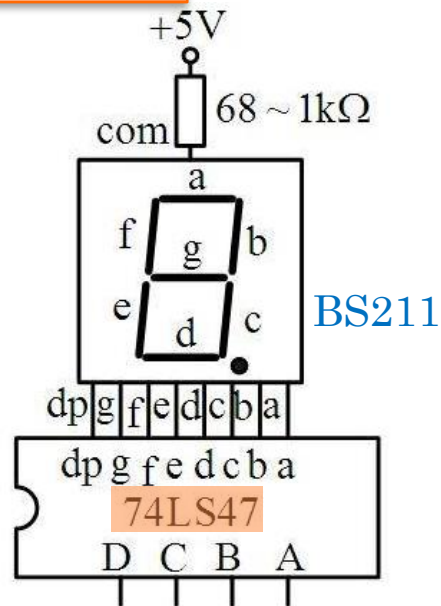
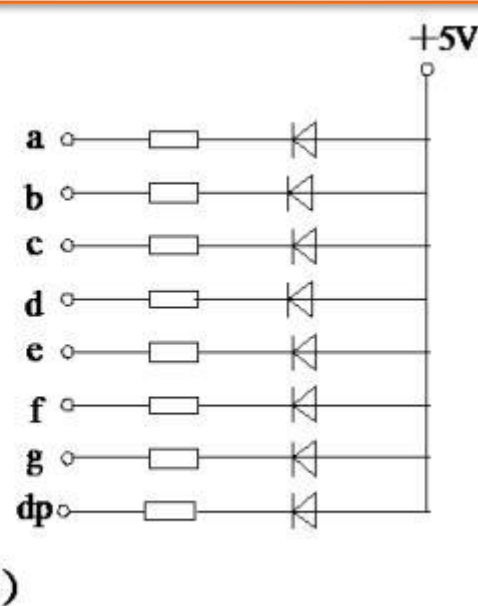
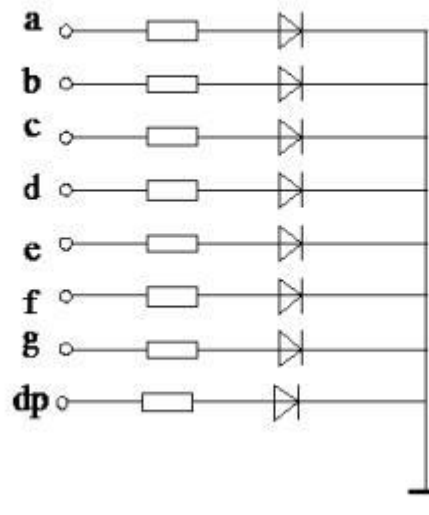
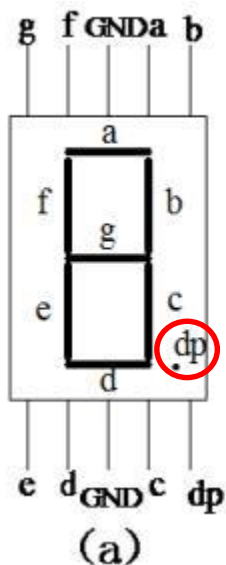
$$\bar{Y}_7 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

$$\bar{Y}_8 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

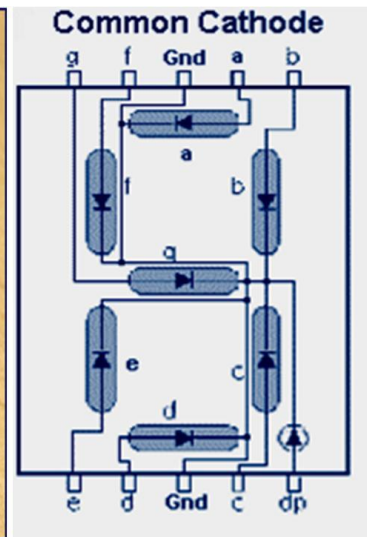
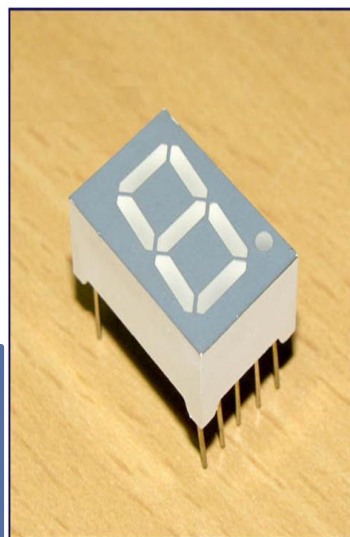
$$\bar{Y}_9 = \overline{A_3 \cdot A_2 \cdot A_1 \cdot A_0}$$

显示译码器

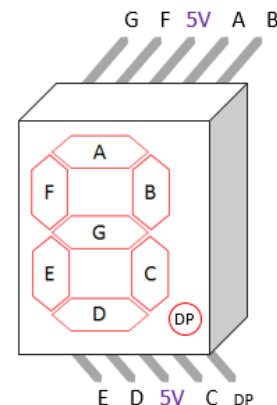
输入需要二-十进制代码，输出是译码结果，用来驱动七段发光器件



共阴极连接的
发光二极管
Common
Cathode



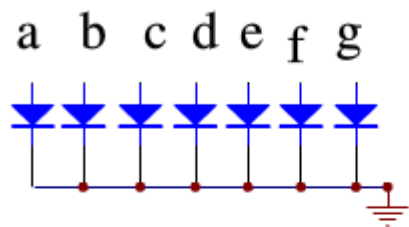
共阳极连接的发光二极管
Common Anode



7-segment display
七段发光显示器件 /
七段数码管

共阴极和共阳极连接的发光二极管

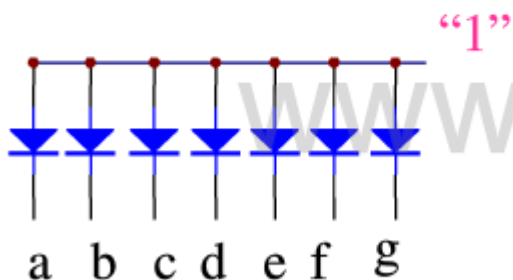
Connection: 连接方式不同分成共阴极和共阳极两种



BS201A
共阴极数码管

Common-cathode (共阴极)

diodes $\longrightarrow$ logic **high** $\longrightarrow$ bright



BS201B
共阳极数码管

Common-anode (共阳极)

diodes $\longrightarrow$ logic **low** $\longrightarrow$ bright

7448七段显示译码器 + BS201共阴极连接的发光二级管



segment outputs							display
a	b	c	d	e	f	g	
1	1	1	1	1	1	0	0
0	1	1	0	0	0	0	1
1	1	0	1	1	0	1	2
1	1	1	1	0	0	1	3
0	1	1	0	0	1	1	4
1	0	1	1	0	1	1	5
0	0	1	1	1	1	1	6
1	1	1	0	0	0	0	7
1	1	1	1	1	1	1	8
1	1	1	0	0	1	1	9

7-Segment Display

7448输出高电平有效

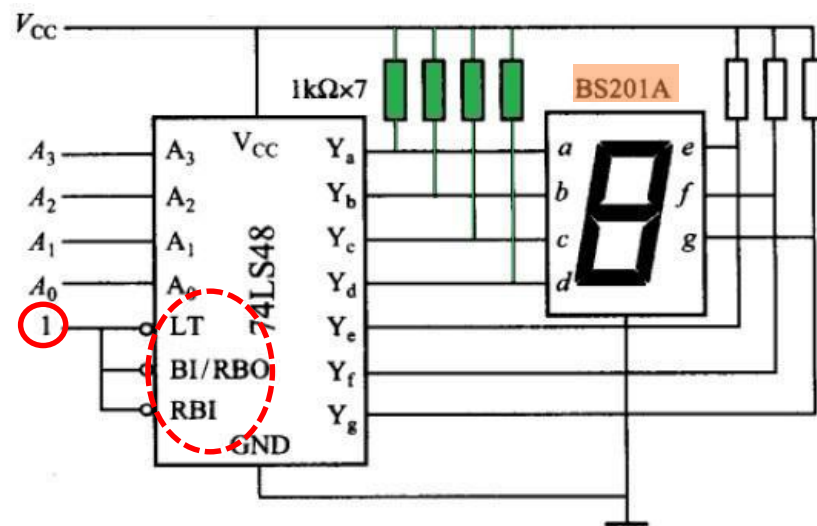
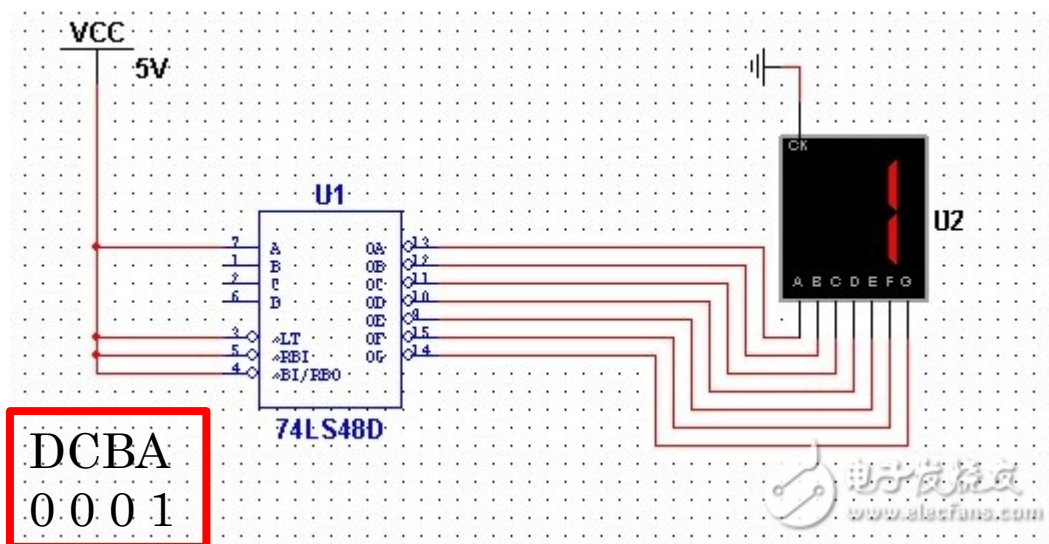


图 6.5.4 用 74LS48 控制一位 LED 显示电路

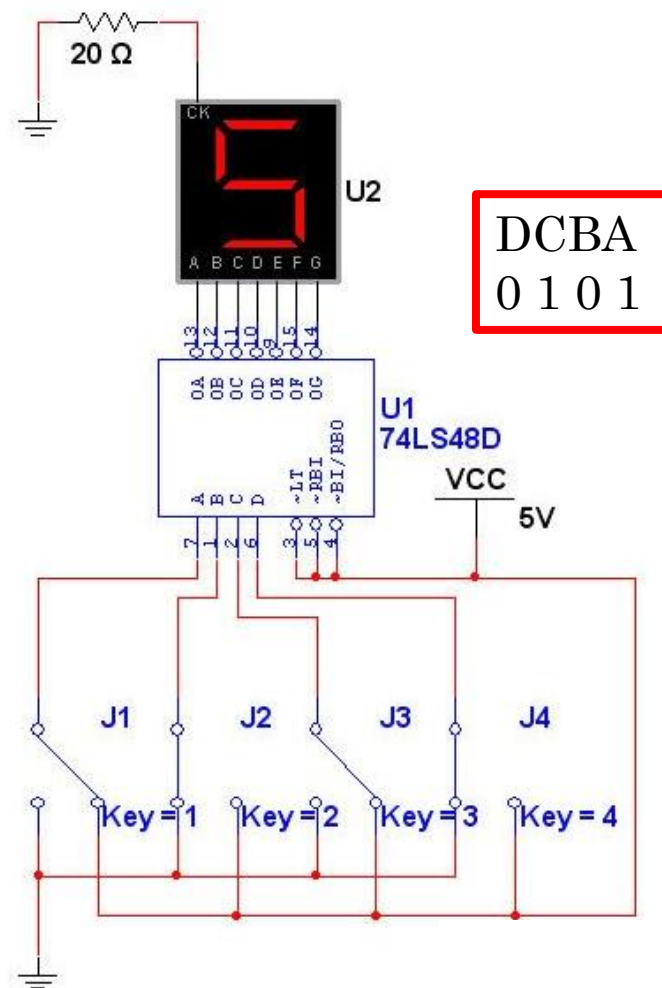
Common Cathode
共阴极

7448除了BCD码输入外，
还有三个辅助控制端。

七段显示译码器示例



- 74LS48 的输入端A B C D是8421码，**D是高位**，通过A B C D 的高低电平控制数码管的显示。
- 如上图：A是高电平，B C D 是低电平，表示十进制“1”，因此数码管显示“1”。
- 千万别把数码管选错了，共阴和共阳极数码管不同。



7448 TRUTH TABLE

$\overline{RBO}$	发光器件	Truth Table														
1	有显示	Decimal Or Function	Inputs						Outputs							
0	无显示		LT	$\overline{RBI}$	D A_3	C A_2	B A_1	A A_0	$\overline{BI}/\overline{RBO}$	a	b	c	d	e	f	g
		0 (Note 1)	H	H	L	L	L	L	H	H	H	H	H	H	H	L
		1 (Note 1)	H	X	L	L	L	H	H	L	H	H	L	L	L	L
		2	H	X	L	L	H	L	H	H	H	L	H	H	L	H
		3	H	X	L	L	H	H	H	H	H	H	H	L	L	H
		4	H	X	L	H	L	L	H	L	H	H	L	L	H	H
		5	H	X	L	H	L	H	H	H	L	H	H	L	H	H
		6	H	X	L	H	H	L	H	L	L	H	H	H	H	H
		7	H	X	L	H	H	H	H	H	H	H	L	L	L	L
		8	H	X	H	L	L	L	H	H	H	H	H	H	H	H
		9	H	X	H	L	L	H	H	H	H	H	L	L	H	H
		10	H	X	H	L	H	L	H	L	L	L	H	H	L	H
		11	H	X	H	L	H	H	H	L	L	H	H	L	L	H
		12	H	X	H	H	L	L	H	L	H	L	L	L	H	H
		13	H	X	H	H	L	H	H	H	L	L	H	L	H	H
		14	H	X	H	H	H	L	H	L	L	L	H	H	H	H
		15 (Note 2)	H	X	H	H	H	H	L	L	L	L	L	L	L	L
		$\overline{BI}$ (Note 2)	X	X	X	X	X	X	L	L	L	L	L	L	L	L
		$\overline{RBI}$ (Note 3)	H	L	L	L	L	L	L	L	L	L	L	L	L	L
		LT (Note 4)	L	X	X	X	X	X	H	H	H	H	H	H	H	H

$\overline{BI}$ 熄灭信号输入

$\overline{RBI}$
动态灭零输入信号

LT试灯输入信号

$\overline{BI}$: Blanking Input
 $\overline{RBO}$: Ripple Blanking Output
 $\overline{RBI}$: Ripple Blanking Input

七段显示译码器7448功能表

十进制 或功能	输 入						$\overline{BI}/$	输 出							字 形
	$\overline{LT}$	$\overline{RBI}$	D	C	B	A	$\overline{RBO}$	a	b	c	d	e	f	g	
0	1	1	0	0	0	0	1	1	1	1	1	1	1	0	0
1	1	×	0	0	0	1	1	0	1	1	0	0	0	0	1
2	1	×	0	0	1	0	1	1	1	0	1	1	0	1	2
3	1	×	0	0	1	1	1	1	1	1	1	0	0	1	3
4	1	×	0	1	0	0	1	0	1	1	0	0	1	1	4
5	1	×	0	1	0	1	1	1	0	1	1	0	1	1	5
6	1	×	0	1	1	0	1	0	0	1	1	1	1	1	6
7	1	×	0	1	1	1	1	1	1	1	0	0	0	0	7
8	1	×	1	0	0	0	1	1	1	1	1	1	1	1	8
9	1	×	1	0	0	1	1	1	1	1	1	0	1	1	9
10	1	×	1	0	1	0	1	0	0	0	1	1	0	1	a
11	1	×	1	0	1	1	1	0	0	1	1	0	0	1	b
12	1	×	1	1	0	0	1	0	1	0	0	0	1	1	c
13	1	×	1	1	0	1	1	1	0	0	1	0	1	1	d
14	1	×	1	1	1	0	1	0	0	0	1	1	1	1	e
15	1	×	1	1	1	1	1	0	0	0	0	0	0	0	
消隐	×	×	×	×	×	×	0	0	0	0	0	0	0	0	
脉冲消隐	1	0	0	0	0	0	0	0	0	0	0	0	0	0	
灯测试	0	×	×	×	×	×	1	1	1	1	1	1	1	1	8

7448辅助端说明

○ $\overline{LT}$: 试灯输入信号

- $\overline{LT} = 0, \overline{RBO} = 1$, 无论其它是什么输入, 输出 $a \sim g$ 都为1。
- 用于测试7448自身的输出以及显示器件是否正常。

○ $\overline{RBI}$: 动态灭零输入信号

- $\overline{LT} = 1, \overline{RBI} = 0, A_3A_2A_1A_0 = 0000$, 输出 $a \sim g$ 都为0, 称为灭0。

○ $\overline{BI}/\overline{RBO}$: 熄灭信号输入/动态灭零输出信号

- 一个特殊的控制端, 既可以作为输入, 也可以作为输出。
- 作为输入端 $\overline{BI}$ 时, $\overline{BI} = 0$, 无论其它输入端输入什么值, 输出 $a \sim g$ 都为0, 不显示字形。
- 作为输出端 $\overline{RBO}$ 时, 受控于 $\overline{LT}$ 端和 $\overline{RBI}$ 端: 有显示时 $\overline{RBO} = 1$; 无显示时 $\overline{RBO} = 0$

① $\overline{LT} = 0 \rightarrow \overline{RBO} = 1$, 试灯信号

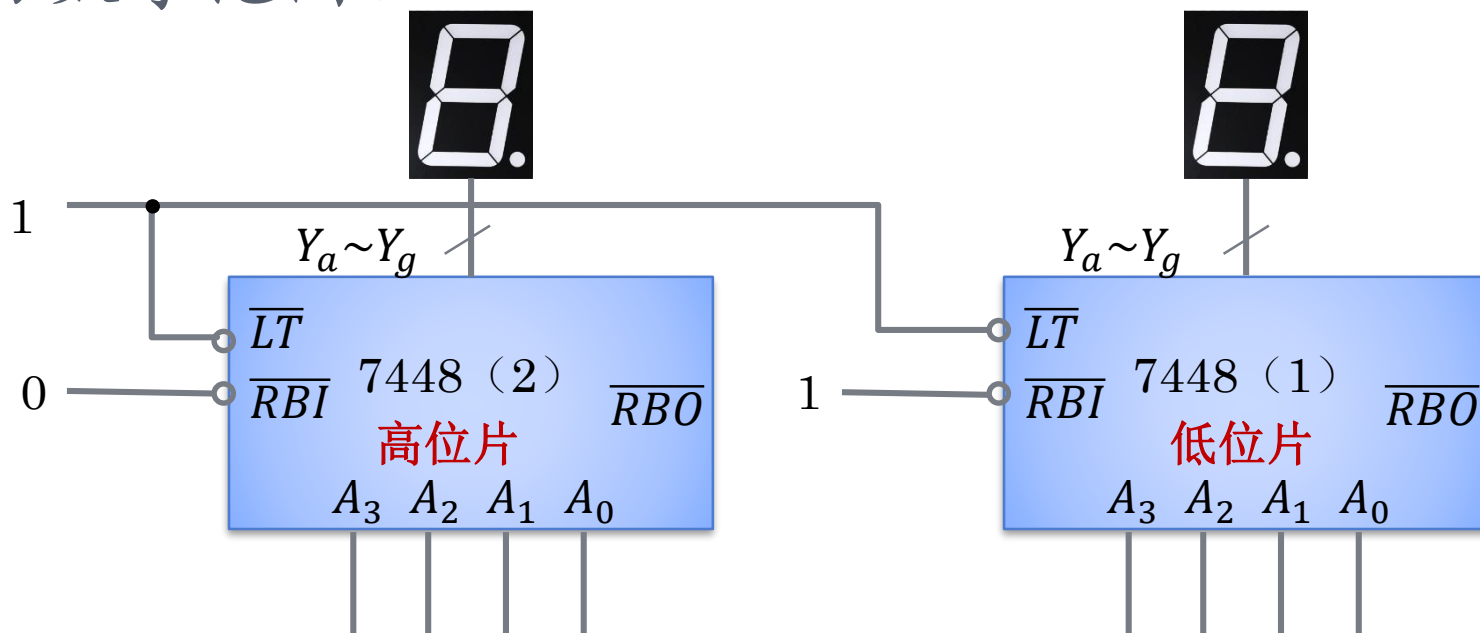
② $\overline{LT} = 1, \overline{RBI} = 0, A_3A_2A_1A_0 = 0000 \rightarrow \overline{RBO} = 0$ 灭0

③ $\overline{LT} = 1, \overline{RBI} = x, A_3A_2A_1A_0 = xxxx$ (只要不是全0) $\rightarrow \overline{RBO} = 1$ 正常显示

④ 这一端主要用于显示多位数字时, 多个译码器之间的连接, 对高位显示器件灭0。

(见下一页例题)

例：当两个显示译码器输入8421BCD码时，分析显示器显示的数字范围。



(1) 显示的数码范围为 1 ~ 9

$\overline{LT} = 1, \overline{RBI} = 0,$
当 $A_3A_2A_1A_0 = 0000 \rightarrow \overline{RBO} = 0$ 灭0

(2) 显示的数码范围为 0 ~ 9

$\overline{LT} = 1, \overline{RBI} = 1,$
当 $A_3A_2A_1A_0 = 0000 \rightarrow \overline{RBO} = 1$ 正常显示

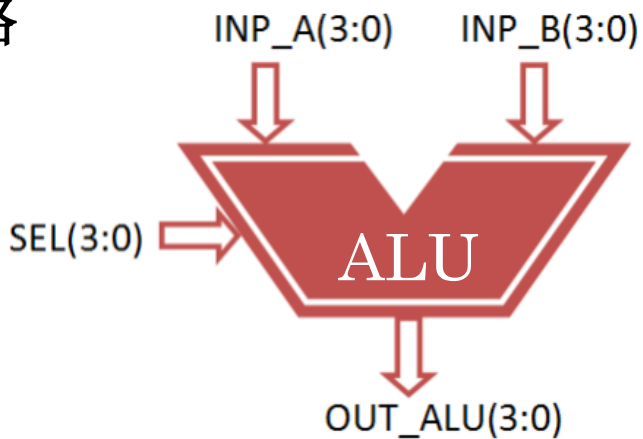
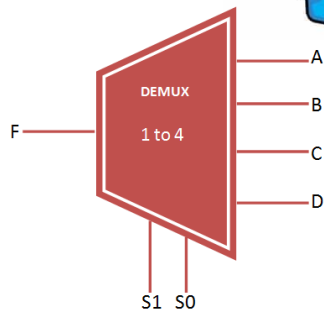
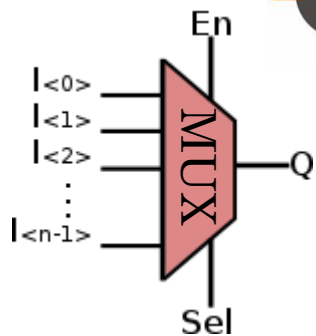
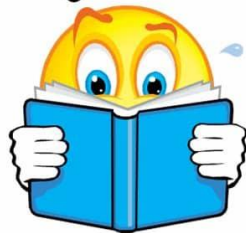
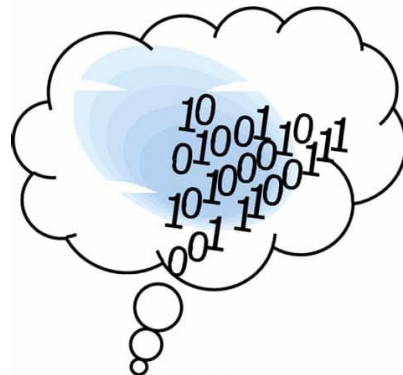
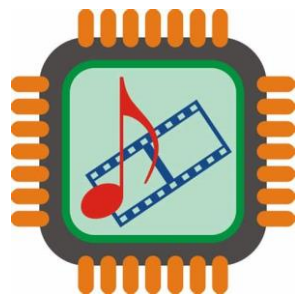
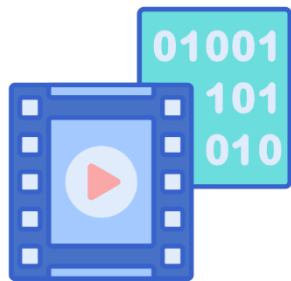
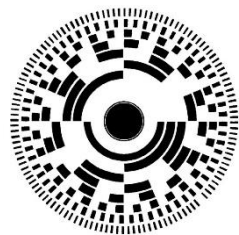
$A_3A_2A_1A_0 \neq 0000$ 的其它输入，高位片低位片都能正常显示数码

集成编码器和译码器总结

分类	Chip	In-Out	辅助控制端	备注
编码器	74148	8-3线	$\overline{EI}, EO(Y_S), \overline{GS}(\overline{Y_{EX}})$	两片74148可构成一个16-4线优先编码器
	74147	10-4线		只有9个输入信号。当9个输入端全为1时，代表输入的是十进制数0。
译码器	74139	双2-4线	$\overline{E_a}, \overline{E_b}$	
	74138	3-8线	$E_3, \overline{E_2}, \overline{E_1}$	①两片74138可构成一个4-16线译码器； ②74138还可以构成数据分配器
	7442	4-10线		NBCD-Decimal译码，输出低电平有效。
	7448	4-7	$\overline{LT}, \overline{RBI}, \overline{BI}/\overline{RBO}$	七段显示译码器

内容

- 6.1 编码器
- 6.2 译码器
- 6.3 数据选择器
- 6.4 数据分配器
- 6.5 数值比较器
- 6.6 奇偶校验位产生与校验电路
- 6.7 算术运算电路
- 6.8 算术逻辑单元ALU
- 本章小结



数据选择器 (DATA SELECTOR, MULTIPLEXER, MUX)

特点

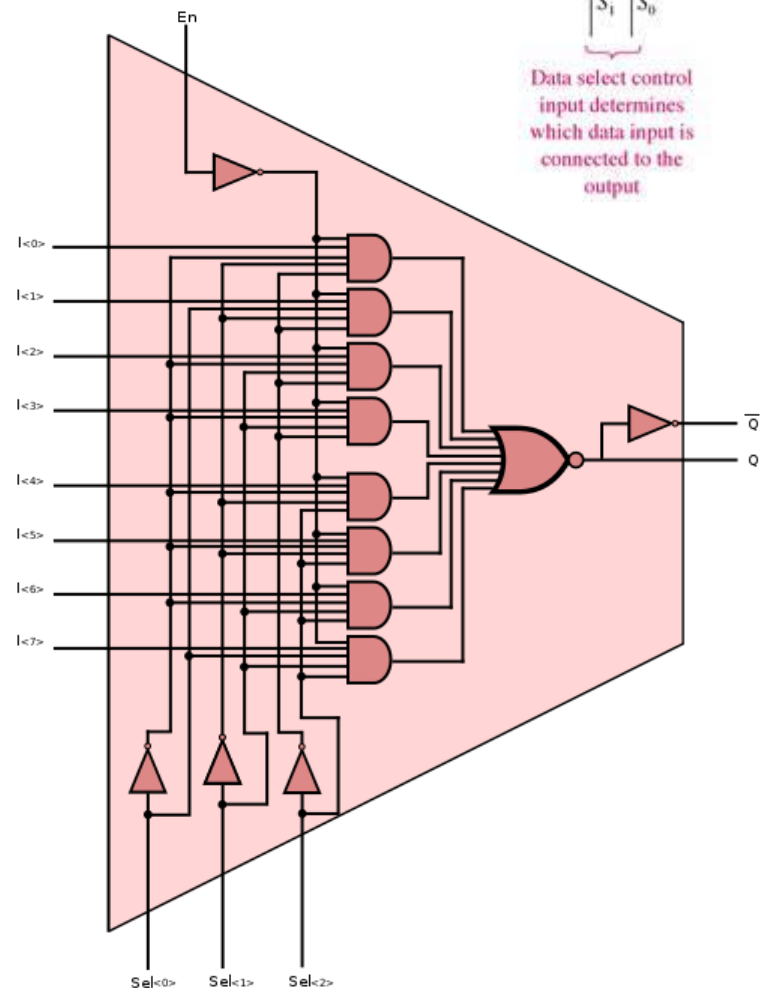
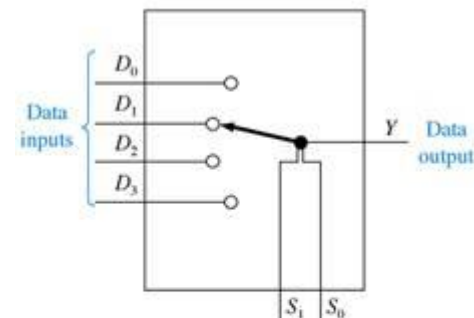
- 能够从一组输入数据中，选择出所需要的一个数据
- 并将其输出到**唯一**的数据通道上
- 相当于一个**多路到一路**的开关

中规模集成数据选择器有

- 双4选1
- 8选1
- 16选1

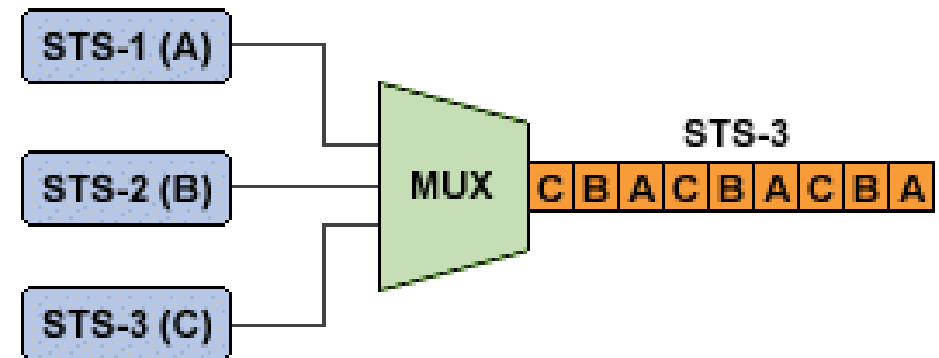
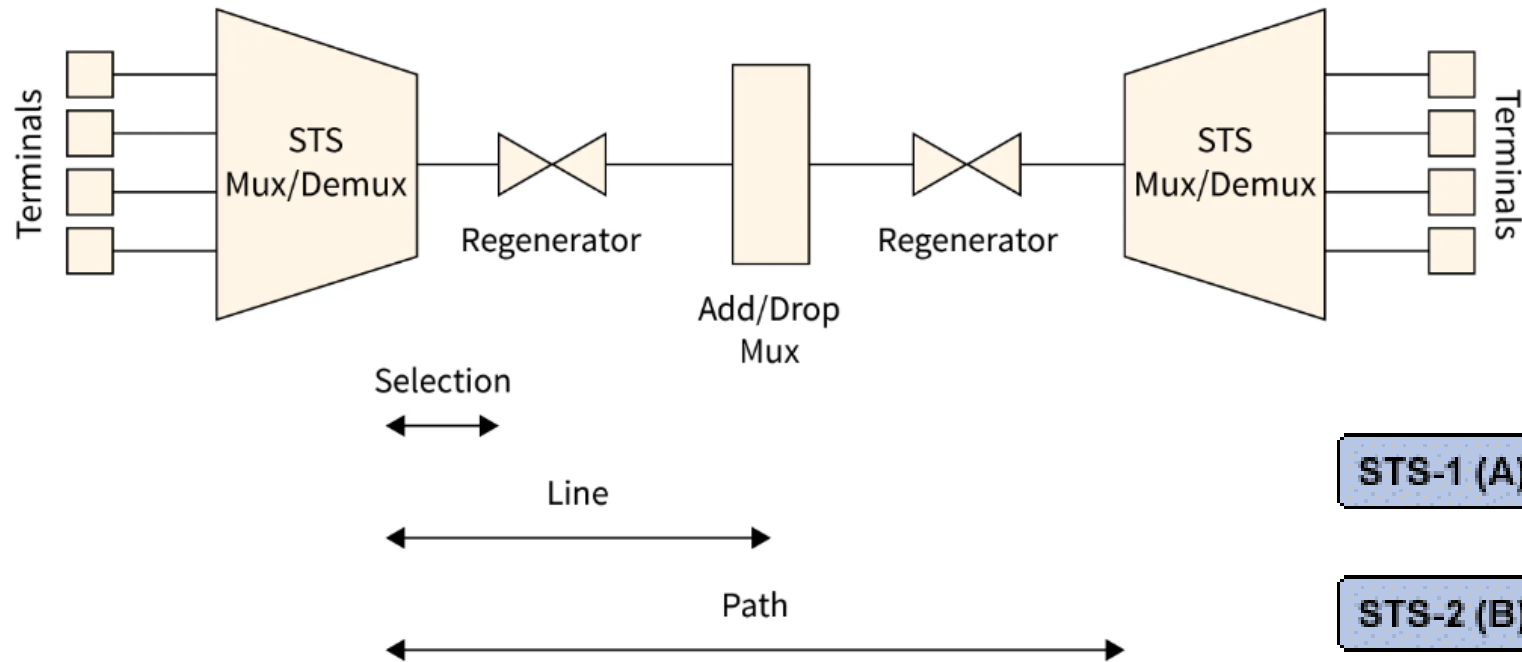
数据选择器应用

- 在数据通信中应用广泛，将**低速多路**并行信号**转换为一路高速**信号（**并行到串行转换**），以便于数据的高速传输

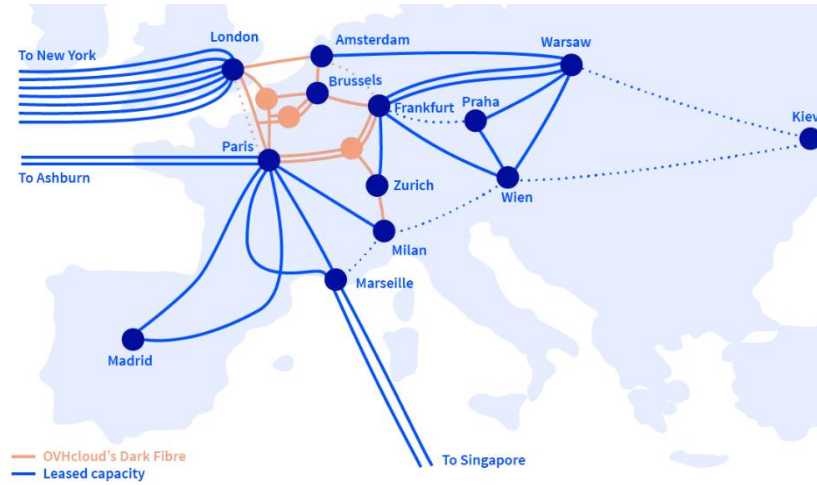
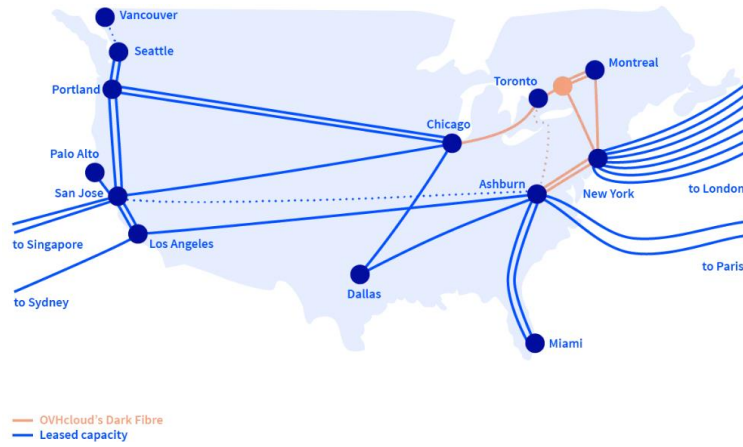


SONET SIGNAL HIEARARCHY

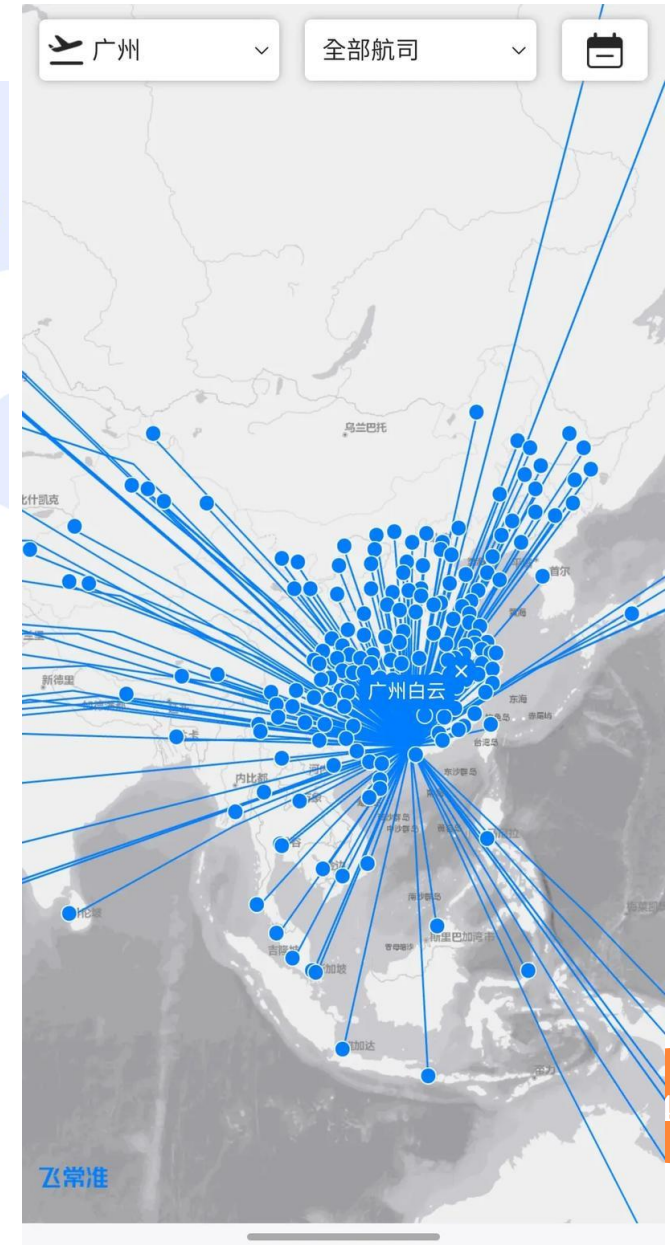
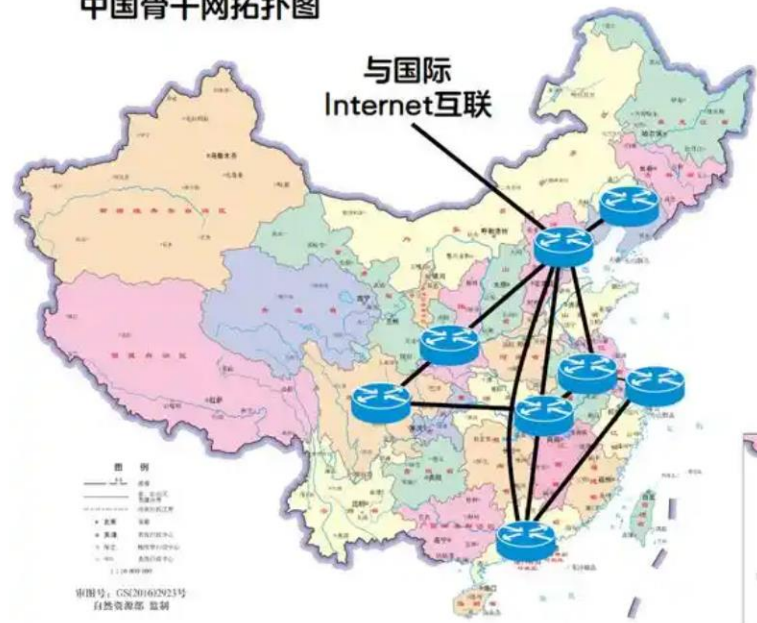
SONET signal	Bit rates	Equivalent SDH signal
STS-1 OC-1	51.84 Mb/s	STM-0
STS-3 OC-3	155.52 Mb/s	STM-1
STS-12 OC-12	622.08 Mb/s	STM-4
STS-48 OC-48	2488.32 Mb/s	STM-16
STS-192 OC-192	9953.28 Mb/s	STM-64
STS-768 OC-768	39813.12 Mb/s	STM-256



网络、航空枢纽



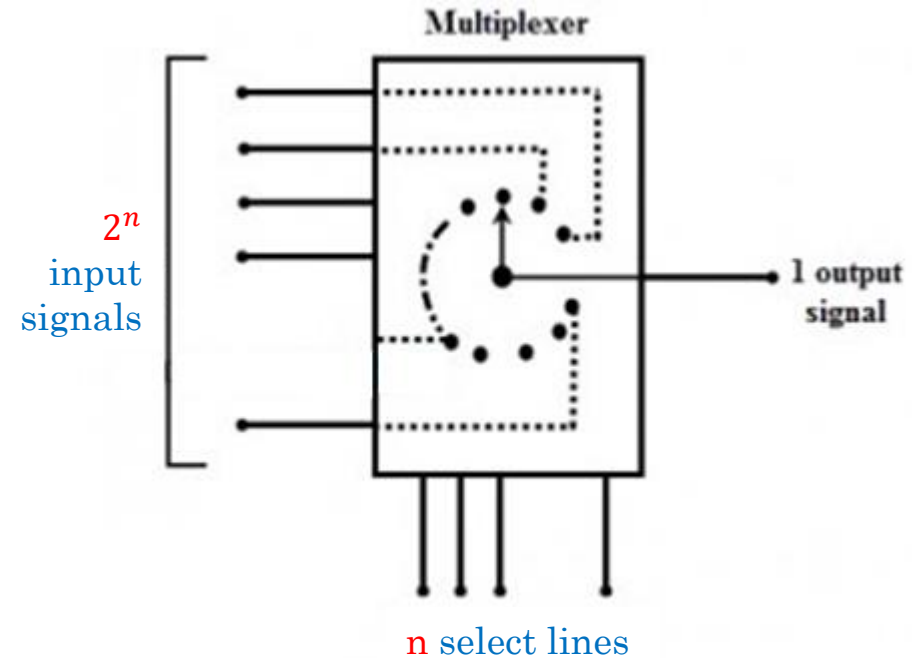
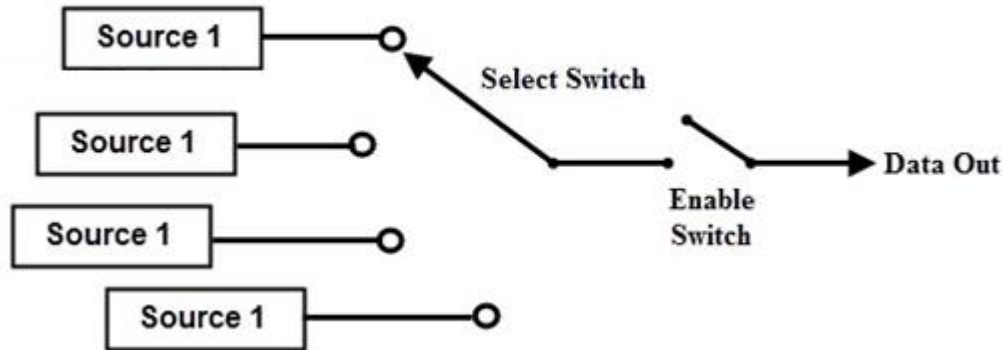
中国骨干网拓扑图



GENERAL IDEA OF A DATA SELECTOR

Generally the number of data inputs to a multiplexer is **a power of two** such as 2, 4, 8, 16, etc.

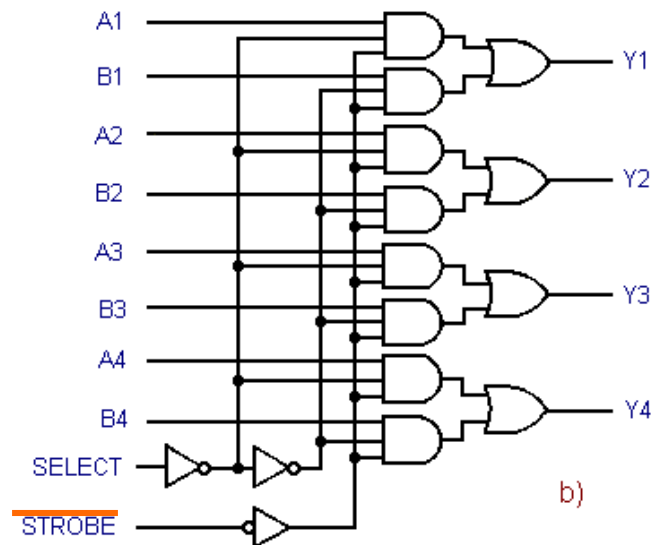
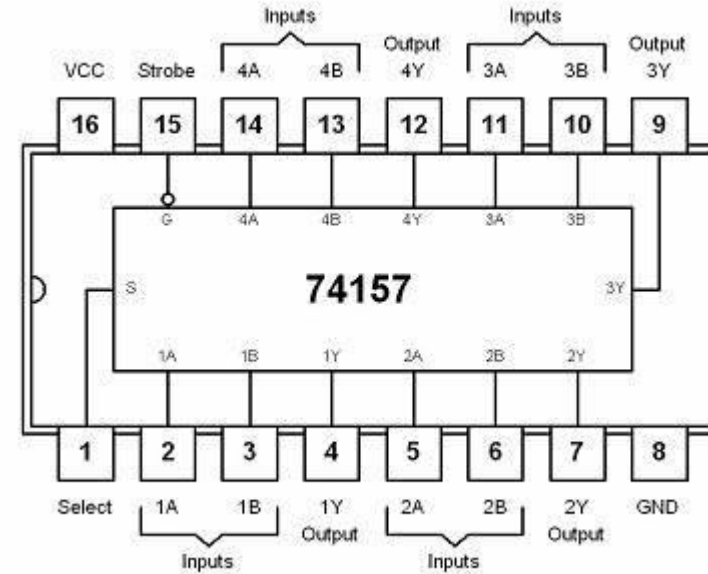
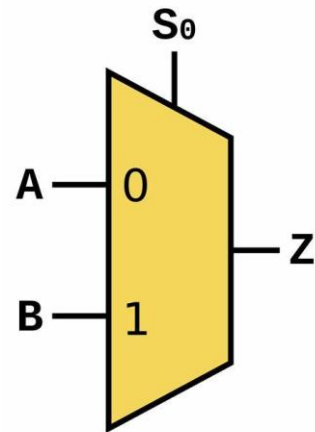
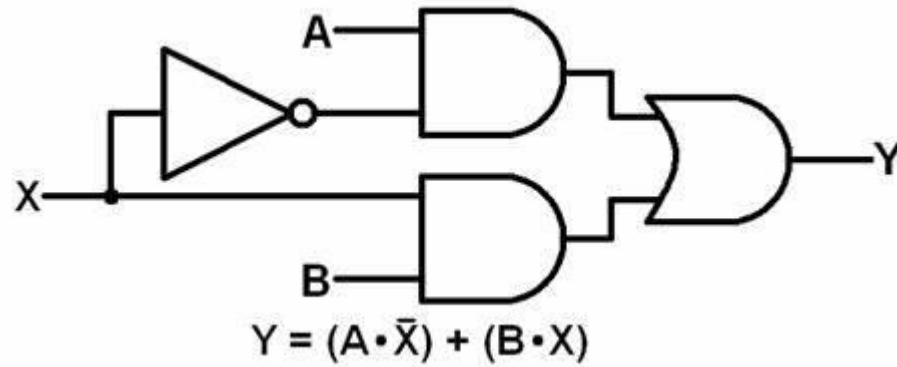
Some of the mostly used multiplexers include 2-to-1, 4-to-1, 8-to-1 and 16-to-1 multiplexers.



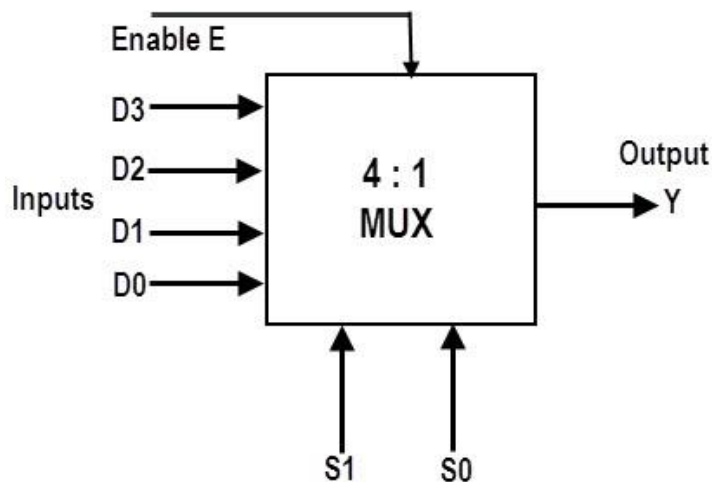
2^n - to - 1 multiplexer

Generally **four** 2 line to 1 line **multiplexers** are fabricated in a single IC (74157)

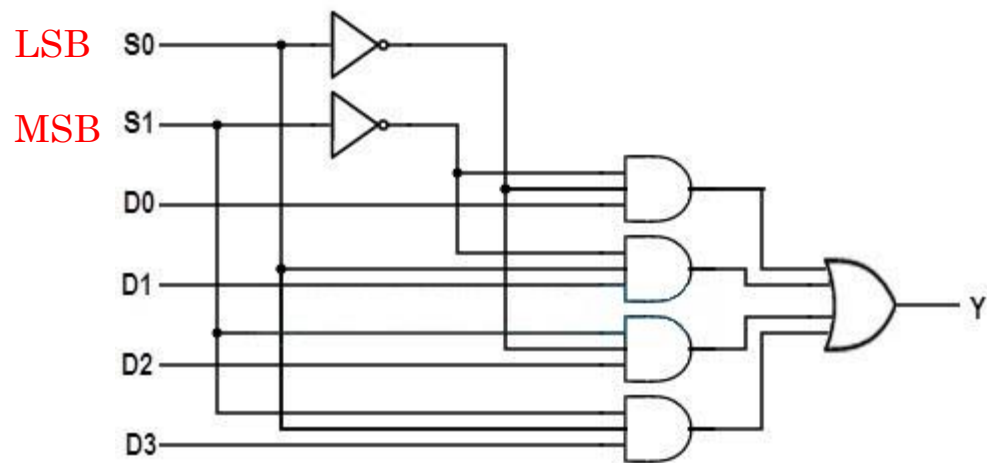
二选一数据选择器



四选一数据选择器



Select Data Inputs		Output
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3



$$Y = \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_1} \cdot S_0 \cdot D_1 + S_1 \cdot \overline{S_0} \cdot D_2 + S_1 \cdot S_0 \cdot D_3$$

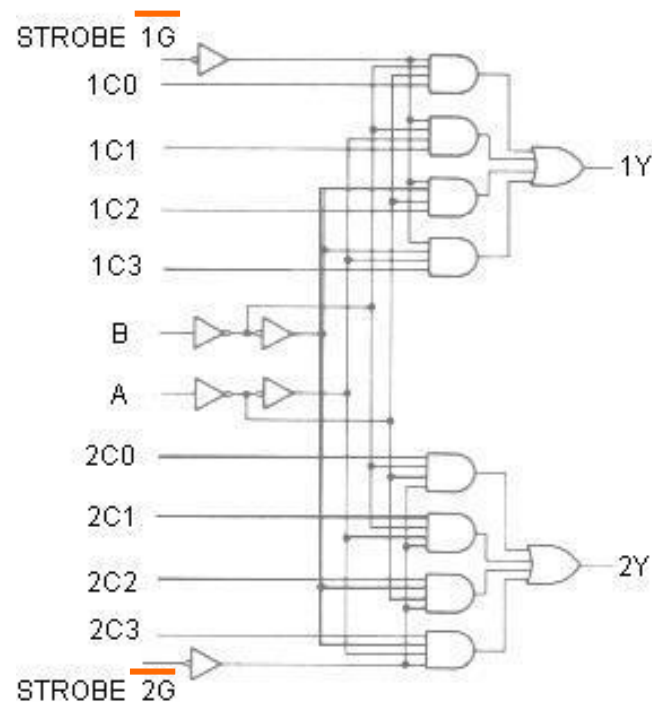
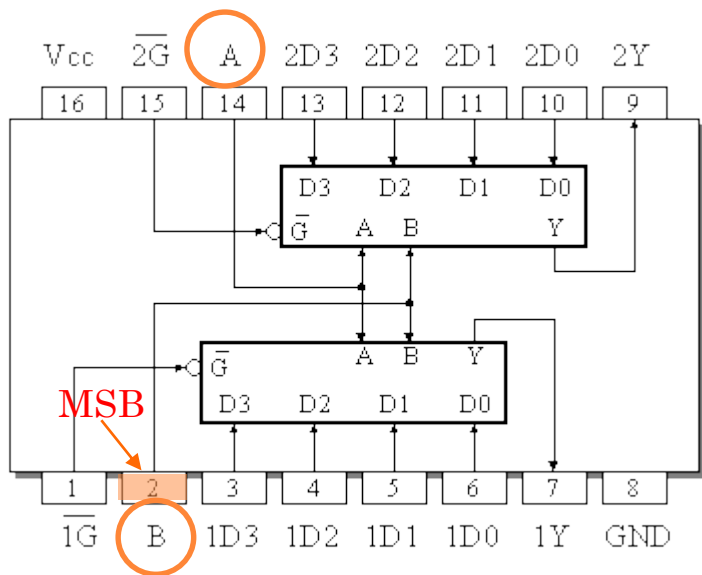
$$= \sum_{i=0}^3 m_i \cdot D_i$$

Usually, the **enable input (or strobe)** can be used to cascade (串联) two or more multiplexer ICs to construct a multiplexer with large number of inputs.

74153: 双4选1数据选择器

DUAL 4-INPUT MULTIPLEXER

- 集成了两个相同功能的4选1
- 公用控制信号A, B, 即地址端
- 两个使能端 $\overline{1G}$, $\overline{2G}$

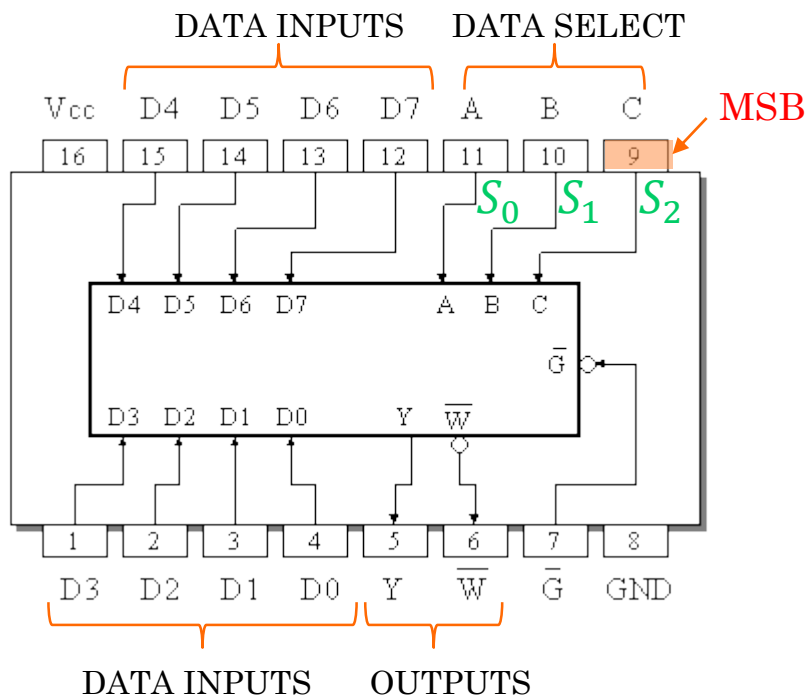


使能 $\overline{G}$	B	A	Y
1	X	X	0
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3

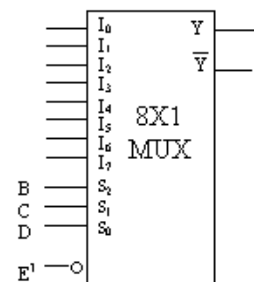
$$Y = \overline{B} \cdot \overline{A} \cdot D_0 + \overline{B} \cdot A \cdot D_1 + B \cdot \overline{A} \cdot D_2 + B \cdot A \cdot D_3 = \sum_{i=0}^3 m_i \cdot D_i$$

74151

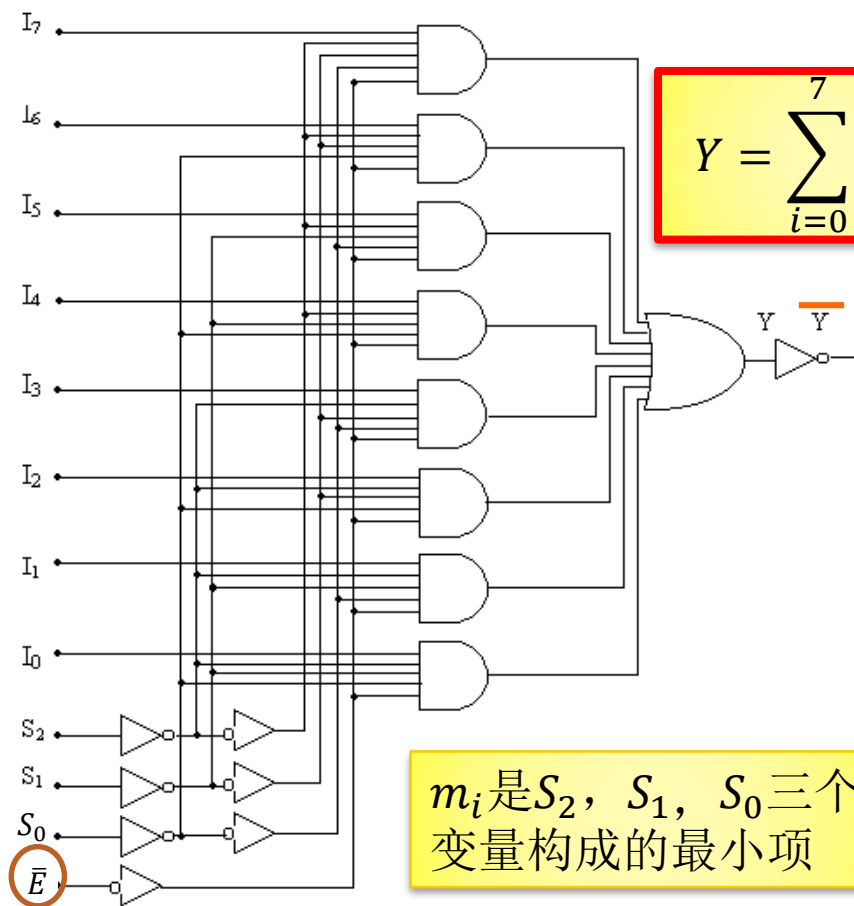
8选1数据选择器



An enable input $\bar{E}$ can be used to **expand** the size of the multiplexer



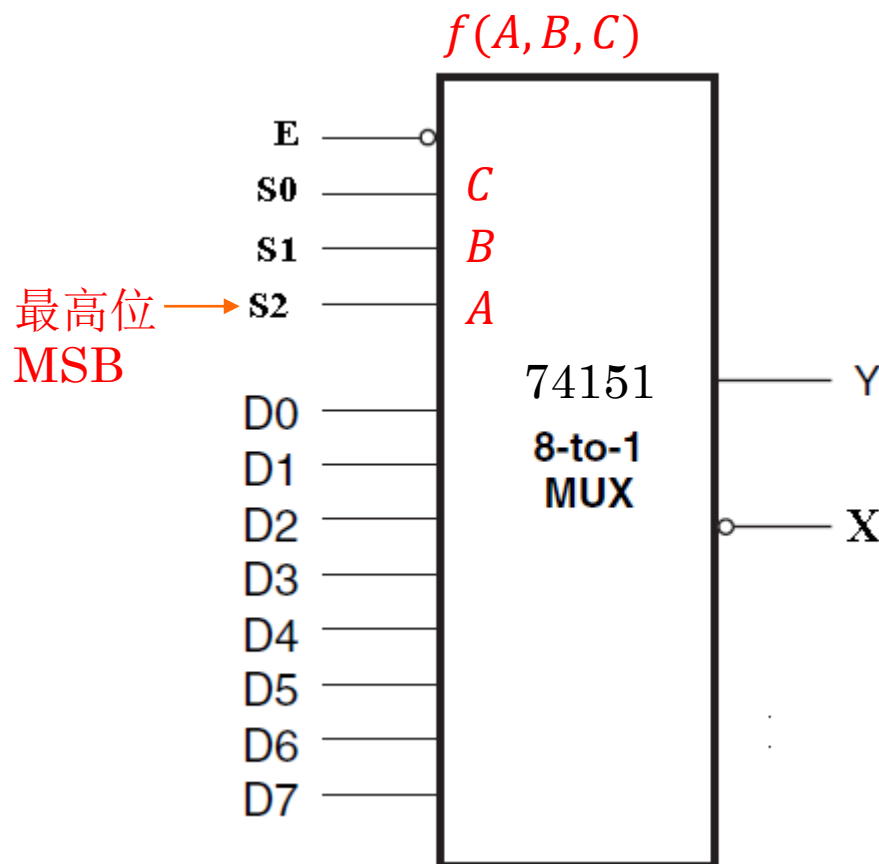
GİRİŞLER	ÇIKIŞLAR
E ¹ S ₂ S ₁ S ₀	Y ¹ Y
1 X X X	1 0
0 0 0 0	I ₀ ¹ I ₀
0 0 0 1	I ₁ ¹ I ₁
0 0 1 0	I ₂ ¹ I ₂
0 0 1 1	I ₃ ¹ I ₃
0 1 0 0	I ₄ ¹ I ₄
0 1 0 1	I ₅ ¹ I ₅
0 1 1 0	I ₆ ¹ I ₆
0 1 1 1	I ₇ ¹ I ₇



$$Y = \sum_{i=0}^7 m_i \cdot I_i$$

m_i 是 S_2 , S_1 , S_0 三个变量构成的最小项

74151 8-1 MUX TRUTH TABLE

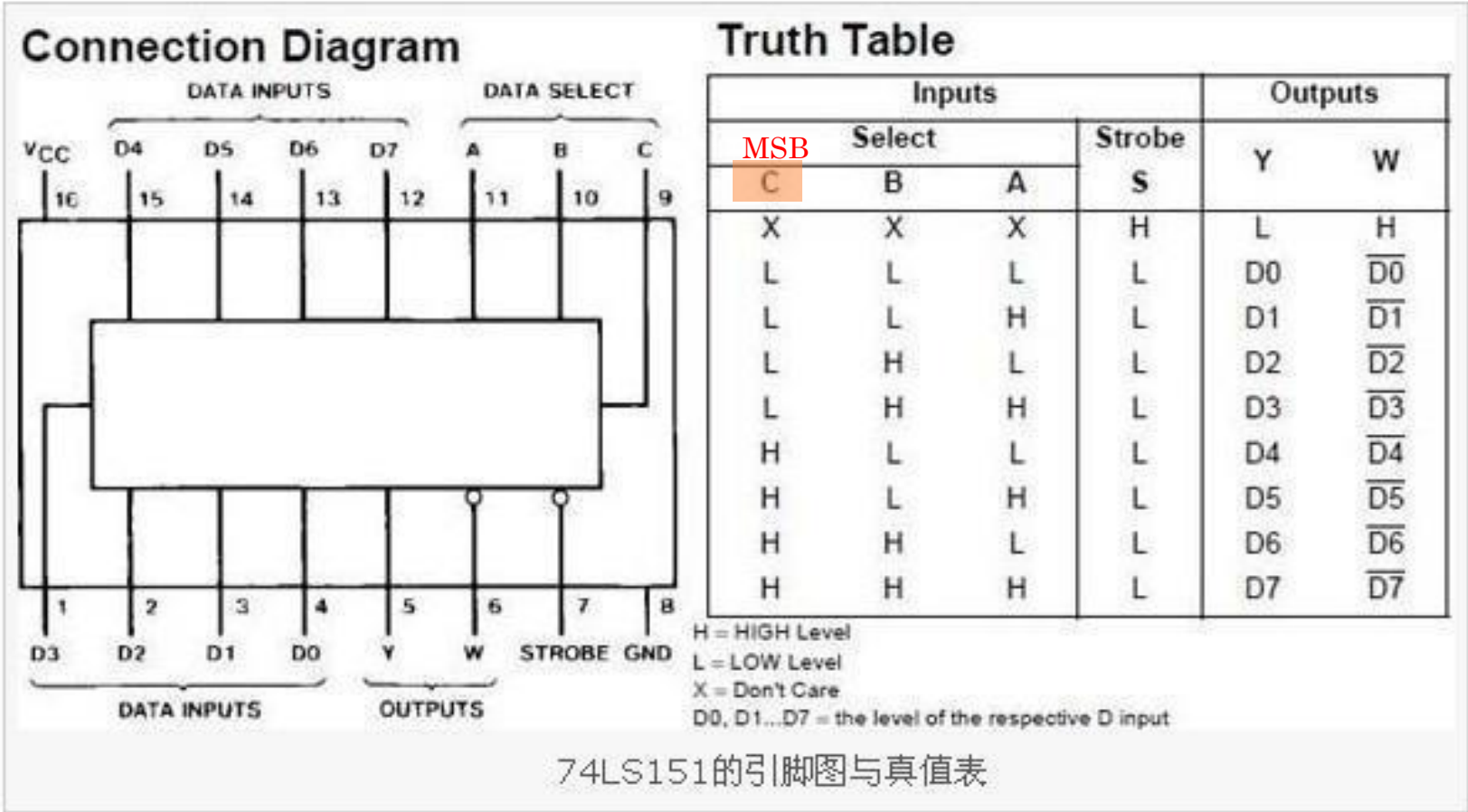


Inputs				Output	
Select			Enable E	Y X	
S2	S1	S0			
X	X	X	H	L	H
L	L	L	L	D0	$\overline{D0}$
L	L	H	L	D1	$\overline{D1}$
L	H	L	L	D2	$\overline{D2}$
L	H	H	L	D3	$\overline{D3}$
H	L	L	L	D4	$\overline{D4}$
H	L	H	L	D5	$\overline{D5}$
H	H	L	L	D6	$\overline{D6}$
H	H	H	L	D7	$\overline{D7}$

E : ENABLE input
 S0, S1, S2 : Select inputs
 D0-D7 : Data inputs
 Y, X : outputs

$$Y = \sum_{i=0}^7 m_i \cdot D_i = \overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_2} \cdot \overline{S_1} \cdot S_0 \cdot D_1 + \overline{S_2} \cdot S_1 \cdot \overline{S_0} \cdot D_2 + \overline{S_2} \cdot S_1 \cdot S_0 \cdot D_3 + S_2 \cdot \overline{S_1} \cdot \overline{S_0} \cdot D_4 + S_2 \cdot \overline{S_1} \cdot S_0 \cdot D_5 + S_2 \cdot S_1 \cdot \overline{S_0} \cdot D_6 + S_2 \cdot S_1 \cdot S_0 \cdot D_7$$

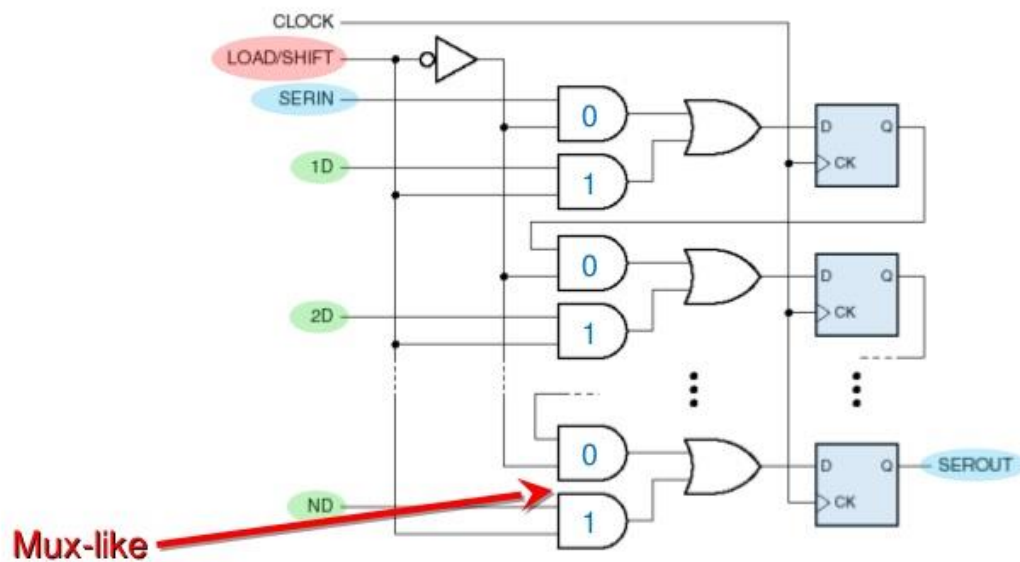
74151 8-1 MUX 引脚图与真值表



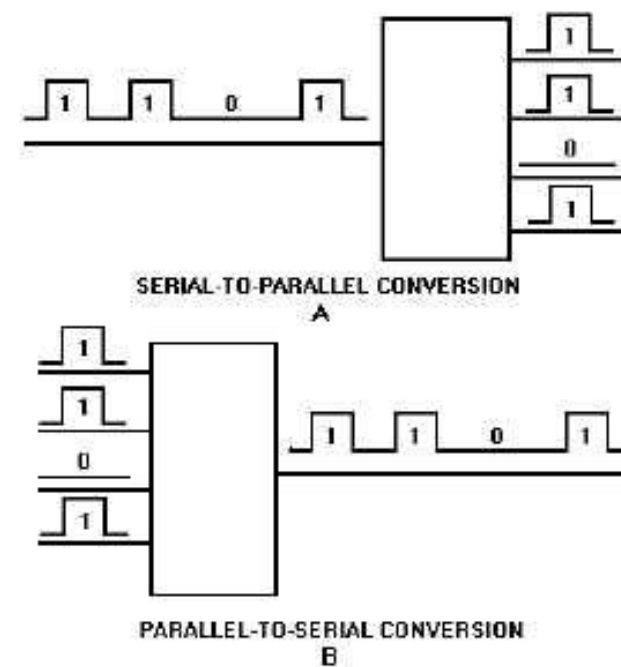
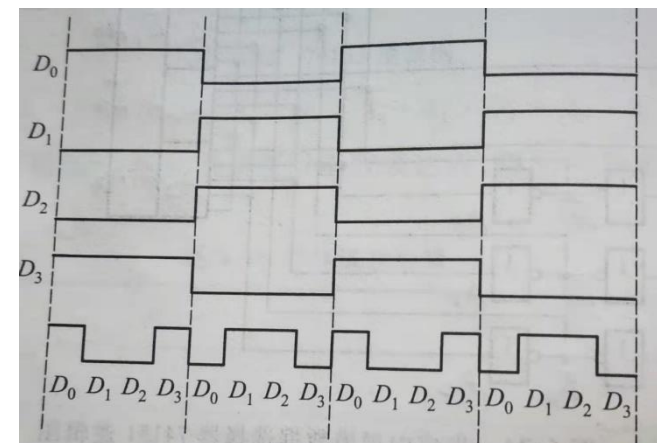
实现数据的并行到串行转换
即多路低速数据转换为一路高速数据

Parallel-to-Serial Conversion

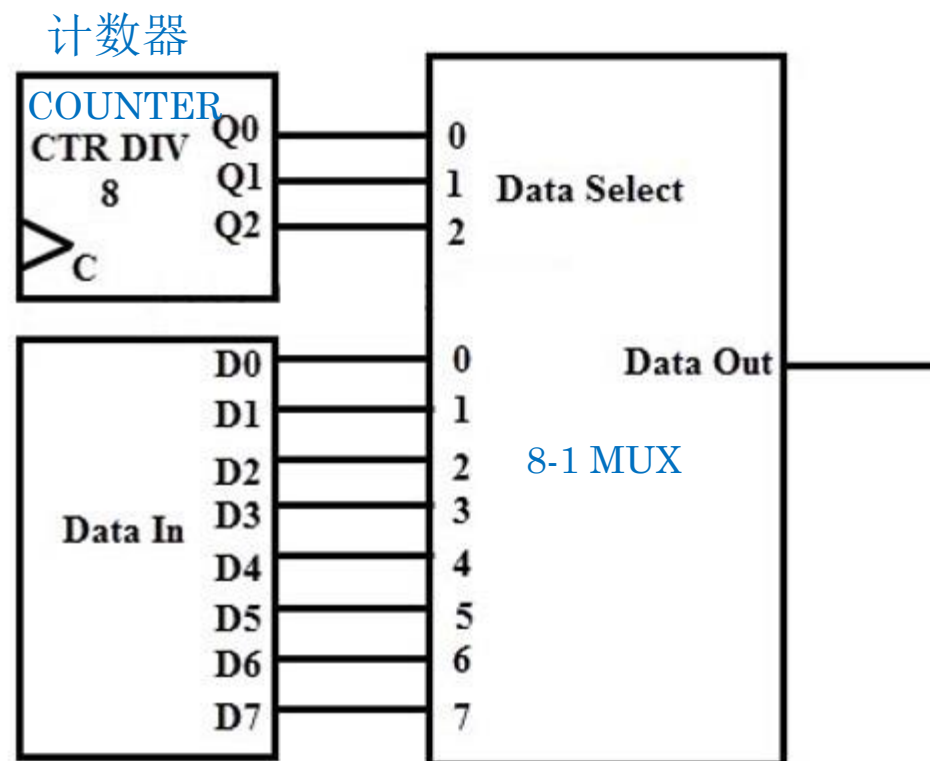
- Logic circuit for a parallel-in, serial-out shift register



配合计数器，从00~11循环，
使MUX依次选择 $D_0 \sim D_3$ 输出



PARALLEL TO SERIAL CONVERSION



MULTIPLEXER QUIZ

1. The number of control lines for a 8-to-1 multiplexer is

- A) 2 B) 3 C) 4 D) 5

2. IC 74LS153 is a

- A) 4:1mux B) 8:1 mux C) 16:1 mux D) Dual 4:1 mux

数据选择器构成逻辑函数发生器 (1)

- 逻辑函数变量数 $\equiv$ 数据选择器的地址端数量时，
 - n 个地址端的数据选择器可以很方便地实现 n 变量的任意函数。
- 逻辑函数变量数 $<$ 数据选择器的地址端数量时，
 - 只要将数据选择器相应的地址端不用（接0或者1），就可以实现逻辑函数。

$$f(A, B, C) = \bar{A}\bar{B}(C + \bar{C}) + \bar{A}B(C + \bar{C}) + A\bar{B}(C + \bar{C}) + AB(C + \bar{C})$$

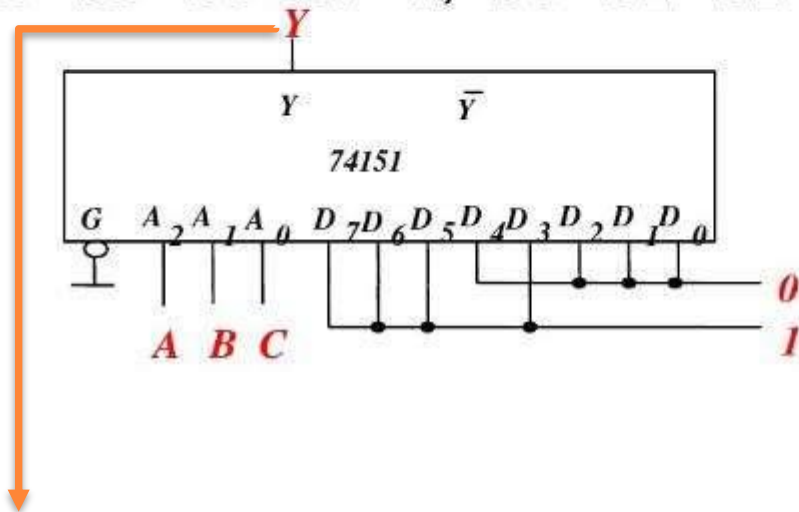
- 逻辑函数变量数 $>$ 数据选择器的地址端数量时
 - 方法1: 多片数据选择器（使得地址端数量与变量相同）
 - 方法2: 降维卡诺图（使其卡诺图的维数与地址端一致）

数据选择器构成逻辑函数发生器 (2)

逻辑函数变量数 = 数据选择器的地址端数量

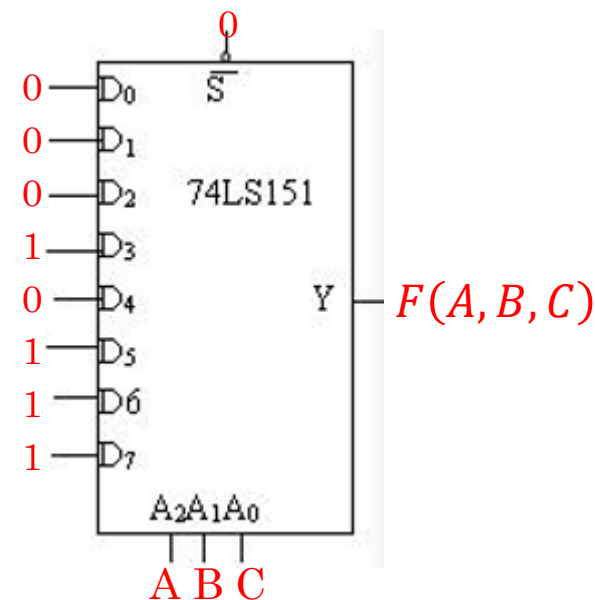
$$Y = m_0D_0 + m_1D_1 + m_2D_2 + m_3D_3 + m_4D_4 + m_5D_5 + m_6D_6 + m_7D_7$$

显然， Y 式中出现的最小项，对应的数据输入端应接**1**， Y 式中出现的最小项，对应的数据输入端应接**0**。即
 $D_3 = D_5 = D_6 = D_7 = 1$ ； $D_0 = D_1 = D_2 = D_4 = 0$ 。



$$F(A, B, C) = \sum m(3, 5, 6, 7)$$

注意：
地址端的顺序一定要与变量顺序相对应，否则就与函数最小项序号不一致，导致错误的逻辑。



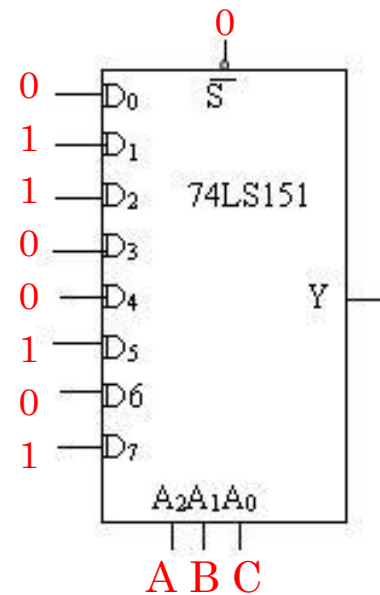
按照教材 P166 例6.3的图6.28形式来画

练习

用集成八选一数据选择器 74LS151 实现逻辑函数 $Y = AC + \overline{A}B\overline{C} + \overline{A}\overline{B}C$ 。

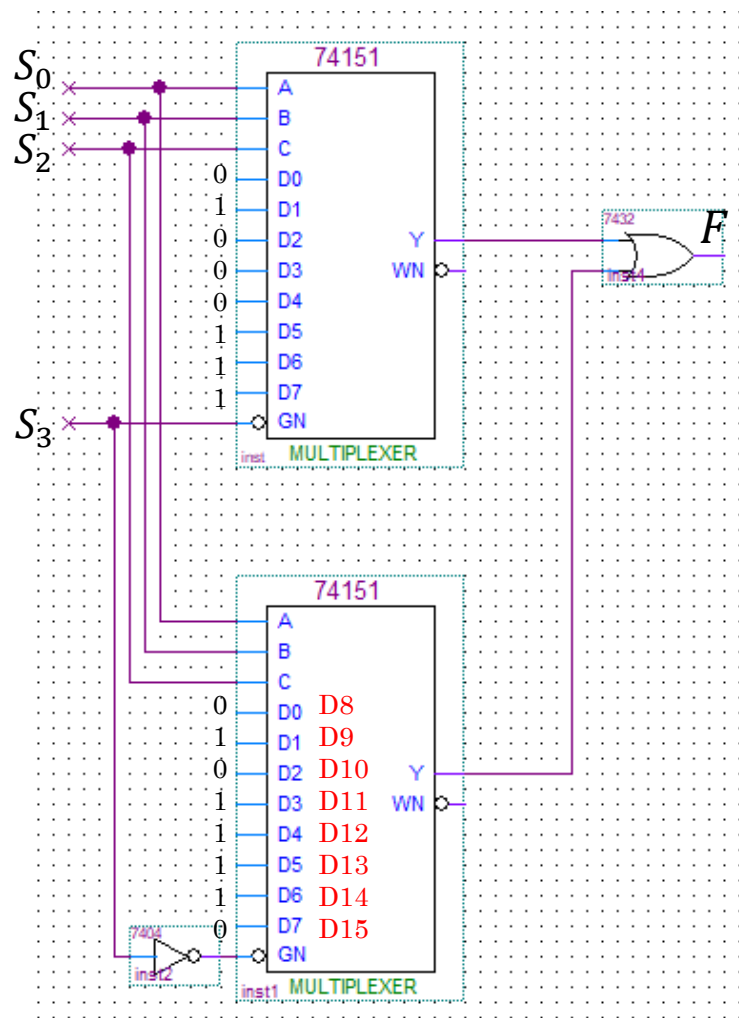
A \ BC	BC			
	00	01	11	10
0		1		1
1		1	1	

$$F(A, B, C) = \sum m(1, 2, 5, 7)$$



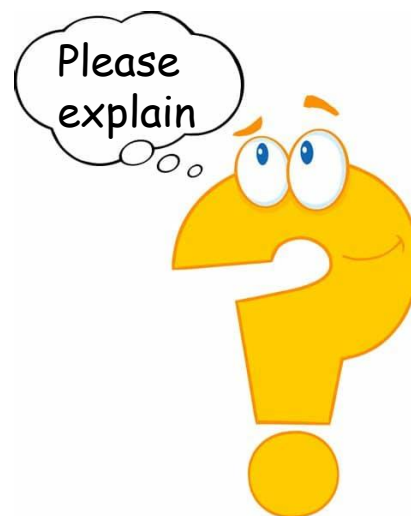
数据选择器构成逻辑函数发生器 (3)

逻辑函数变量数 > 数据选择器的地址端数量



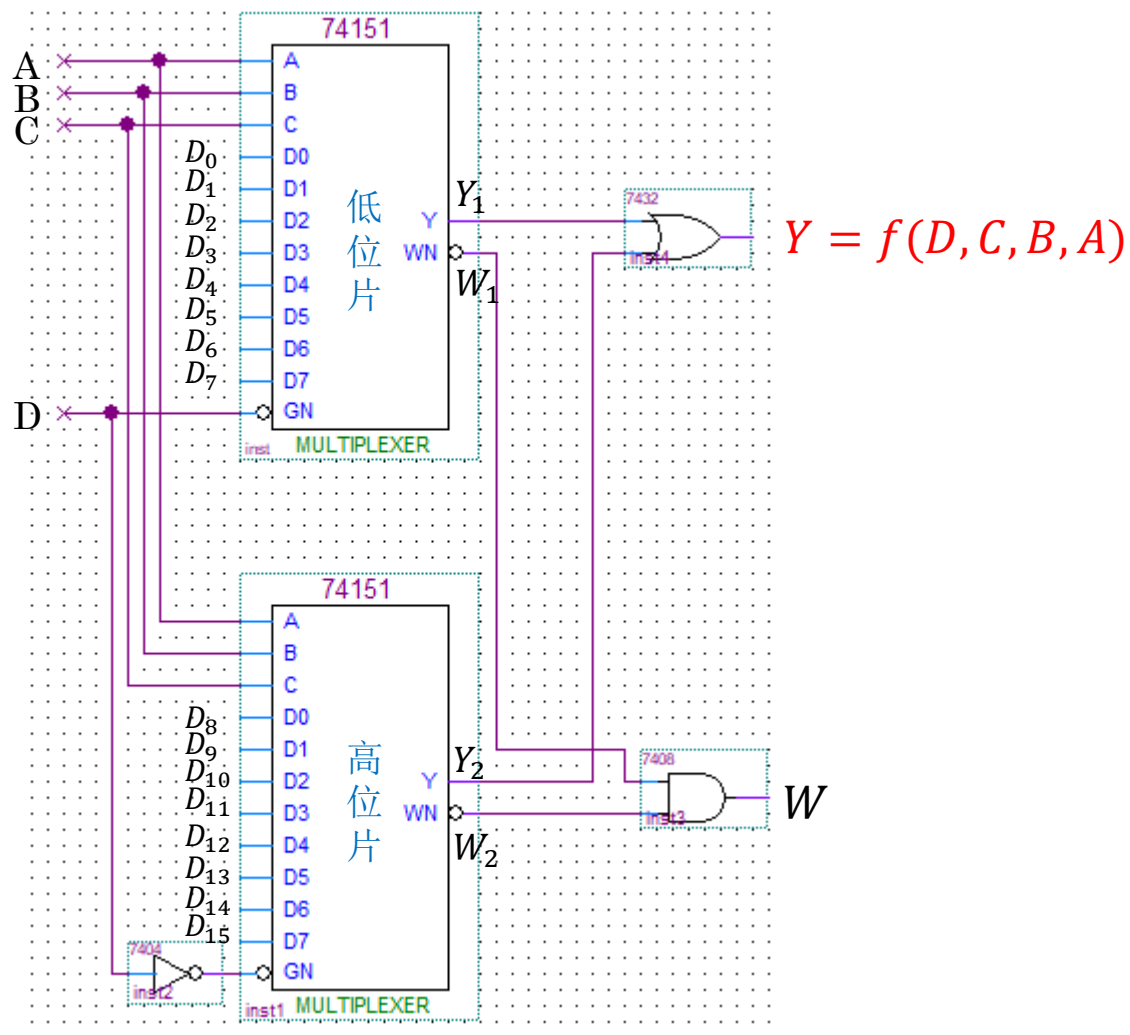
- 用两片8选1数据选择器74151实现函数

$$F(S_3, S_2, S_1, S_0) = \sum m(1, 5, 6, 7, 9, 11, 12, 13, 14)$$



教材 P166 例6.4 解1

HOW ABOUT $W = \bar{Y}$



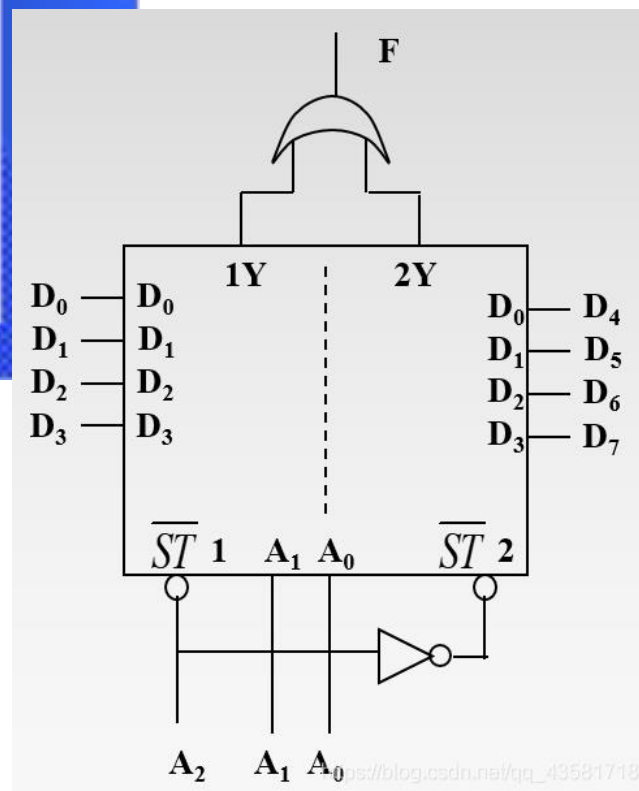
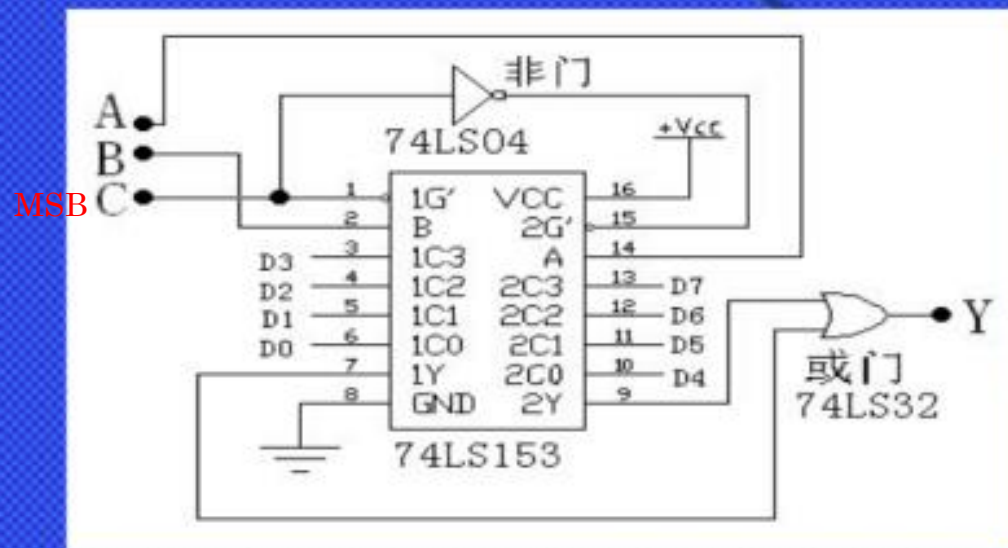
- $Y_1 = \sum_{i=0}^7 m_i D_i$
- $Y_2 = \sum_{i=0}^7 m_i D_{i+8}$
- m_i 是 C, B, A 三个变量构成的最小项
- $Y = \bar{D}Y_1 + DY_2$
- $W_1 = \bar{Y}_1, \quad W_2 = \bar{Y}_2$
- $W = \bar{Y} = \overline{\bar{D}Y_1 + DY_2}$

$$= \overline{\bar{D}Y_1} \cdot \overline{DY_2}$$

$$= (D + \bar{Y}_1) \cdot (\bar{D} + \bar{Y}_2)$$

$$= (D + W_1) \cdot (\bar{D} + W_2)$$

四选一扩展成八选一之参考图



数据选择器构成逻辑函数发生器 (4)

逻辑函数变量数 > 数据选择器的地址端数量

- 若仅仅使用一片74151实现函数

$$F(A, B, C, D) = \sum m(1, 5, 6, 7, 9, 11, 12, 13, 14)$$

则先将要实现的函数卡诺图降维。

AB \ CD	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

AB \ CD	00	01	11	10
00	0	0	1	0
01	1	1	1	1
11	0	1	0	1
10	0	1	1	0

AB \ C	00	01	11	10
0	D	D	1	D
1	0	1	$\overline{D}$	D

用到消冗余因子公式

AB	00	01	11	10
	$\overline{C}D$	$\overline{C}D + C \cdot 1$	$\overline{C} \cdot 1 + C \cdot \overline{D}$	D
		$C + D$	$\overline{C} + \overline{D}$	

降维原理
(消去某一维 X)

原卡诺图中，
 $X = 0$ ，卡诺图方格值为M
 $X = 1$ ，卡诺图方格值为N

新卡诺图对应方格值为：
 $\overline{X}M + XN$

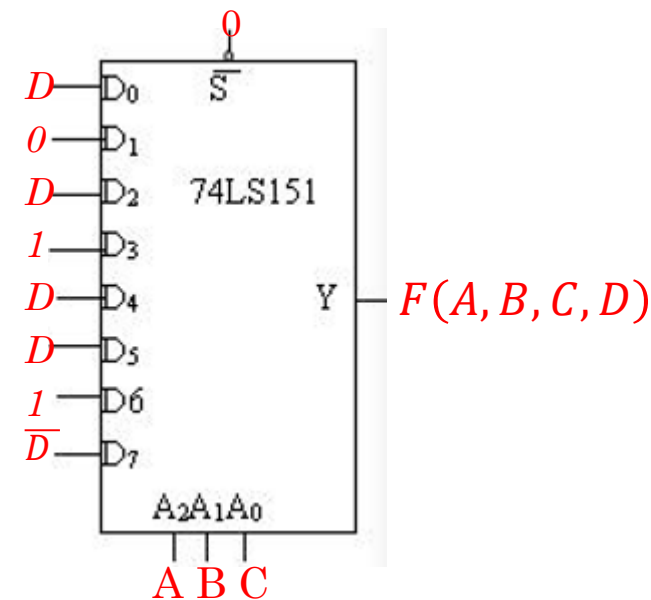
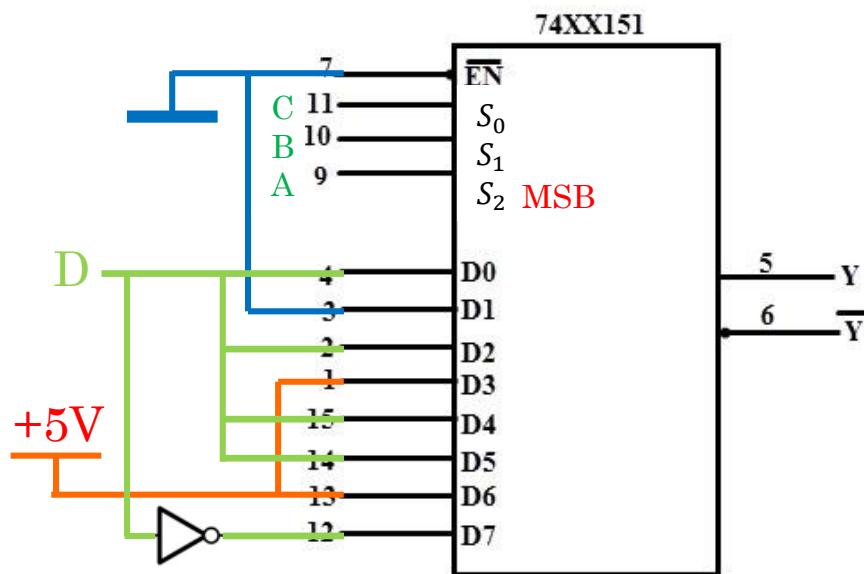
数据选择器构成逻辑函数发生器 (5)

逻辑函数变量数 > 数据选择器的地址端数量

- 若仅仅使用一片74151实现函数 $F(A, B, C, D) = \sum m(1, 5, 6, 7, 9, 11, 12, 13, 14)$

$\begin{matrix} AB \\ C \end{matrix}$	00	01	11	10
0	D 0	D 2	1 6	D 4
1	0 1	1 3	$\overline{D}$ 7	D 5

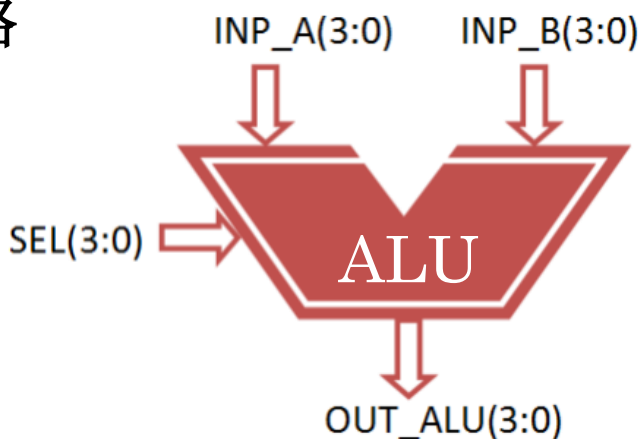
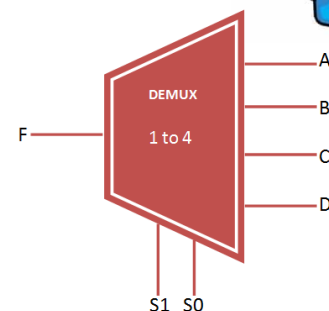
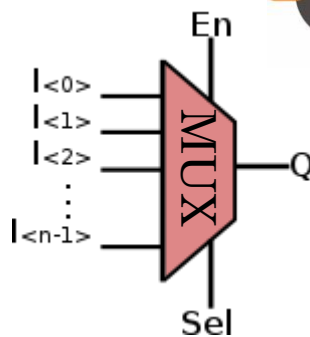
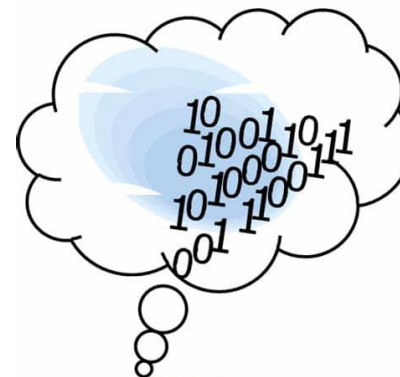
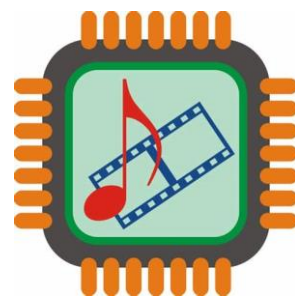
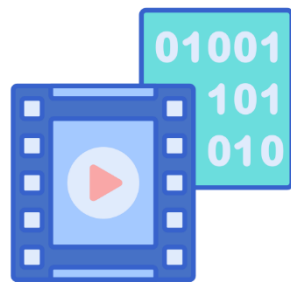
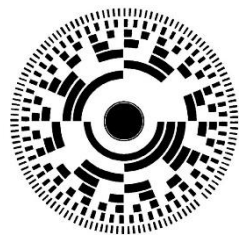
$\begin{matrix} AB \\ C \end{matrix}$	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5



按照教材 P167 例6.4解2的图6.33形式来画

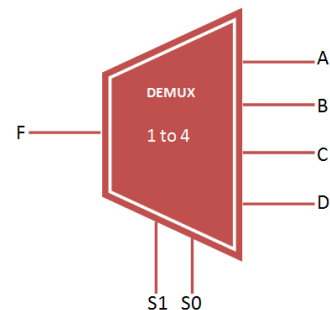
内容

- 6.1 编码器
- 6.2 译码器
- 6.3 数据选择器
- 6.4 数据分配器
- 6.5 数值比较器
- 6.6 奇偶校验位产生与校验电路
- 6.7 算术运算电路
- 6.8 算术逻辑单元ALU
- 本章小结

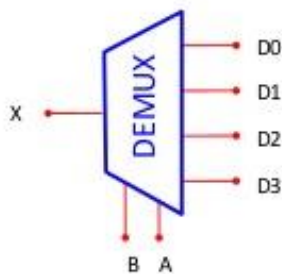
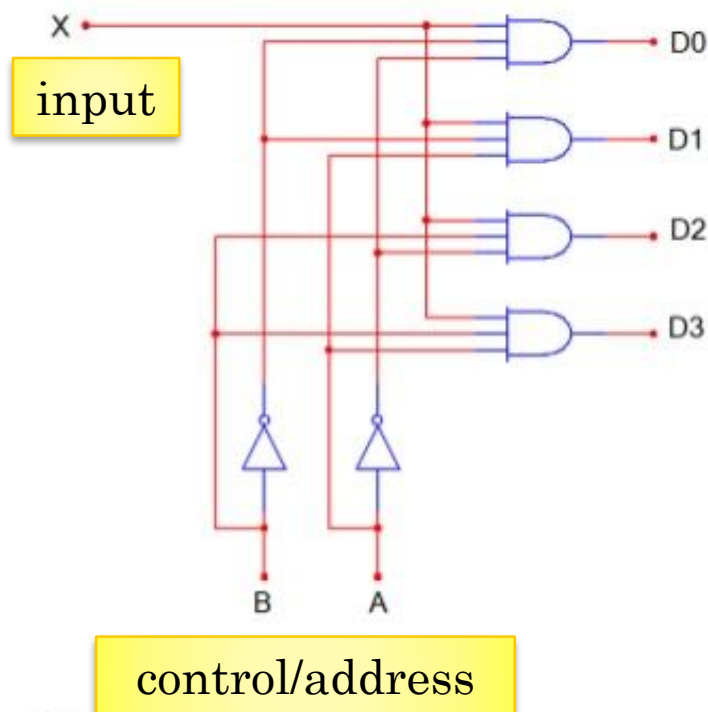


数据分配器 (DEMULTIPLEXER, DEMUX)

- 数据分配器与数据选择器的功能相反，是将输入来的一个数据送到若干个数据通道中的一个。



1-to-4 De-Multiplexer (DEMUX)



B	A	D0	D1	D2	D3
0	0	X	0	0	0
0	1	0	X	0	0
1	0	0	0	X	0
1	1	0	0	0	X

- 相当于一个输入对应多个输出的开关
- 控制端（即地址端）决定这一路数据送到哪个输出通道。

$$D_0 = X \cdot \overline{A_1} \cdot \overline{A_0}$$

$$D_1 = X \cdot \overline{A_1} \cdot A_0$$

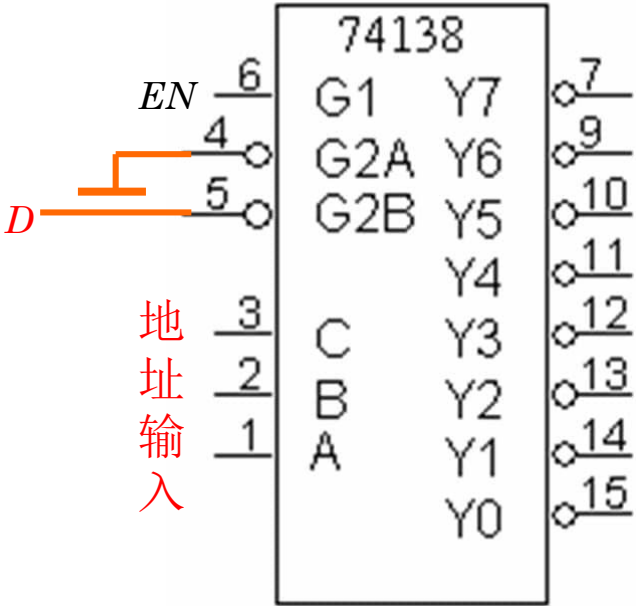
$$D_2 = X \cdot A_1 \cdot \overline{A_0}$$

$$D_3 = X \cdot A_1 \cdot A_0$$

74138译码器作为数据分配器

- 数据分配器（DEMUX）的功能实际上与带有使能端的译码器（DECODER）功能一致。
- 把其中一个使能端作为数据输入，可以构成一个1-8路数据分配器。
 - 将数据D从 $\overline{G_{2A}}$ 或者 $\overline{G_{2B}}$ 输入，经过地址选择后的输出端仍然得到D
 - 将数据D从 G_1 输入，经过地址选择后的输出端得到的却是 $\overline{D}$

输 入						输 出							
G_1	$\overline{G_{2B}}$	$\overline{G_{2A}}$	C	B	A	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
L	L	x	x	x	x	H	H	H	H	H	H	H	H
H	L	D	L	L	L	D	H	H	H	H	H	H	H
H	L	D	L	L	H	H	D	H	H	H	H	H	H
H	L	D	L	H	L	H	H	D	H	H	H	H	H
H	L	D	L	H	H	H	H	H	D	H	H	H	H
H	L	D	H	L	L	H	H	H	H	D	H	H	H
H	L	D	H	L	H	H	H	H	H	H	D	H	H
H	L	D	H	H	L	H	H	H	H	H	H	D	H
H	L	D	H	H	H	H	H	H	H	H	H	H	D





4.2 译码器/数据分配器



4.2.3 数据分配器

数据分配是将一个数据源来的数据根据需要送到多个不同的通道上去,实现数据分配功能的逻辑电路称为数据分配器。它的作用相当于多个输出的单刀多掷开关,其示意图如图 4.2.9 所示。

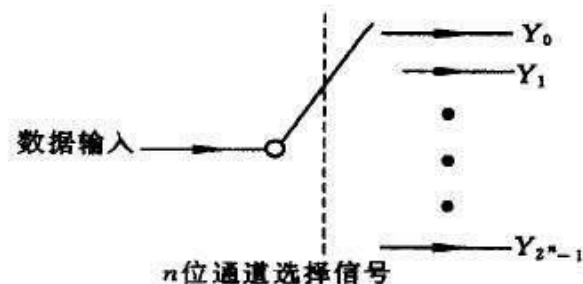


图 4.2.9 数据分配器示意图

数据分配器可以用唯一地址译码器实现。

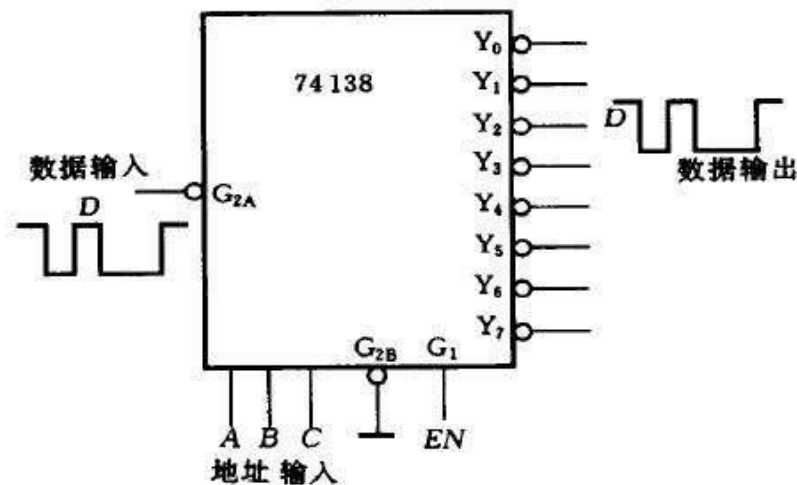
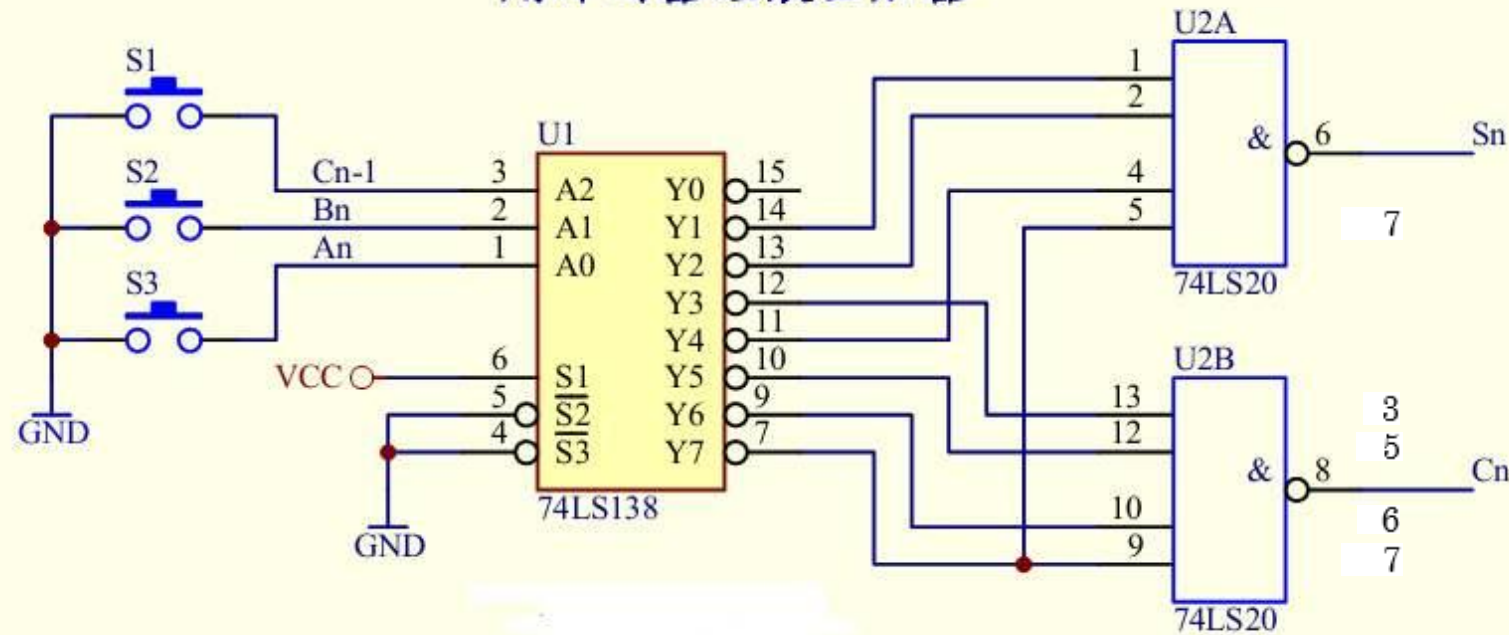


图 4.2.10 用 74138 作为数据分配器

用译码器组装全加器

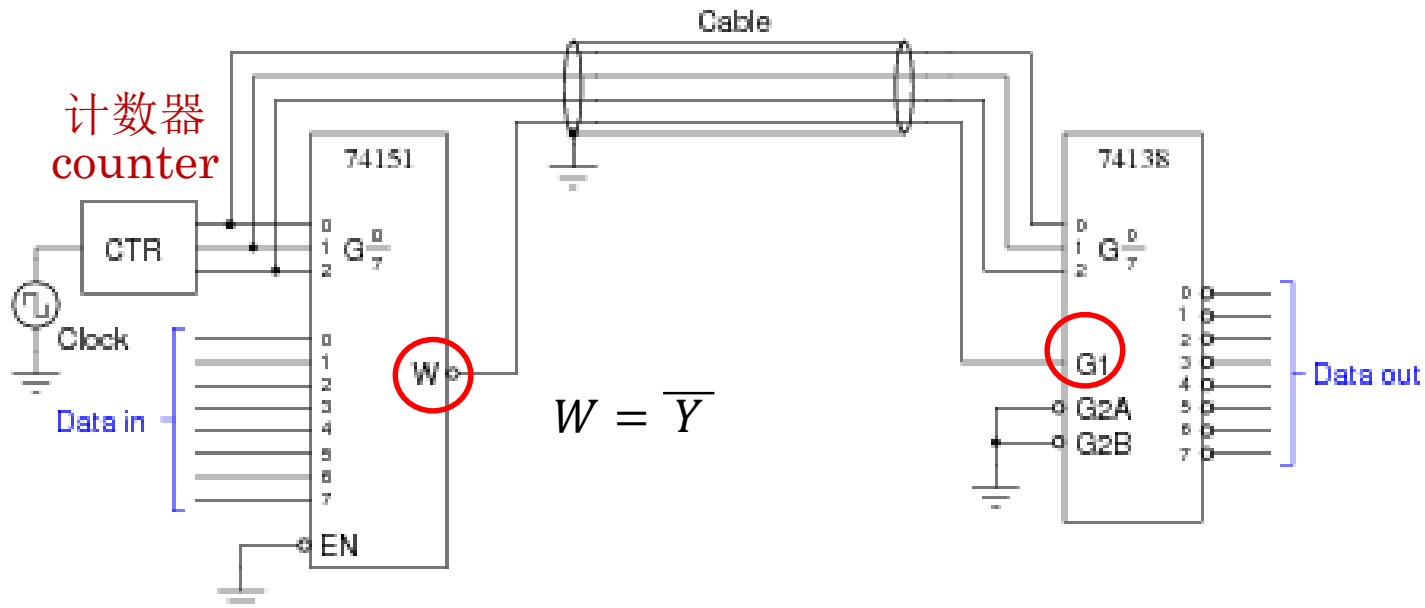


$$S_n = A \oplus B \oplus C_{-1}$$

BC	00	01	11	10
A				
0	0	1	3	2
1	4	5	7	6

$$C_n = AB + (A + B)C_{-1}$$

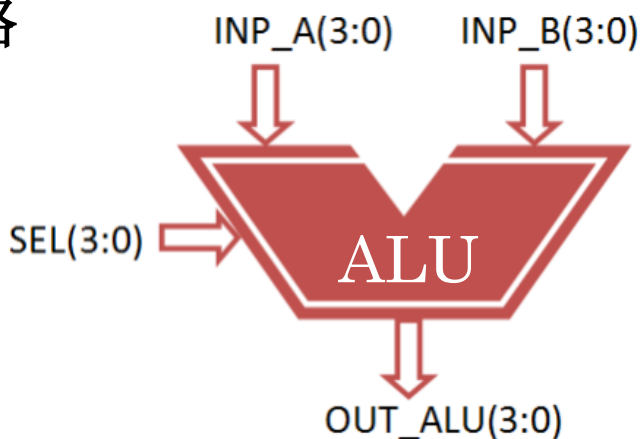
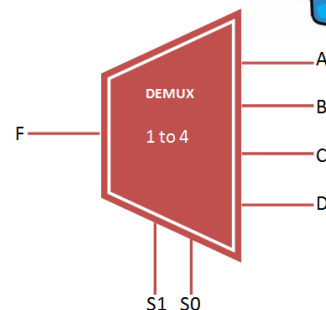
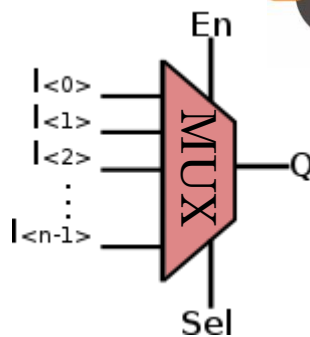
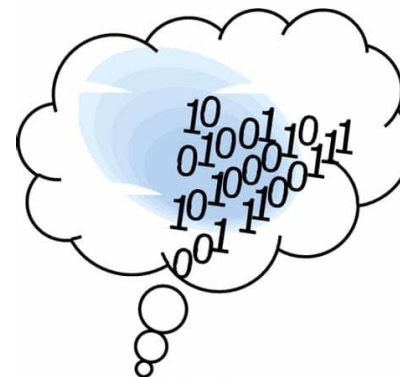
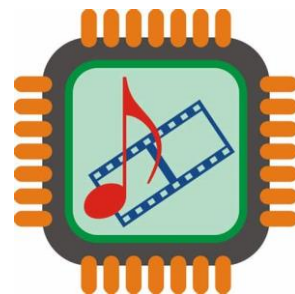
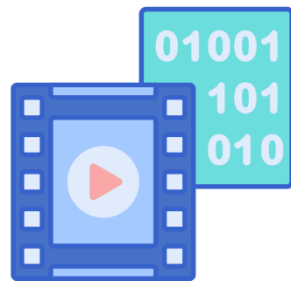
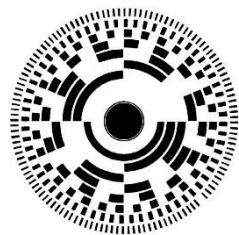
BC	00	01	11	10
A				
0	0	1	3	2
1	4	5	7	6



Multiplexers and Demultiplexers Worksheet - Digital Circuits

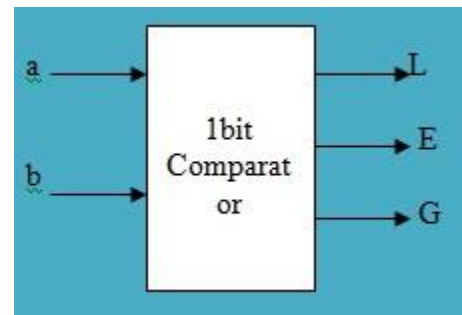
内容

- 6.1 编码器
- 6.2 译码器
- 6.3 数据选择器
- 6.4 数据分配器
- 6.5 数值比较器
- 6.6 奇偶校验位产生与校验电路
- 6.7 算术运算电路
- 6.8 算术逻辑单元ALU
- 本章小结



数值比较器

MAGNITUDE COMPARATOR

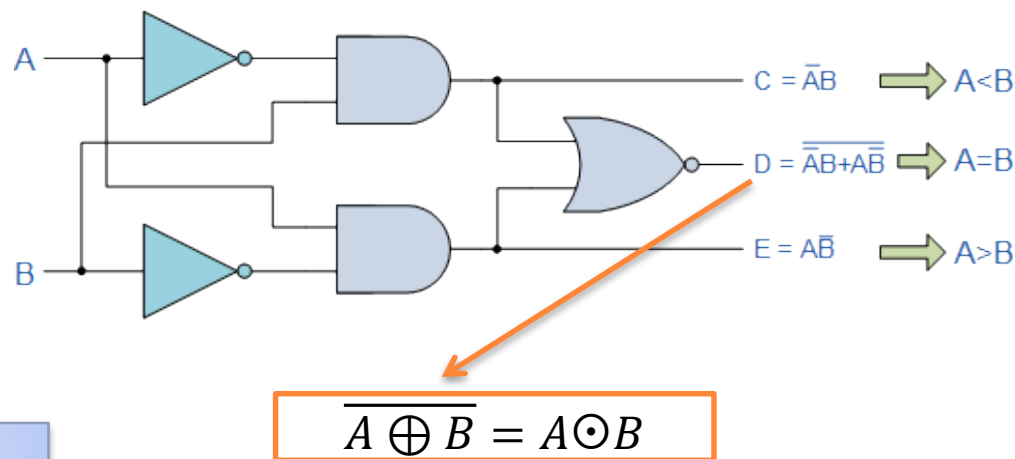


- 数字系统尤其是计算机中，经常需要比较两个数值的大小。
- 数值比较器就是能够对两个数值 A 和 B 比较大小的逻辑电路。
- 比较结果： $G: A > B$ 或者 $L: A < B$ 或者 $E: A = B$

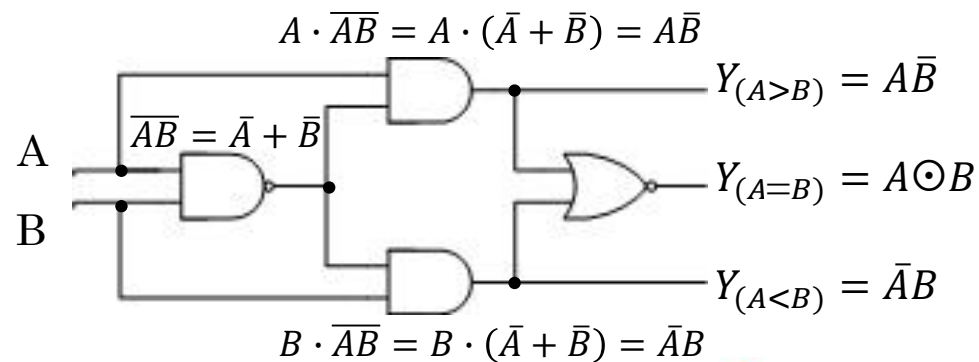
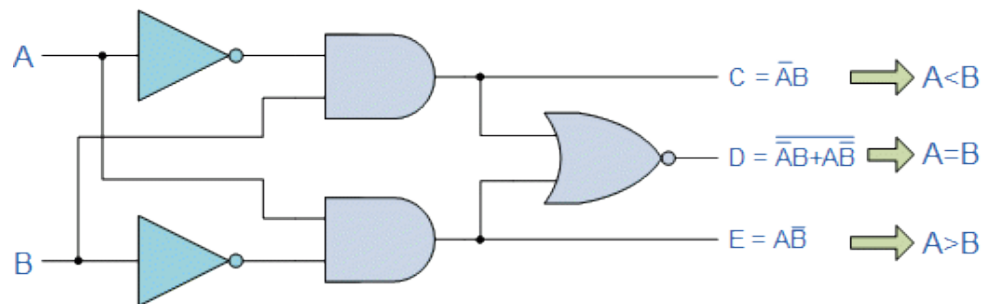
1位二进制数值比较器（1 bit magnitude comparator）

A_0	B_0	L	E	G
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

$$F(A = B) = \overline{F(A < B) + F(A > B)}$$



1位二进制数值比较器（三种逻辑图画法）



P \ Q	0	1
0		
1	1	

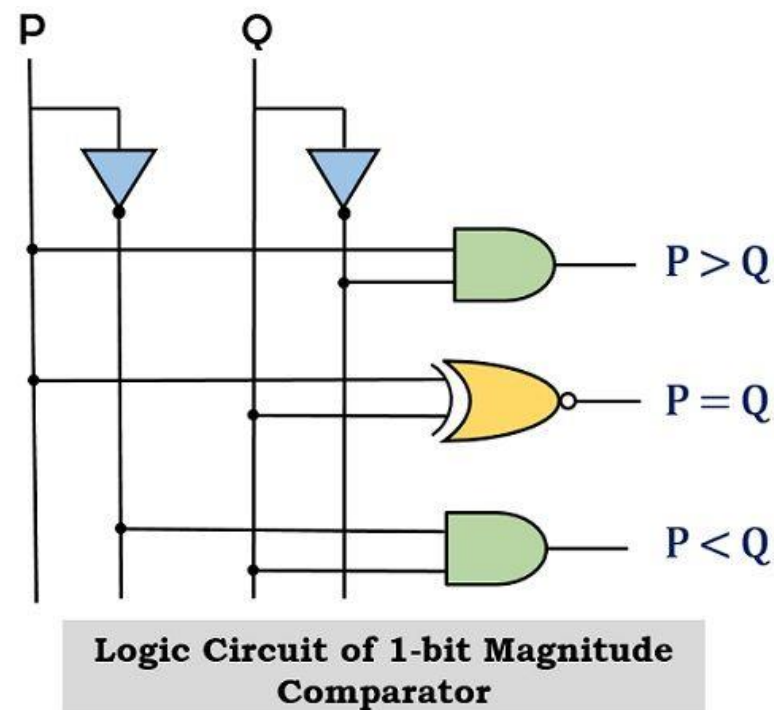
$$P > Q \rightarrow P\bar{Q}$$

P \ Q	0	1
0	1	
1		1

$$P = Q \rightarrow \bar{P}\bar{Q} + PQ$$

P \ Q	0	1
0		1
1		

$$P < Q \rightarrow \bar{P}Q$$



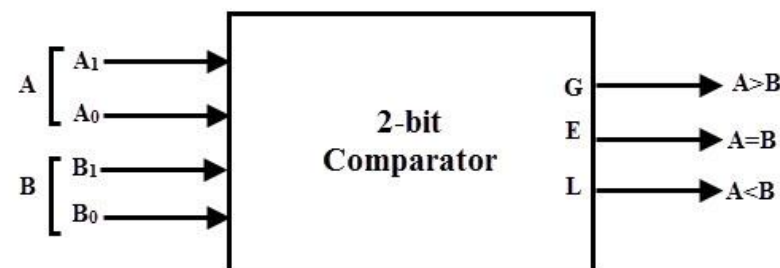
2位数值比较器

2-BIT MAGNITUDE COMPARATOR

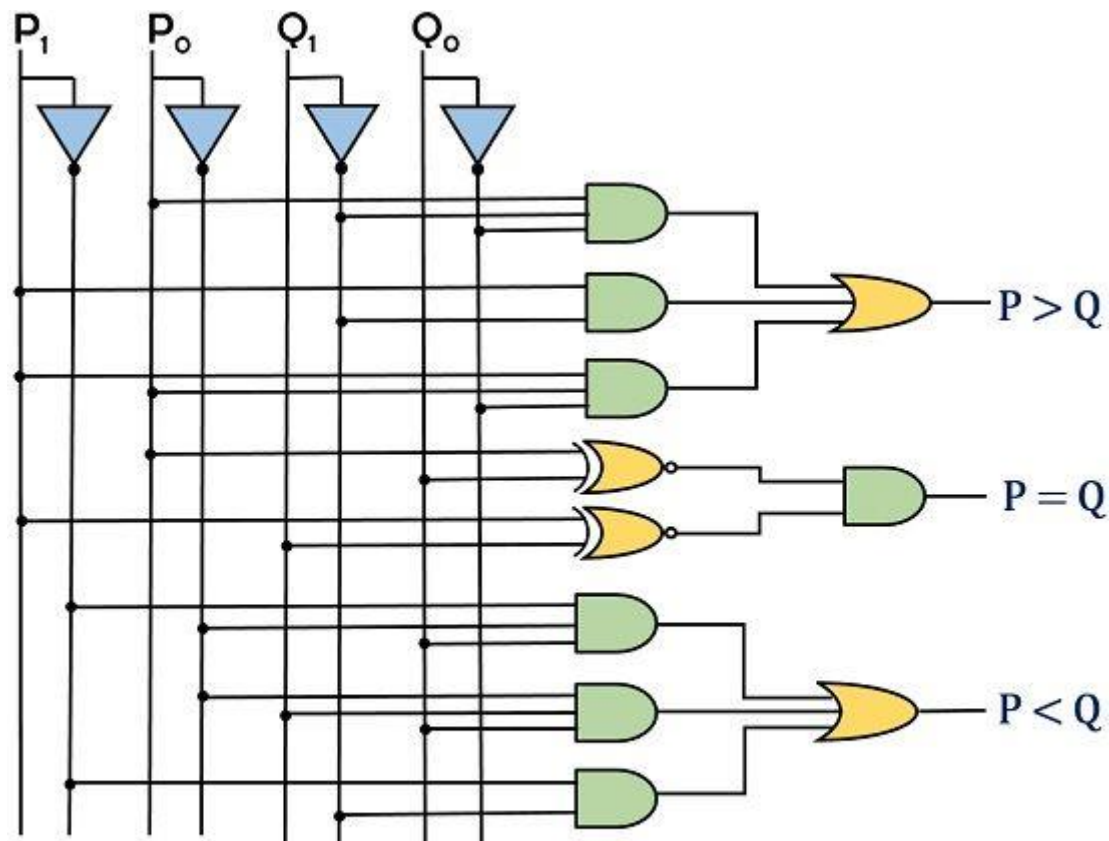
- 2位的数值比较器，有4个输入变量，也可以按照1-bit comparator的方法设计。

Table 1. Truth Table of 2-Bit Magnitude Comparator

INPUT				OUTPUT		
A1	A0	B1	B0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0



2位数值比较器 (2)



Logic Circuit of 2-bit Magnitude Comparator

$P_1P_0 \backslash Q_1Q_0$	00	01	11	10
00				
01	1			
11	1	1		1
10	1	1		

$$P > Q \rightarrow P_0 \bar{Q}_1 \bar{Q}_0 + P_1 \bar{Q}_1 + P_1 P_0 \bar{Q}_0$$

$P_1P_0 \backslash Q_1Q_0$	00	01	11	10
00	1			
01		1		
11			1	
10				1

$$P = Q \rightarrow (P_0 \odot Q_0) (P_1 \odot Q_1)$$

$P_1P_0 \backslash Q_1Q_0$	00	01	11	10
00		1	1	1
01			1	1
11				
10			1	

$$P < Q \rightarrow \bar{P}_1 \bar{P}_0 Q_0 + \bar{P}_0 Q_1 Q_0 + \bar{P}_1 Q_1$$

多位数值比较器逻辑表达式推导

- 对于更多位的数值比较器，由于变量数的增加，列真值表很麻烦。
 - 需要针对多位数值比较器的工作情况，找出规律，直接写出比较结果的函数表达式，以简化设计方法。
- 按照两个数比较大小的方法
 - 总是先比较其最高位，如果最高位不一样，就可以知道这两个数的大小，
 - 只有最高位相等，两个数的大小才取决于次高位，以此类推。

$$F_{A>B} = A_n \overline{B_n} + (A_n \odot B_n) \overline{A_{n-1} B_{n-1}} + \cdots + (A_n \odot B_n) (A_{n-1} \odot B_{n-1}) \cdots (A_1 \odot B_1) \overline{A_0 B_0}$$

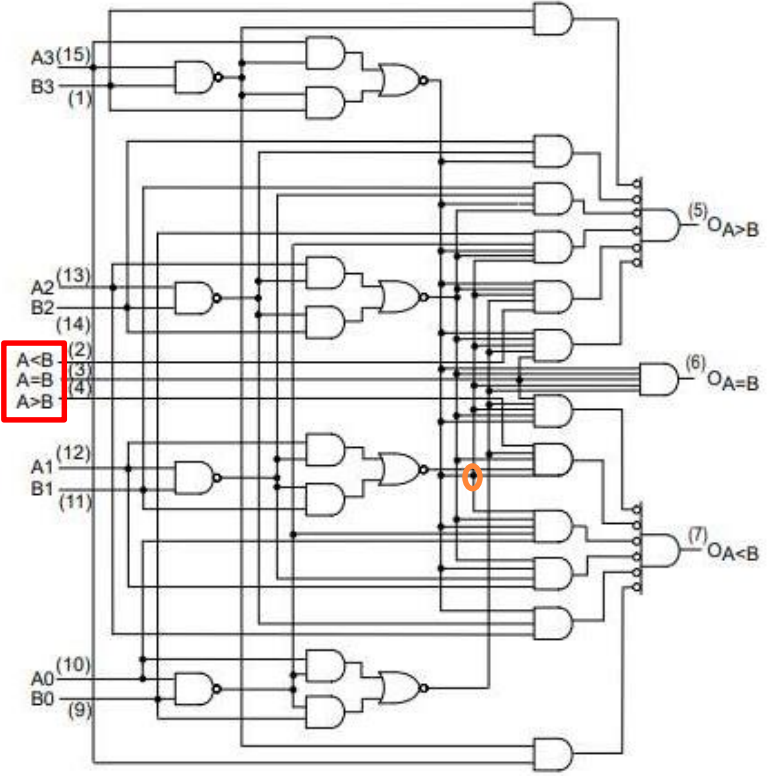
$$F_{A=B} = (A_n \odot B_n) (A_{n-1} \odot B_{n-1}) \cdots (A_1 \odot B_1) (A_0 \odot B_0)$$

$$F_{A<B} = \overline{A_n} B_n + (A_n \odot B_n) \overline{A_{n-1} B_{n-1}} + \cdots + (A_n \odot B_n) (A_{n-1} \odot B_{n-1}) \cdots (A_1 \odot B_1) \overline{A_0 B_0}$$

4位数值比较器 (7485)

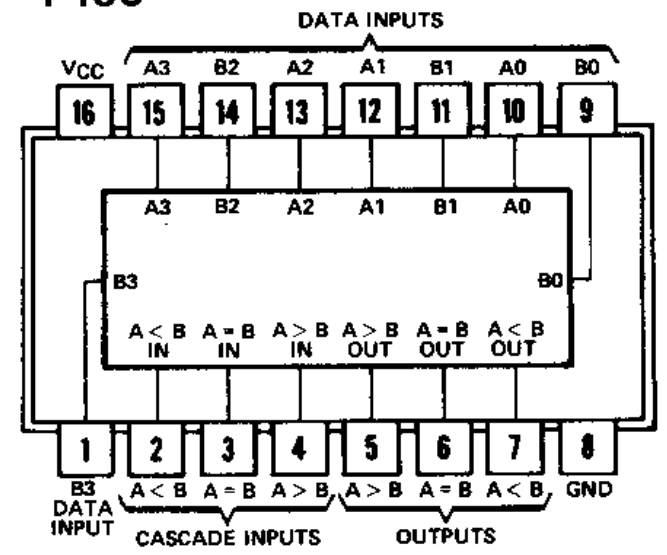
当输入的两个4位二进制数的数码完全相同时，两个数的比较结果输出就取决于这三个输入端

级联输入



7485

COMPARING INPUTS				CASCADING INPUTS			OUTPUTS		
A ₃ ,B ₃	A ₂ ,B ₂	A ₁ ,B ₁	A ₀ ,B ₀	I _{A>B}	I _{A<B}	I _{A=B}	O _{A>B}	O _{A<B}	O _{A=B}
A ₃ >B ₃	X	X	X	X	X	X	H	L	L
A ₃ <B ₃	X	X	X	X	X	X	L	H	L
A ₃ =B ₃	A ₂ >B ₂	X	X	X	X	X	H	L	L
A ₃ =B ₃	A ₂ <B ₂	X	X	X	X	X	L	H	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ >B ₁	X	X	X	X	H	L	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ <B ₁	X	X	X	X	L	H	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ >B ₀	X	X	X	H	L	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ <B ₀	X	X	X	L	H	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ =B ₀	H	L	L	H	L	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ =B ₀	L	H	L	L	H	L
A ₃ =B ₃	A ₂ =B ₂	A ₁ =B ₁	A ₀ =B ₀	X	X	H	L	L	H



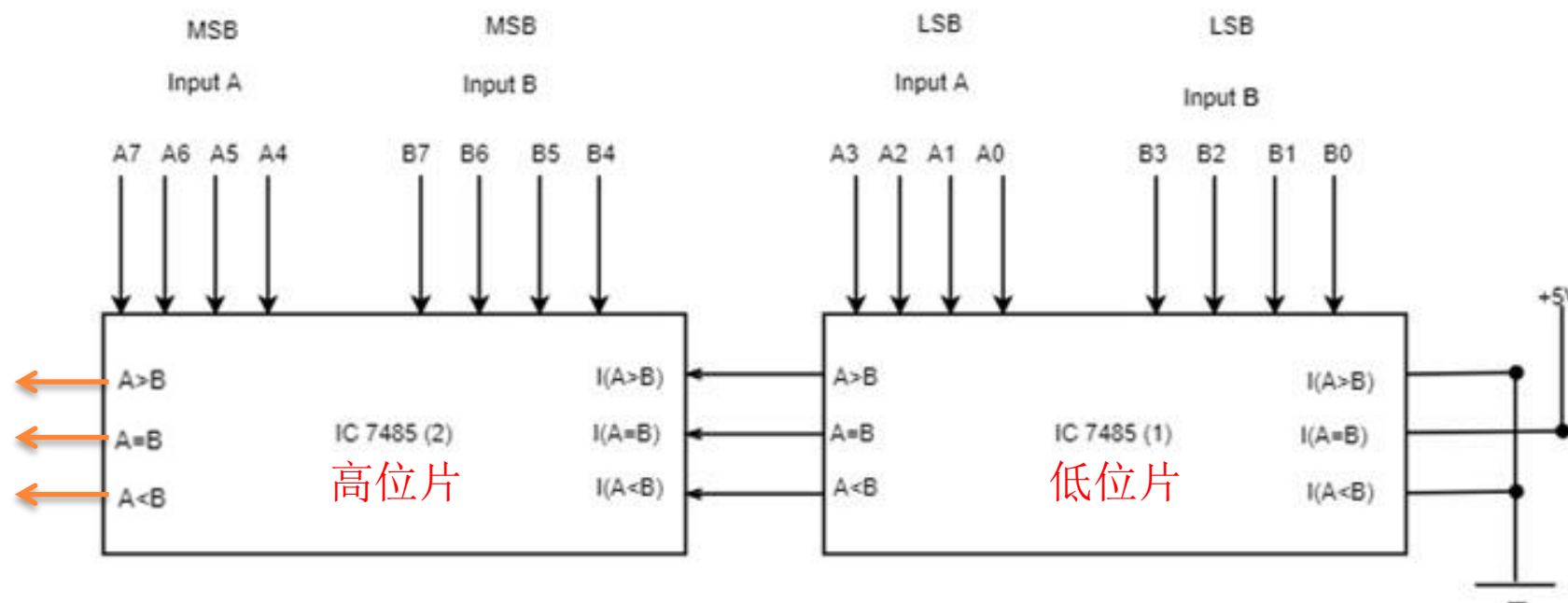
7485功能表

比较输入				级联输入			输出		
$A_3 B_3$	$A_2 B_2$	$A_1 B_1$	$A_0 B_0$	$I_{(A>B)}$	$I_{(A<B)}$	$I_{(A=B)}$	$Y_{(A>B)}$	$Y_{(A<B)}$	$Y_{(A=B)}$
$A_3>B_3$	×	×	×	×	×	×	1	0	0
$A_3<B_3$	×	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2>B_2$	×	×	×	×	×	1	0	0
$A_3=B_3$	$A_2<B_2$	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1>B_1$	×	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1<B_1$	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0>B_0$	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0<B_0$	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	1	0	0	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	1	0	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	×	×	1	0	0	1
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	1	1	0	0	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	0	0	1	1	0

但是我们课本上的功能表没有描述后两行功能，所以不会在试卷中出现。

$A>B$ 和 $A<B$ 输出端相同，可自行判断 $A=B$

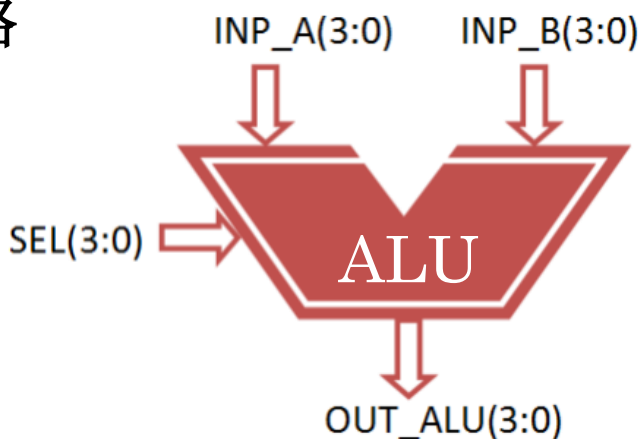
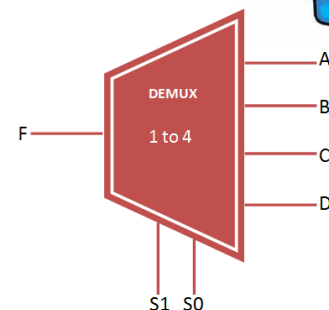
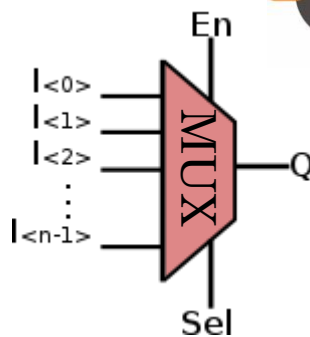
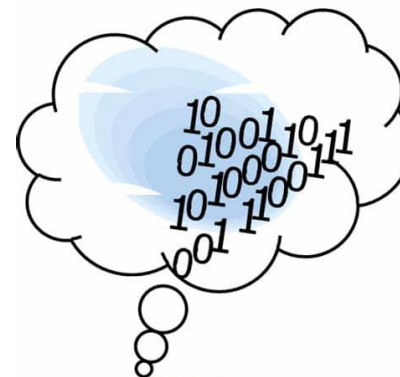
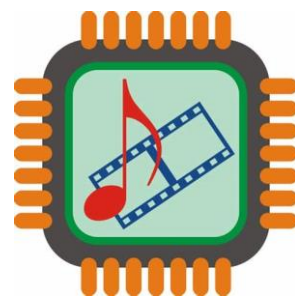
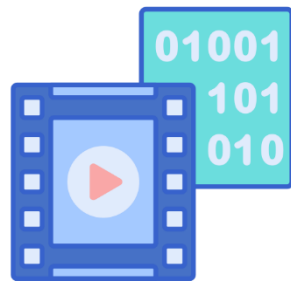
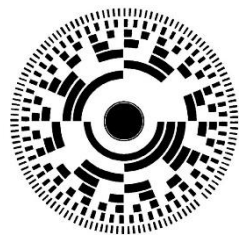
2个8位二进制数比较大小 用2片4位数值比较器7485



- 两个数的高4位若相同，输出结果就取决于高位片的三个级联输入端（连接到低位片的比较结果相应输出上）
- 低位片的3个级联输入端分别接0-1-0
 - 表示当低4位也相同时，这两个数值就相同。

内容

- 6.1 编码器
- 6.2 译码器
- 6.3 数据选择器
- 6.4 数据分配器
- 6.5 数值比较器
- 6.6 奇偶校验位产生与校验电路
- 6.7 算术运算电路
- 6.8 算术逻辑单元ALU
- 本章小结



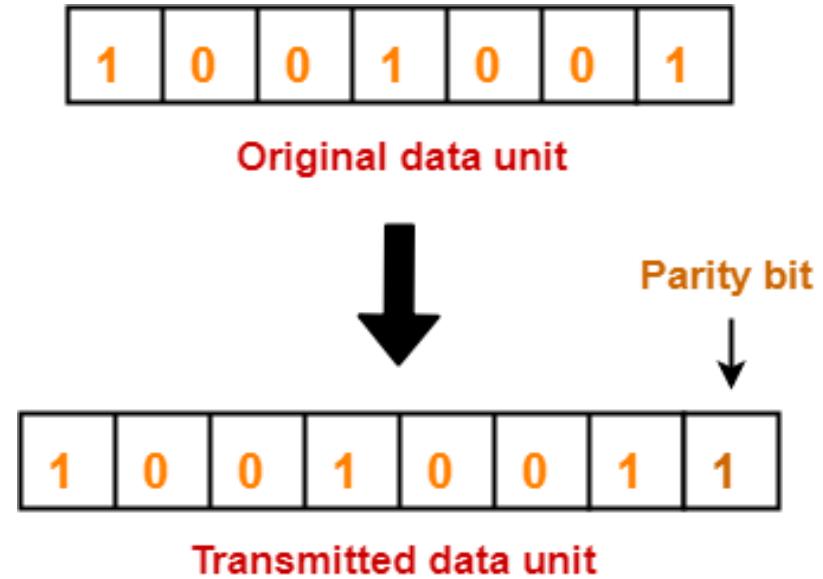
奇偶校验位

PARITY CHECK BIT

	数据和校验位合起来的数码中1的个数
奇校验	奇数
偶校验	偶数

- 奇偶校验分为①校验位的产生和②校验两部分，
 - 所用的电路是同一种电路，因此称为奇偶校验位产生与校验电路。
- 奇偶校验电路是能够对输入数据信号中1的奇偶性进行判断的电路
 - 它包括奇偶校验位的产生电路和奇偶校验电路。
 - 在数据本身的基础上，增加一个校验位，使数据和校验位合起来的数码中1的个数为奇数（奇校验）或者偶数（偶校验）。
- 若与约定的奇偶性一致，则认为信息传输正确，否则要重新传输数据。
- 但是不能发现不改变传输码中包含1个数的奇偶性错误

SINGLE BIT PARITY



Original Data	Even Parity	Odd Parity
0 0 0 0 0 0 0	0	1
0 1 0 1 1 0 1	1	0
0 1 0 1 0 1 0	0	1
1 1 1 1 1 1 1	0	1
1 0 0 0 0 0 0	1	0
0 1 0 0 1 0 0	1	0

奇偶校验电路和校验位产生电路

PARITY CHECKER AND PARITY GENERATOR

- 奇偶校验的基本运算是异或运算。

设有 n 个输入变量 $X_1, X_2, \dots, X_n$ ，函数 $F_{odd} = X_1 \oplus X_2 \oplus \dots \oplus X_n$ 的逻辑功能为：

- 当输入变量为1的个数是奇数时， $F_{odd} = 1$ ；
- 当输入变量为1的个数是偶数时， $F_{odd} = 0$ ；

- $F_{odd} = 1$ 判定校验1的个数为奇数，同时 F_{odd} 本身产生了偶校验位。反之亦然。

Parity checker

奇校验电路实现该功能，

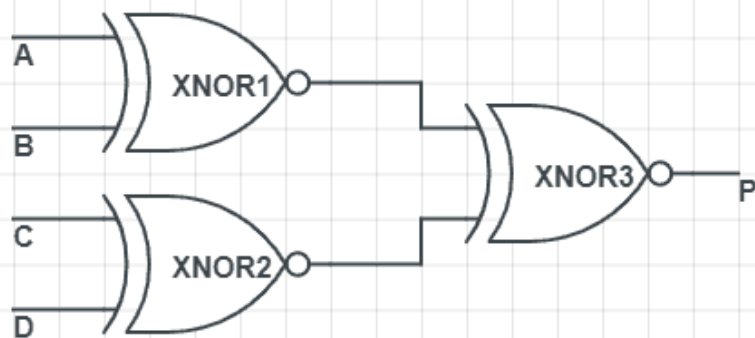
输出端加一个非门→

偶校验电路

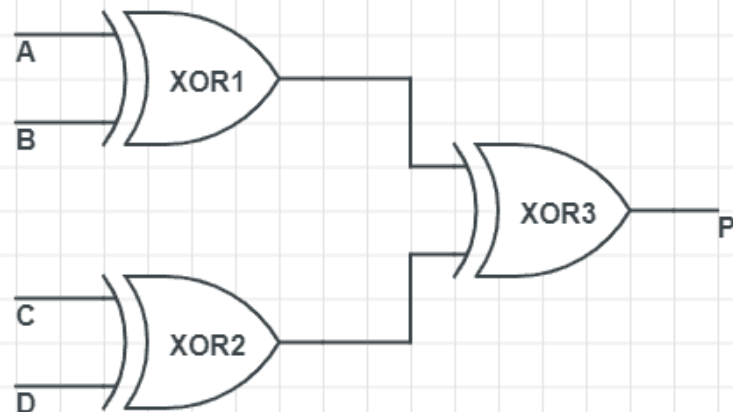
通常合二为一，称为

奇偶校验电路

4-bit odd parity generator

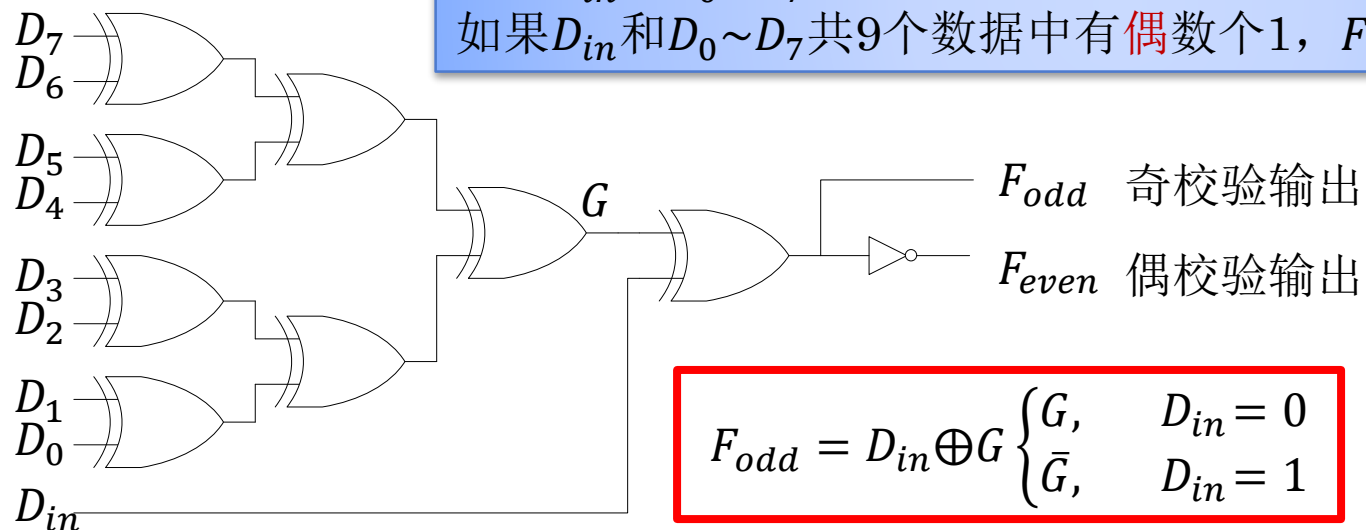


4-bit even parity generator



PARITY GENERATOR/CHECKER

与书上图6.40相对应的图



作用:

只能检测出1位出错
或奇数个位错误

局限性:

不能检测偶数个位出错,
也无法对出错位定位。

应用:

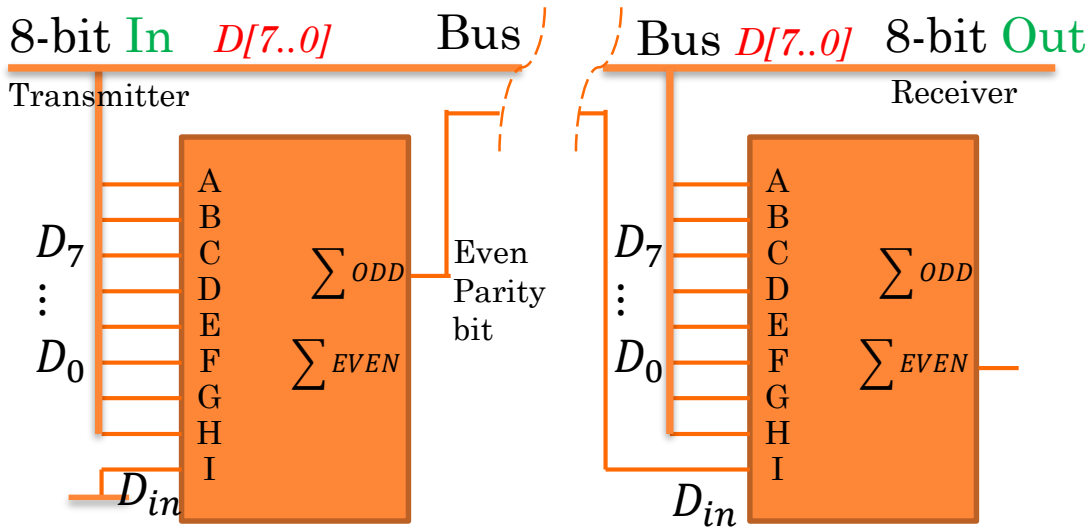
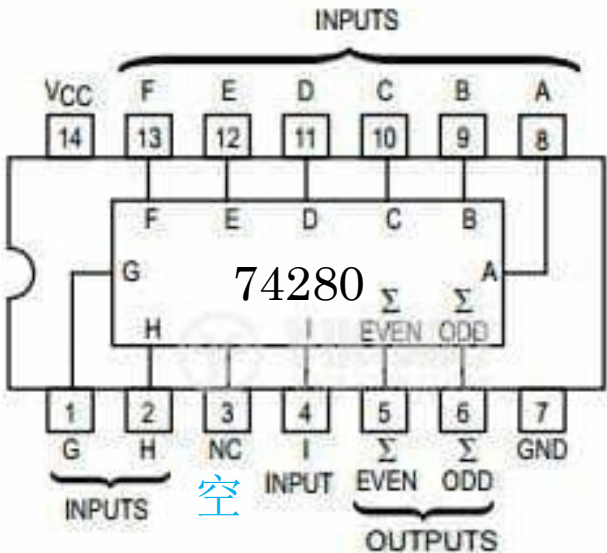
广泛应用于误码率不高
的信息传输系统中。

中规模集成奇偶校验电路74280

逻辑表达式 $F_{ODD} = A \oplus B \oplus C \oplus D \oplus E \oplus F \oplus G \oplus H \oplus I$

$F_{EVEN} = \overline{A \oplus B \oplus C \oplus D \oplus E \oplus F \oplus G \oplus H \oplus I}$

- 既可作为奇偶校验位的产生（发送端）
- 也可作为奇偶校验（接收端）



Even parity error detection

	$D_{in} = 0$	$D_{in} = 1$
发送端	F_{ODD} 用作产生偶校验位	F_{ODD} 用作产生奇校验位
接收端	若无差错, $F_{EVEN} = 1$ 若有差错, $F_{EVEN} = 0$	若无差错, $F_{ODD} = 1$ 若有差错, $F_{ODD} = 0$

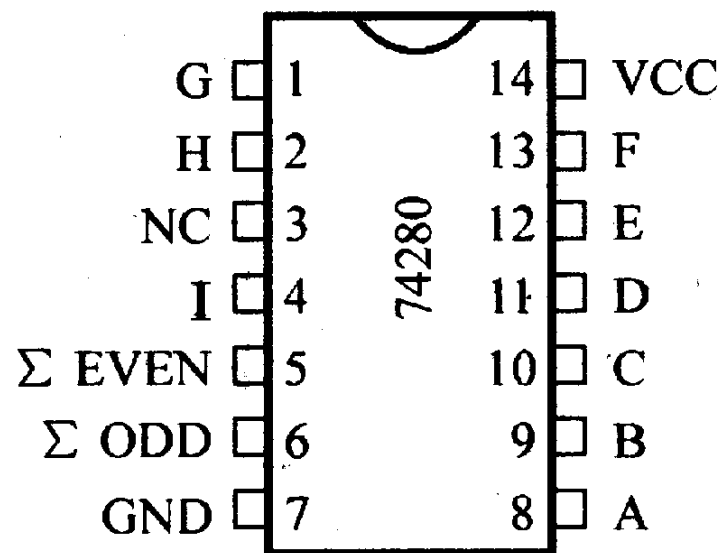
A	B	C	D	E	F	G	H	I	F_{odd}
1	1	0	1	0	0	0	0	0	1
1	1	0	1	0	0	0	0	1	0

74LS280、74S280、74F280、74ALS280、74HC280

9 位奇偶数产生器/校验器

功能表

输入 A 至 I 为高的数目	输出	
	偶	奇
0, 2, 4, 6, 8	H	L
1, 3, 5, 7, 9	L	H

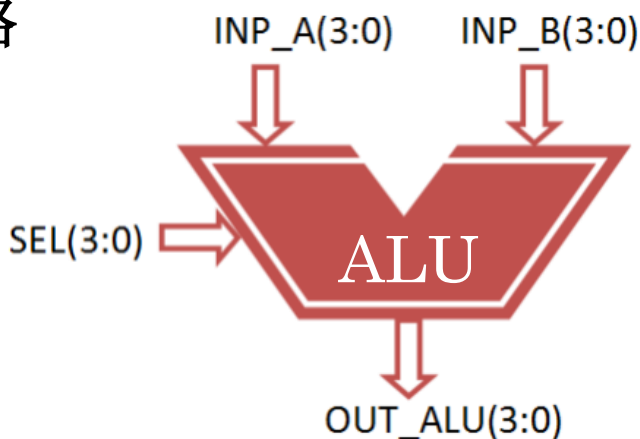
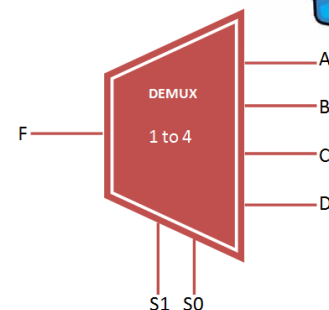
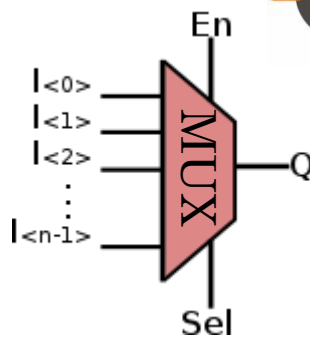
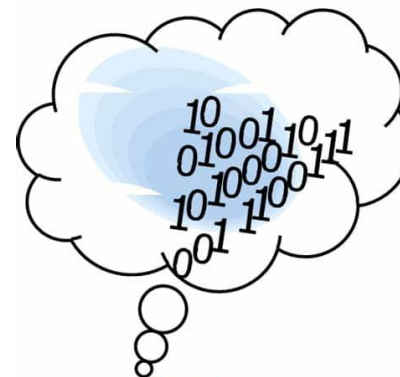
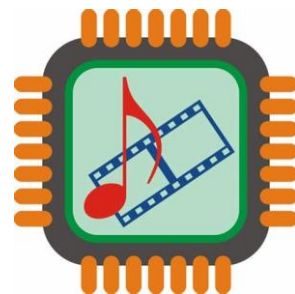
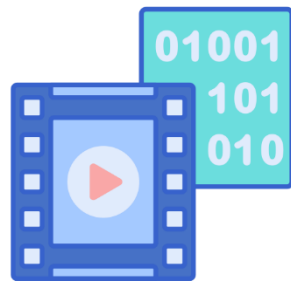
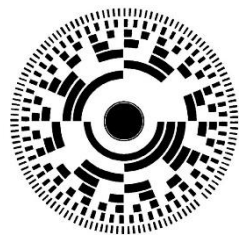


集成数据选择器与分配器、数值比较器总结

分类	Chip	In-Out	辅助控制端	备注
数据选择器	74157	四2-1 MUX	1个控制端select 1个使能端 $\overline{Strobe}$	
	74153	双4-1 MUX	两个公用控制端A,B 两个使能端 $\overline{1G}$, $\overline{2G}$	
	74151	8-1 MUX	3 data select, 1个使能端 $\overline{G}$	输出除了Y, 还有 $\overline{Y}$
数据分配器	74138	3-8线	$E_3, \overline{E_2}, \overline{E_1}$	74138还可以构成数据分配器
数值比较器	7485	2个4位数值比较输入 + 3个级联输入, 3个比较结果输出		两个4位数值比较
奇偶校验电路	74280	8个输入 + 1个 D_{in} , 2个输出		总共14个针脚, 有一个空针脚

内容

- 6.1 编码器
- 6.2 译码器
- 6.3 数据选择器
- 6.4 数据分配器
- 6.5 数值比较器
- 6.6 奇偶校验位产生与校验电路
- 6.7 算术运算电路
- 6.8 算术逻辑单元ALU
- 本章小结



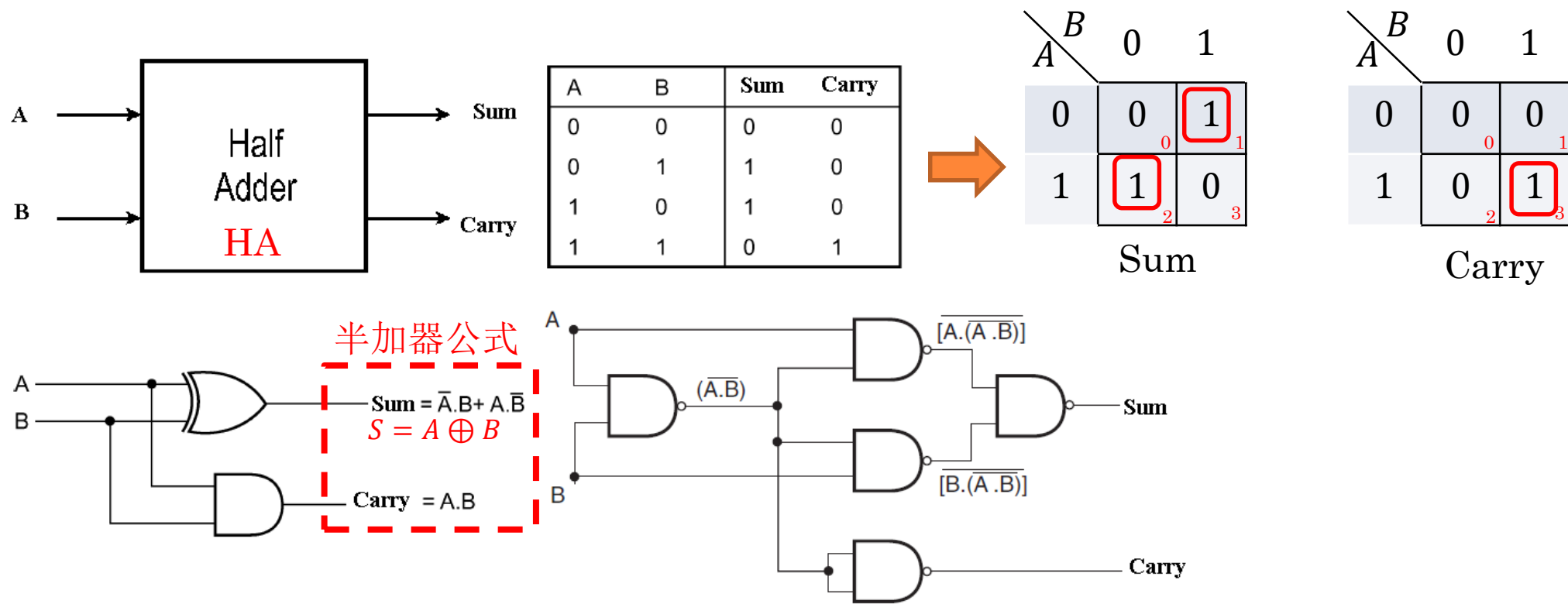
算术运算电路

- 算术运算是数字系统、计算机的基本功能，也是其基本单元之一。
 - 1位二进制加法器
 - 逐位进位的全加器
 - 超前进位的4位二进制全加器74283
 - 减法运算
 - 补码的加、减法共用电路

半加器 (HALF ADDER) 不考虑进位

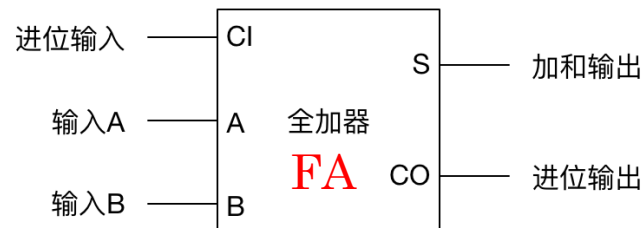
- 半加器有一个异或门与一个与门组成。

定义	代表的意义
A	被加数
B	加数
C_o	向更高位的进位
S	相加得到的和



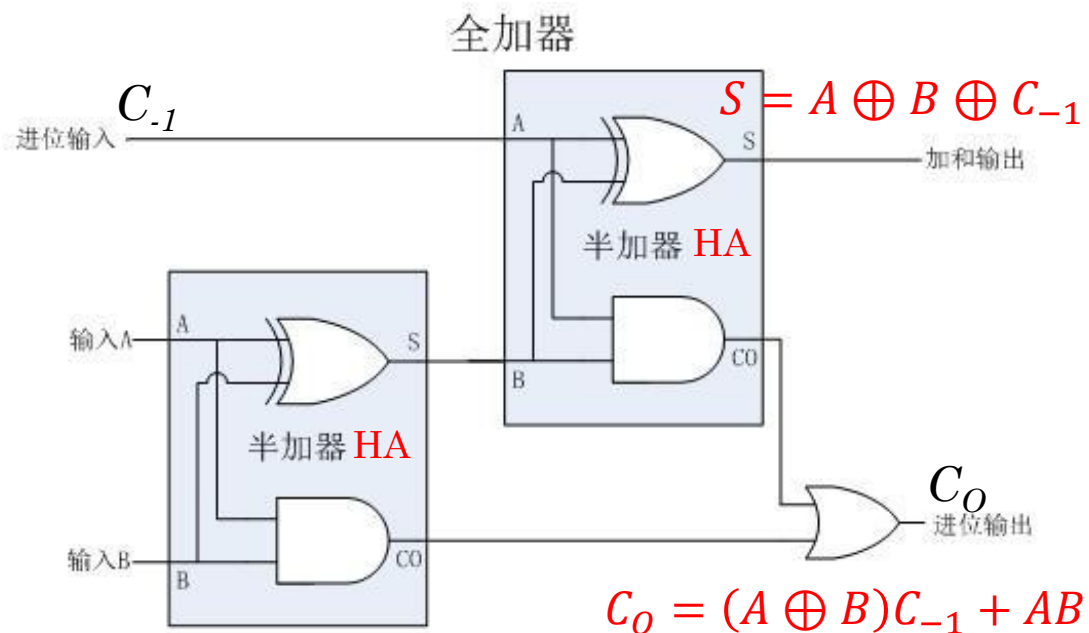
Half adder using NAND gates

1位二进制加法器



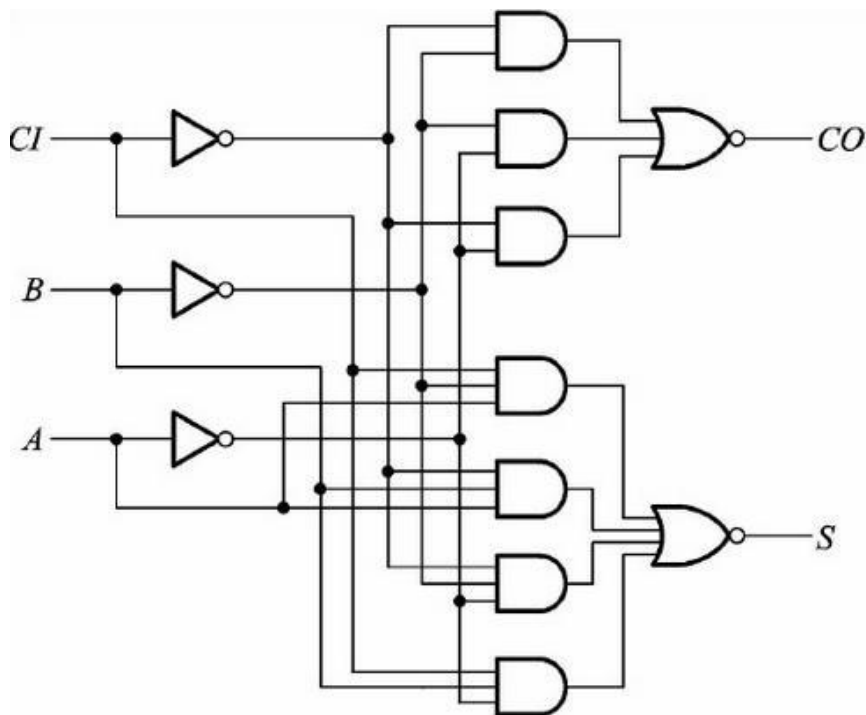
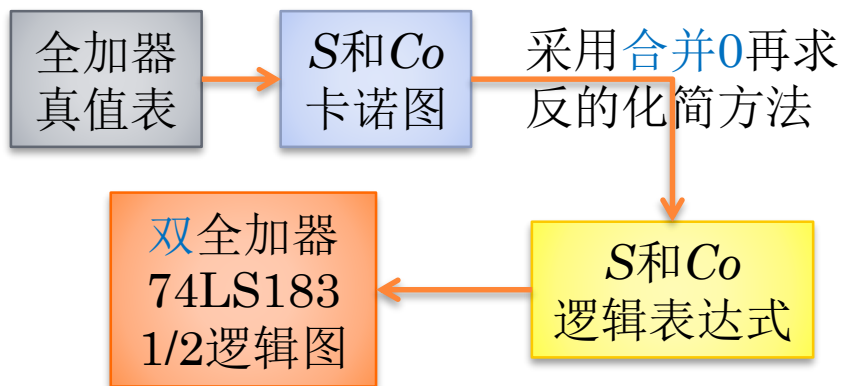
- 1位二进制**全加器 (Full Adder)** 可完成两个1位二进制数和低位进位来的进位信号相加。
 - 除了**最低位**，其它任意位相加都需要考虑有**三个1位二进制数相加**：
加数、**被加数**、低位向本位的**进位**
 - 产生两个**输出**：
 - 相加的和
 - 向更高位的进位

定义	代表的意义
A	被加数
B	加数
C_{-1}	低位向本位的进位
C_O	向更高位的进位
S	相加得到的和



$$C_O = (A + B)C_{-1} + AB \quad \text{同样正确}$$

全加器真值表



		Inputs			Outputs	
序号		A	B	C _I	S	C _O
0		0	0	0	0	0
1		0	0	1	1	0
2		0	1	0	1	0
3		0	1	1	0	1
4		1	0	0	1	0
5		1	0	1	0	1
6		1	1	0	0	1
7		1	1	1	1	1

BC _I A	00	01	11	10
0	0 ₀	0 ₁	1 ₃	0 ₂
1	0 ₄	1 ₅	1 ₇	1 ₆

C_O

$$C_O = \sum m(3,5,6,7) = \overline{A}\overline{B} + \overline{B}\overline{C}_I + \overline{A}\overline{C}_I$$

BC _I A	00	01	11	10
0	0 ₀	1 ₁	0 ₃	1 ₂
1	1 ₄	0 ₅	1 ₇	0 ₆

S

$$S = \sum m(1,2,4,7) = \overline{A}\overline{B}\overline{C}_I + A\overline{B}\overline{C}_I + \overline{A}B\overline{C}_I + AB\overline{C}_I$$

用的组合逻辑功能器件 周玲

1-BIT FULL ADDER BOOLEAN EQUATIONS

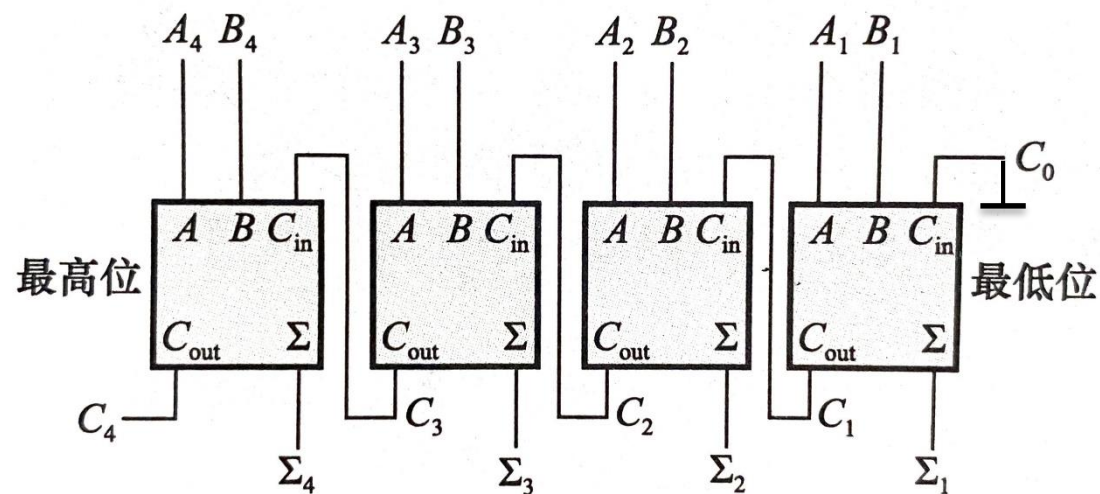
$$\begin{aligned}
 S &= \overline{\bar{A}\bar{B}\bar{C}_I + A\bar{B}C_I + \bar{A}BC_I + ABC_I} \\
 &= \overline{(\bar{A}\bar{B} + AB)\bar{C}_I + (A\bar{B} + \bar{A}B)C_I} \\
 &= \overline{(A \oplus B)\bar{C}_I + (A \oplus B)C_I} \\
 &= \overline{(A \oplus B) \oplus C_I} \\
 &= \underline{A \oplus B \oplus C_I}
 \end{aligned}$$

$$\begin{aligned}
 C_O &= \overline{\bar{A}\bar{B} + \bar{B}\bar{C}_I + \bar{A}\bar{C}_I} \\
 &= \overline{\bar{A}\bar{B} + (\bar{A} + \bar{B})\bar{C}_I} \\
 &= \overline{\bar{A}\bar{B} \cdot (\bar{A} + \bar{B})\bar{C}_I} \\
 &= (A + B)((\bar{A} + \bar{B}) + \bar{C}_I) \\
 &= (A + B)(AB + C_I) \\
 &= AB(A + B) + (A + B)C_I \\
 &= \underline{AB + (A + B)C_I}
 \end{aligned}$$

A	B	C _I	A + B	A ⊕ B	AB	C _O = AB + (A + B)C _I	C _O = AB + (A ⊕ B)C _I
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	1	1	0	0	0
0	1	1	1	1	0	1	1
1	0	0	1	1	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	0	1	1	1
1	1	1	1	0	1	1	1

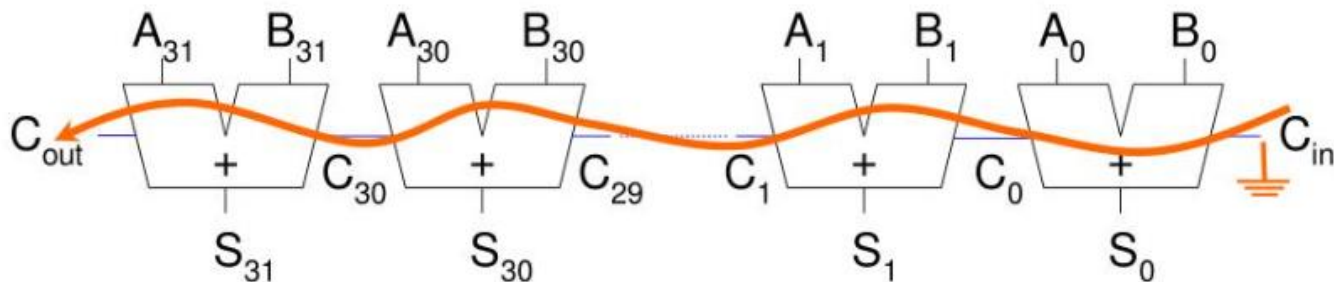
逐位（串行、行波）进位的全加器 4-BIT RIPPLE CARRY ADDER

- 依次将低位全加器的进位输出端 CO 接到高位全加器的进位输入端 CI ，就可以构成多位加法器。
 - 特点：充分体现了电路设计的重用。
 - 缺点：运算速度慢。
 - 在最不利的情况下，做一次加法运算需要经过4个全加器的传输延迟时间，才能得到稳定可靠的运算结果。
 - 应用：由于电路结构比较简单，可用于对运算速度要求不高的设备中。

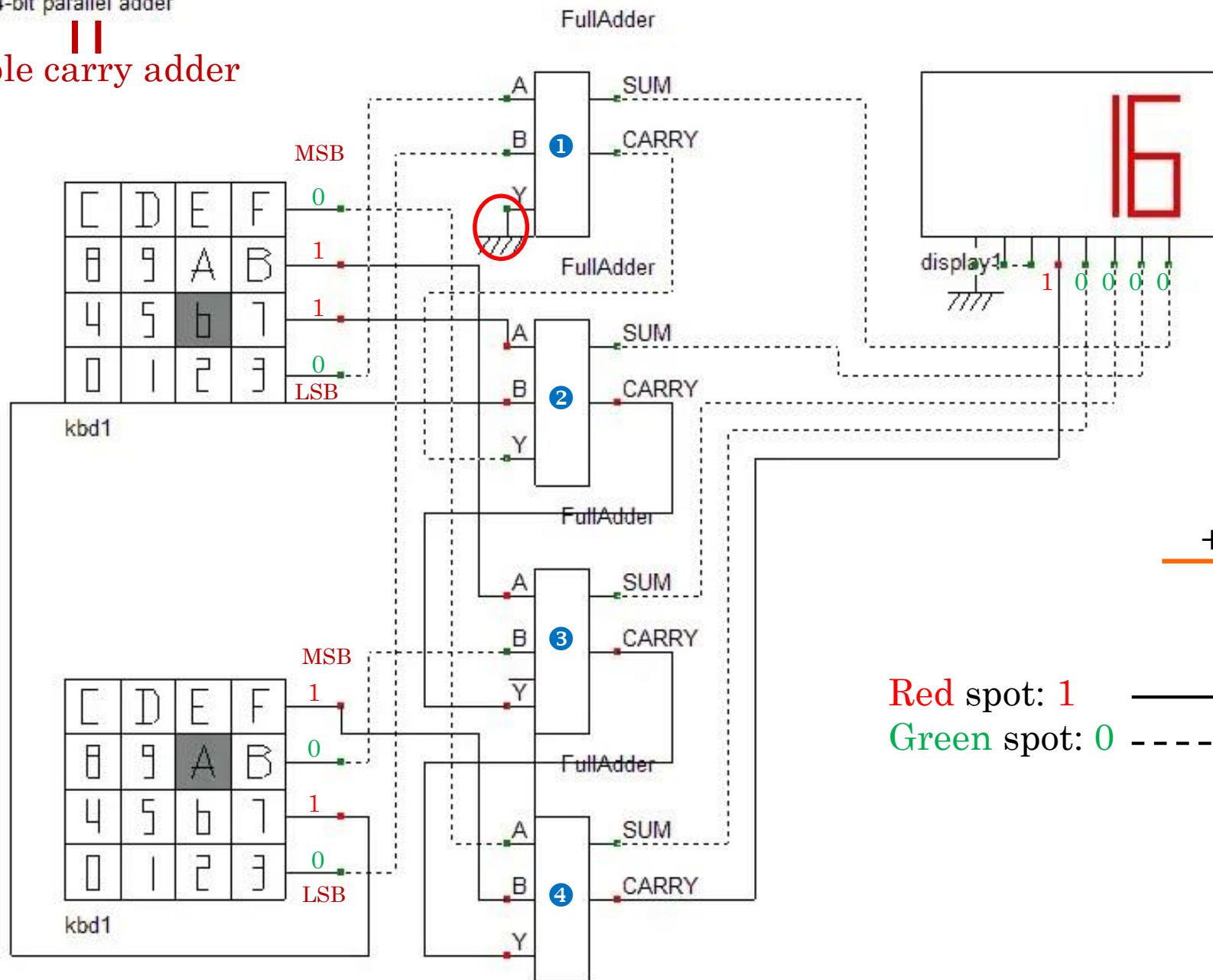


四位串行进位加法器

Example: 32-bit Ripple Carry Adder



4-bit parallel adder
4-bit ripple carry adder

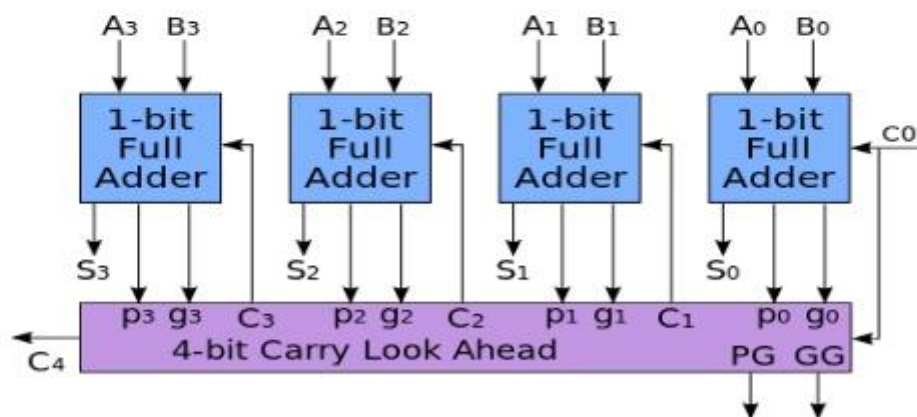


超前进位加法器

CARRY LOOK-AHEAD / FAST CARRY ADDER

Carry Look ahead Adder

A carry-look ahead adder improves speed by reducing the amount of time required to determine carry bits.



- 运算时间得以缩短是用增加电路复杂程度的代价换取的。
- 当加法器的位数增加时，电路的复杂程度也随之急剧上升。



超前进位加法器(CLA ADDER) (2)

Inputs			Outputs	
A	B	C_I	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- 分析全加器真值表，在两种情况下会有进位输出信号产生。

- $AB = 1 \quad \rightarrow C_o = 1$
- $A + B = 1 \ \&\& \ C_I = 1 \quad \rightarrow C_o = 1$
- 两个多位数中第 i 位相加产生的进位输出 $(C_o)_i$ 可表示为:

$$(C_o)_i = A_i B_i + (A_i + B_i) (C_I)_i$$

- 若定义：进位生成函数
进位传送函数

$$\begin{aligned} G_i &= A_i B_i \\ P_i &= A_i + B_i \end{aligned}$$

Carry Generate (G_i)
Carry Propagate (P_i).

- 则

$$(C_o)_{i+1} = G_i + P_i (C_I)_i$$

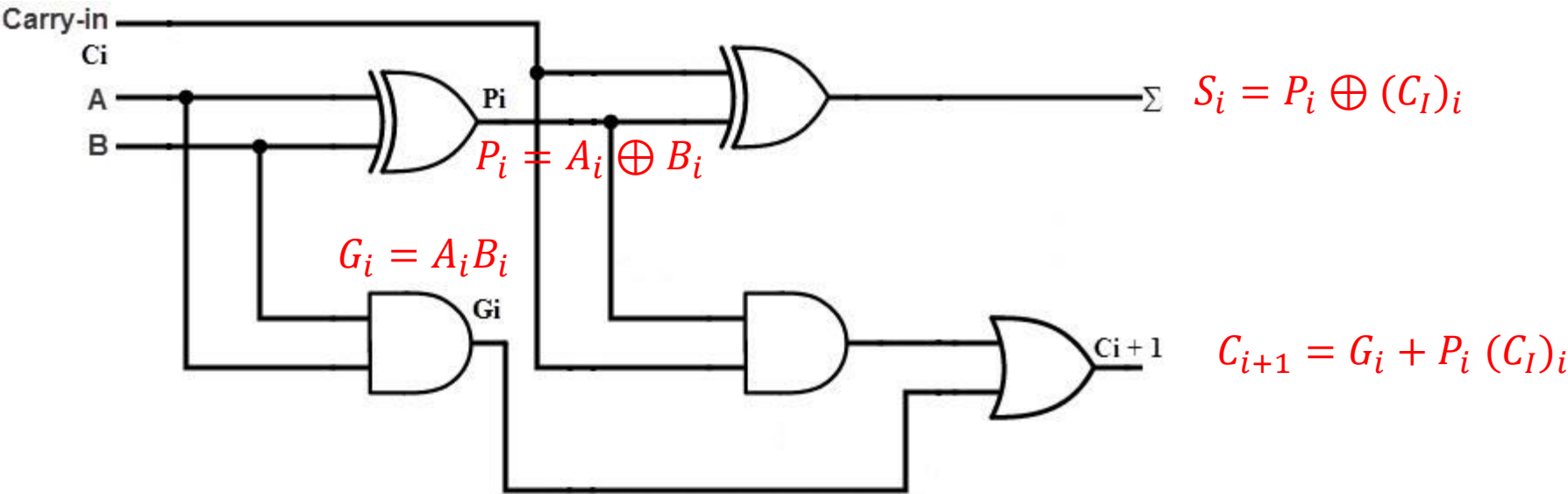
展开可得

$$\begin{aligned} (C_o)_{i+1} &= G_i + P_i (C_I)_i \\ &= G_i + P_i [G_{i-1} + P_{i-1} (C_I)_{i-1}] \\ &= G_i + P_i G_{i-1} + P_i P_{i-1} [G_{i-2} + P_{i-2} (C_I)_{i-2}] \\ &\quad \vdots \\ &= G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + \cdots + P_i P_{i-1} \cdots P_1 G_0 + P_i P_{i-1} \cdots P_0 (C_I)_0 \end{aligned}$$

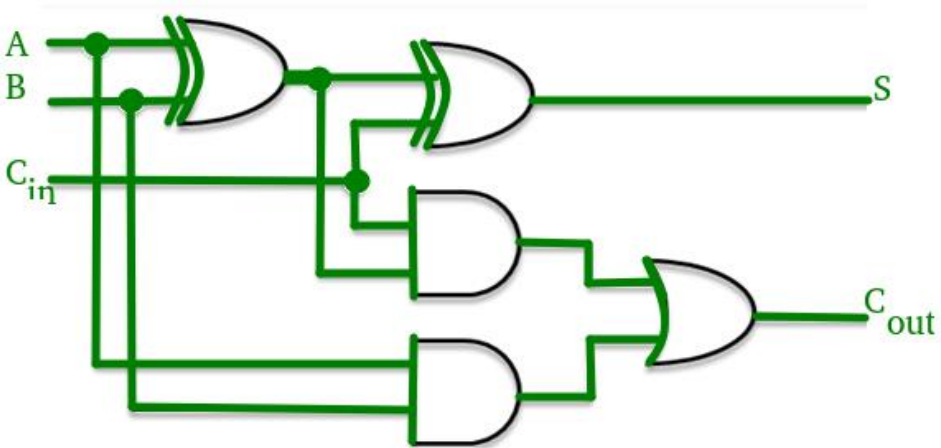
- 第 i 位

$$S_i = A_i \oplus B_i \oplus (C_I)_i$$

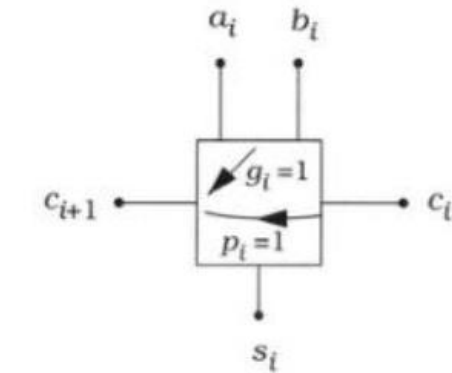
超前进位加法器 (CLA ADDER) (3)



Inputs			Outputs		Condition
A	B	C_i	S	C_o	
0	0	0	0	0	No Carry Generated
0	0	1	1	0	
0	1	0	1	0	
0	1	1	0	1	Carry Propagated
1	0	0	1	0	No Carry
1	0	1	0	1	Carry Propagated
1	1	0	0	1	Carry Generated
1	1	1	1	1	

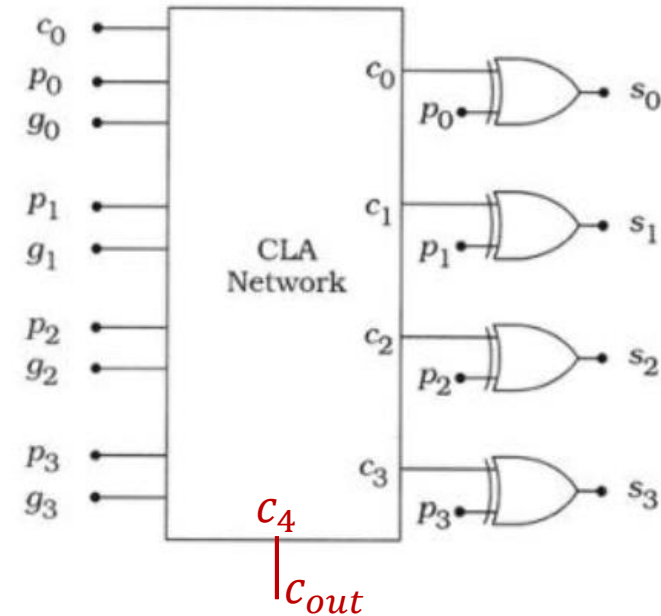
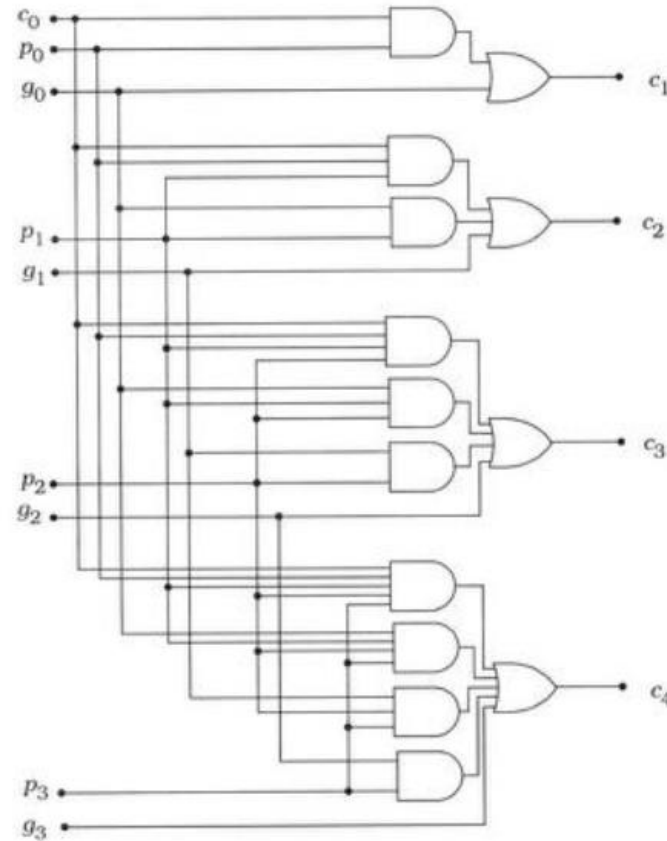


4 STAGE CLA (CARRY-LOOKAHEAD) ADDER



	g_i	p_i
	$a_i \cdot b_i$	$a_i \oplus b_i$
$a_i = b_i = 0$	0	0
$a_i = b_i = 1$	1	0
$a_i \neq b_i$	0	1

$$c_{i+1} = a_i \cdot b_i + c_i \cdot (a_i \oplus b_i)$$



$$C_1 = G_0 + P_0 \cdot C_0$$

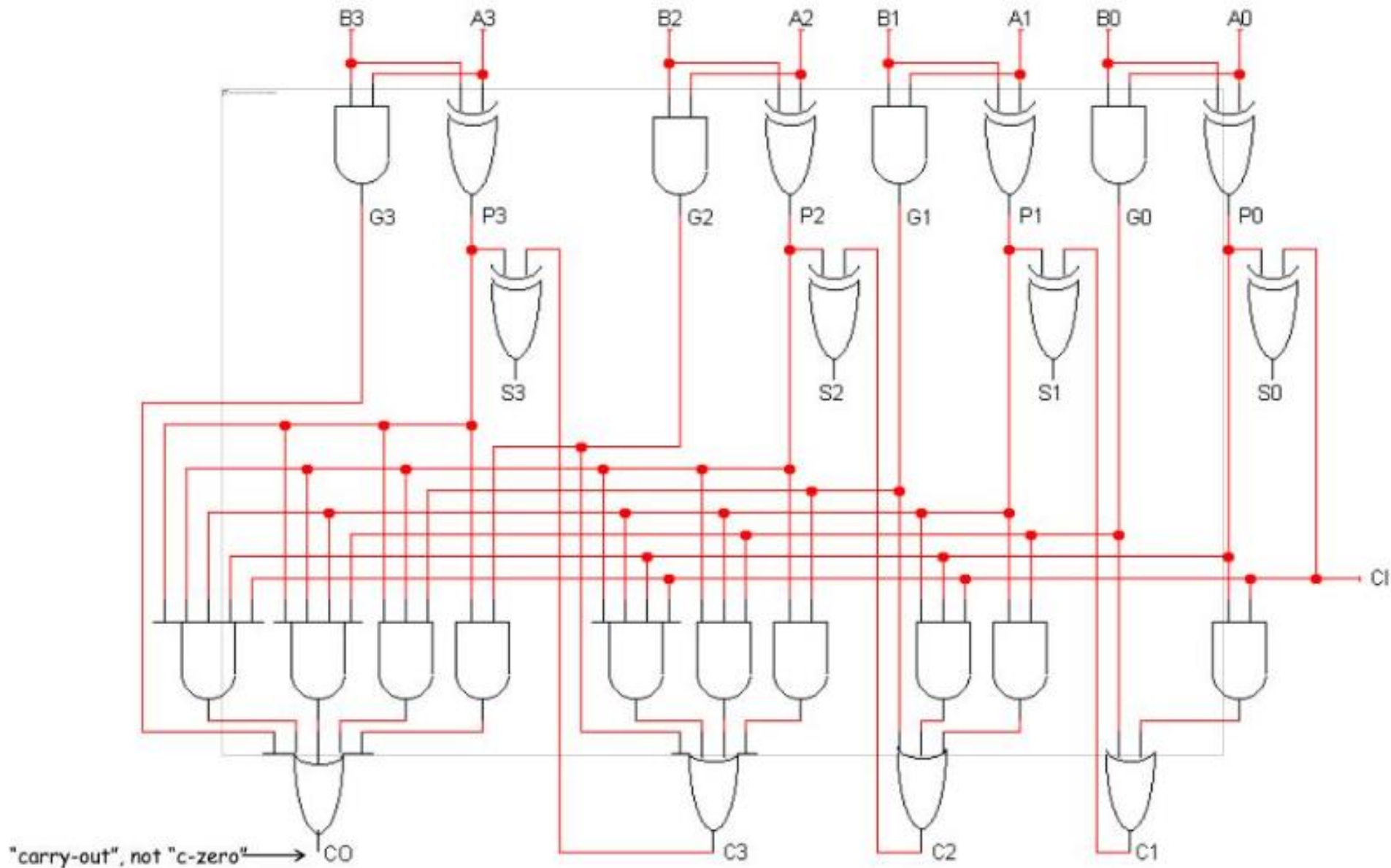
$$C_2 = G_1 + P_1 \cdot C_1 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

$$C_3 = G_2 + P_2 \cdot C_2 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

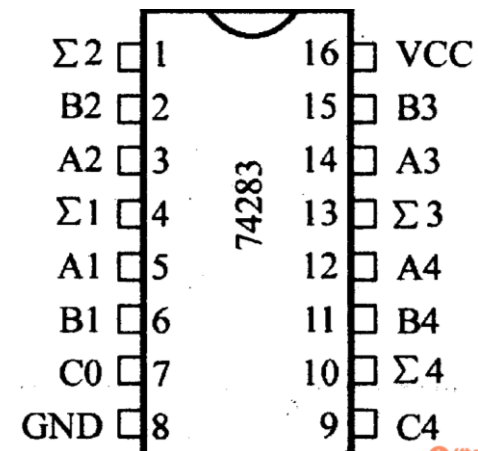
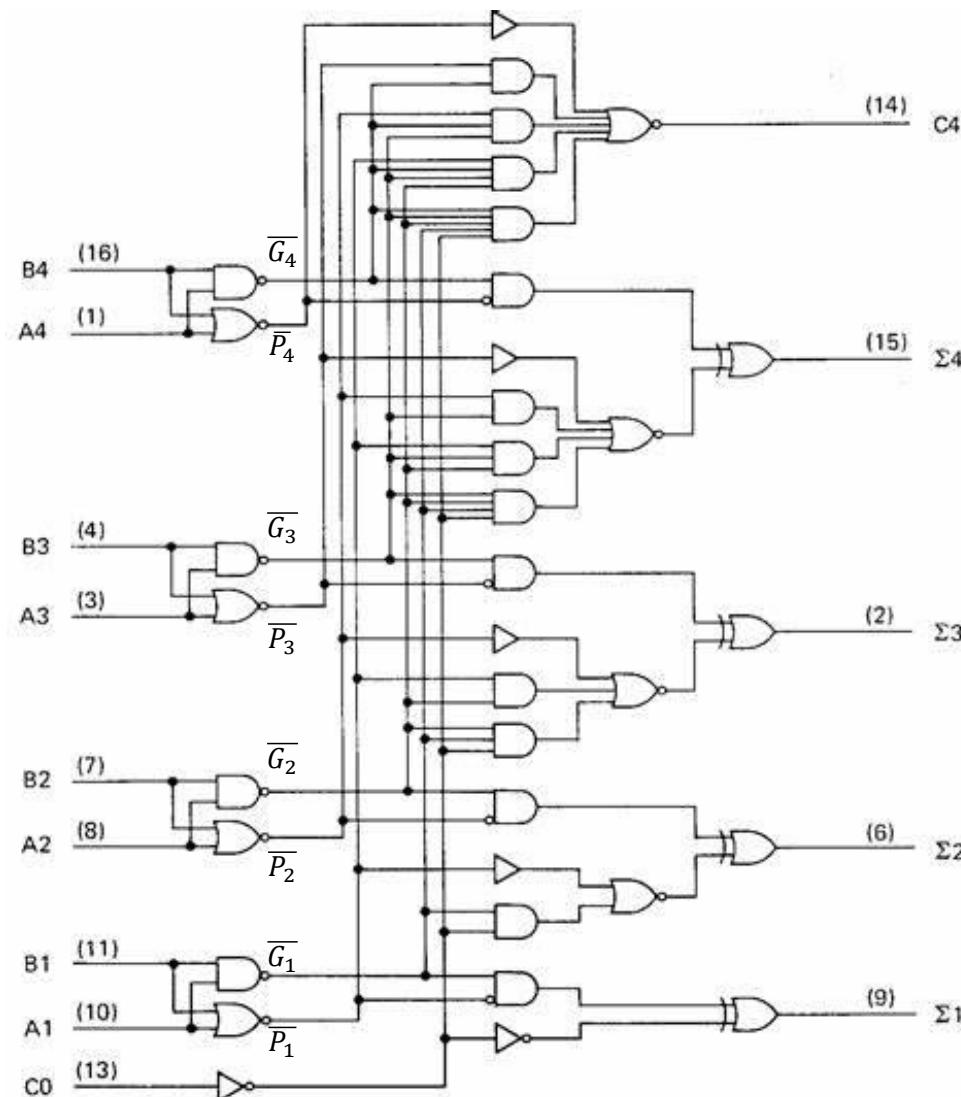
$$CO = C_4 = G_3 + P_3 \cdot C_3$$

$$= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

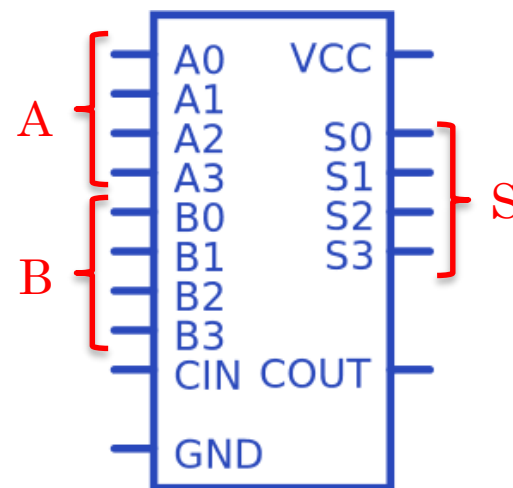
A 4-bit carry lookahead adder circuit



超前进位的4位二进制全加器74283 CARRY LOOK-AHEAD 4 BIT ADDER



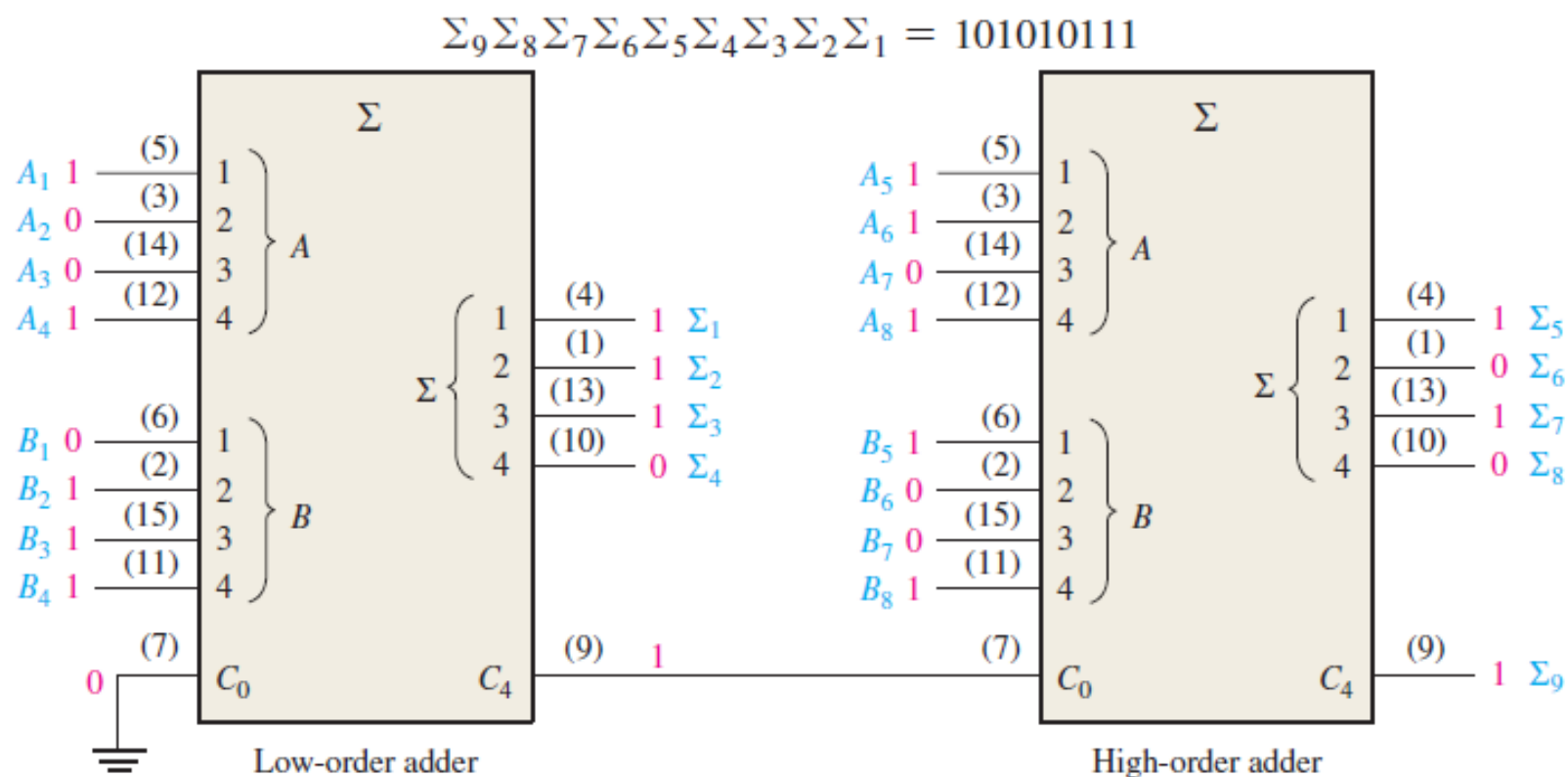
- 74283完成的是两个4位二进制数A, B相加, C_{in} 是低位向本位的进位信号。



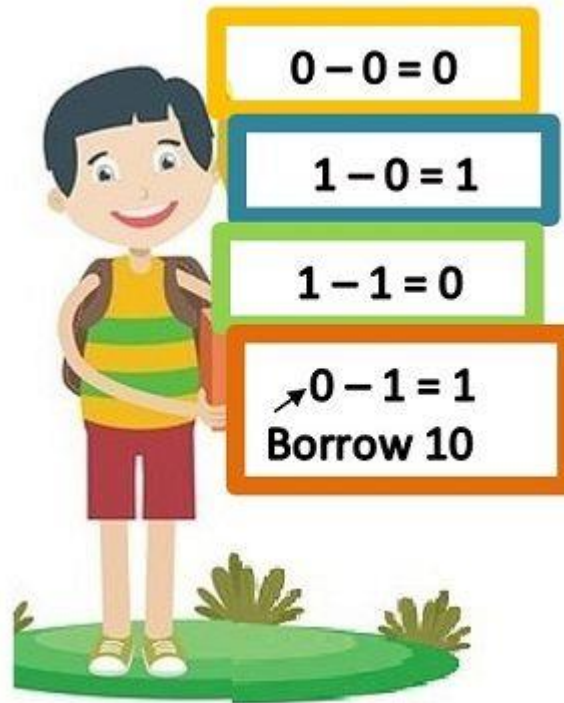
多位二进制全加器

- 可以采用多片74283相连来构成。

$$\begin{array}{r}
 B_8 B_7 B_6 B_5 B_4 B_3 B_2 B_1 \\
 1\ 0\ 0\ 1\ 1\ 1\ 1\ 0\quad B_i \\
 +\quad 1\ 0\ 1\ 1\ 1\ 0\ 0\ 1\quad A_i \\
 \hline
 1\ 0\ 1\ 0\ 1\ 0\ 1\ 1\ 1\quad \text{sum}
 \end{array}$$



减法运算



Borrow is required as 1 cannot be subtracted from 0, $10-1=1$

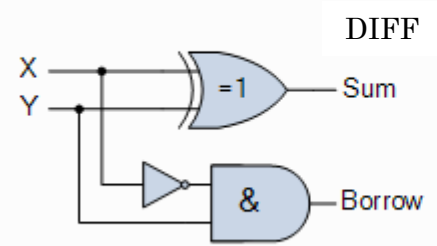
$$\begin{array}{r} 1010 \\ - 1001 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1010 \\ - 1001 \\ \hline 01 \end{array}$$

$$\begin{array}{r} 1010 \\ - 1001 \\ \hline 001 \end{array}$$

$$\begin{array}{r} 1010 \\ - 1001 \\ \hline 0001 \end{array}$$

与全加器HA相比，半减器HS中的被减数加了一个反相器

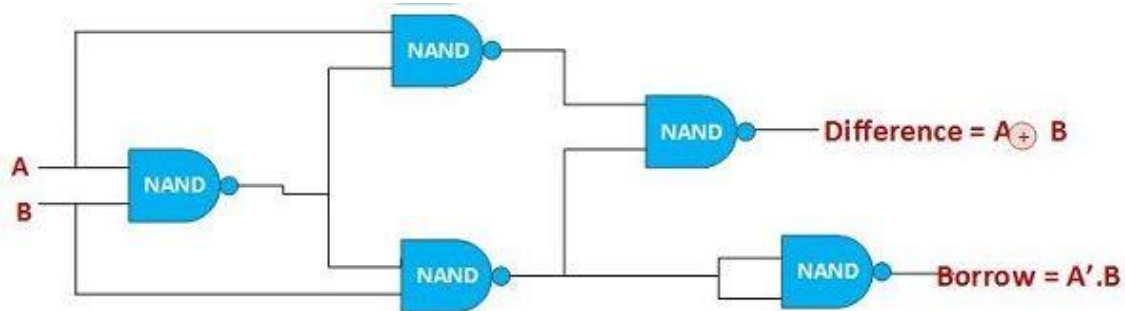
半减器 Symbol		减数 被减数 Truth Table			
	DIFF	Y	X	DIFFERENCE	BORROW
	Sum	0	0	0	0
		0	1	1	0
	Borrow	1	0	1	1
		1	1	0	0

1-BIT 半减器 (HALF SUBTRACTOR)

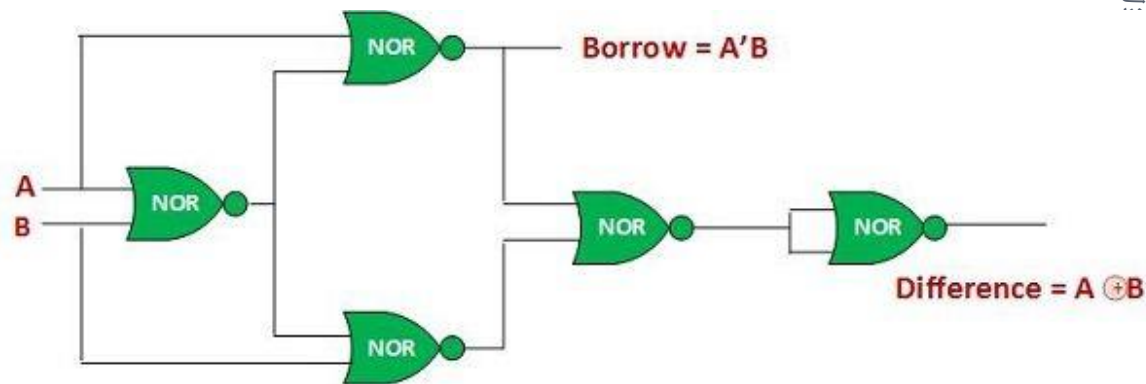
不考虑低位借位

$$DIFF = X \oplus Y$$

$$B_{out} = \bar{X}Y$$



Half Subtractor using NAND Gate



Half Subtractor using NOR Gate

两个二进制数的最低位相减，由于没有更低的位，因此不需要考虑更低位向本位的借位，这种减法器位全减器的特例，也称为半减器。

Full Subtractor(1)

- Subtracting two single-bit binary values, Y, B-in from a single-bit value X produces a difference bit D and a borrow out B-out bit. This is called full subtraction.

被减数 减数 低位向本位的借位 本位向更高位的借位
Full Subtractor Truth Table:

Inputs X Y B-in	Outputs D B-out
0 0 0	0 0
0 0 1	1 1
0 1 0	1 1
0 1 1	0 1
1 0 0	1 0
1 0 1	0 0
1 1 0	0 0
1 1 1	1 1

$$S(X, Y, B-in) = \Sigma (1, 2, 4, 7)$$

$$B-out(x, y, B-in) = \Sigma (1, 2, 3, 7)$$

对比前面的全加器真值表

Difference D

XY		X			
		00	01	11	10
B-in	0	0	2 1	6	4 1
	1	1 1	3	7 1	5

$$D = X'Y'(B-in) + X'Y(B-in)' + XY'(B-in)' + XY(B-in)$$

$$D = X \oplus Y \oplus (B-in)$$

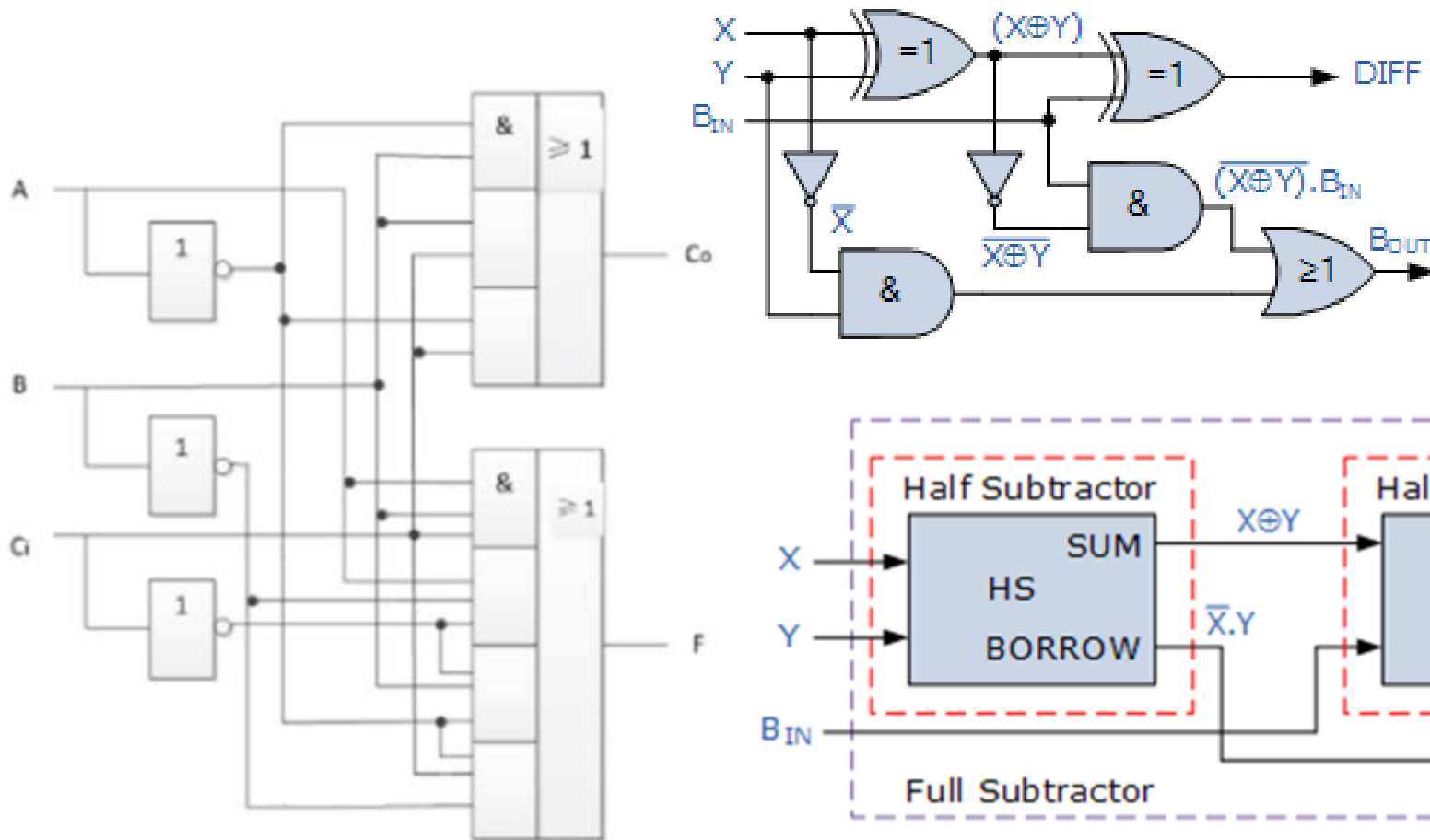
Borrow B-out

XY		X			
		00	01	11	10
B-in	0	0	1	6	4
	1	1	1	7 1	5

$$B-out = X'Y + X'(B-in) + Y(B-in)$$

$$B_{out} = \bar{X}Y + B_{in} \cdot (\bar{X} + Y)$$

1-BIT 全减器 (FULL SUBTRACTOR)

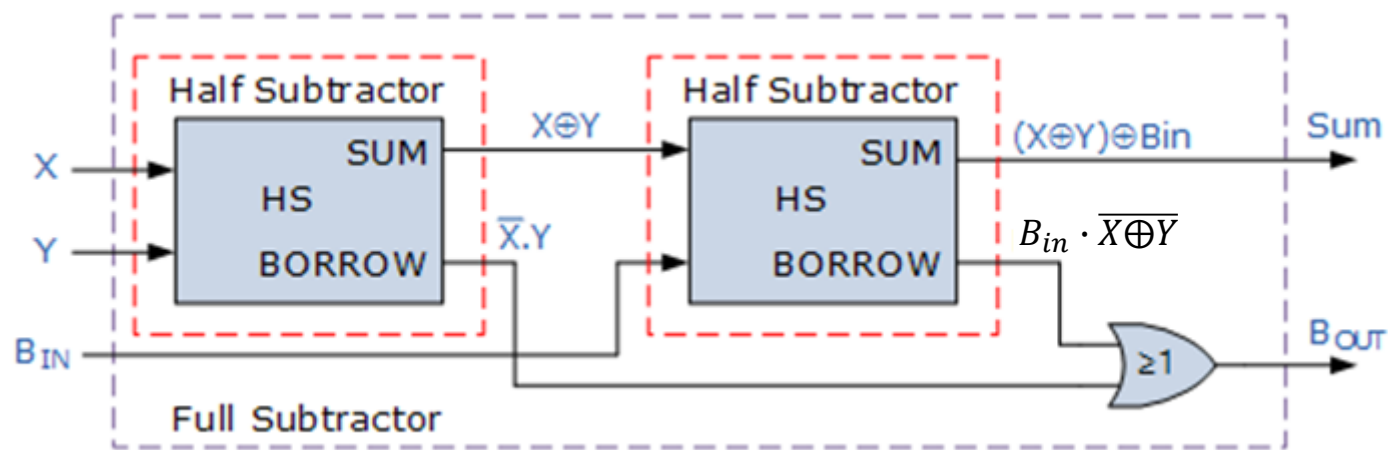


$$DIFF = X \oplus Y \oplus B_{in}$$

$$B_{out} = \bar{X}Y + B_{in} \cdot \overline{X \oplus Y}$$

该 B_{out} 真值表和前面一页

$B_{out} = \bar{X}Y + B_{in} \cdot (\bar{X} + Y)$
的真值表输出一样



补码的加减法共用电路

- 在计算机中通常不另外设计减法器

- 而是将减法运算转换为加法运算

- 一般采用补码的方式完成运算

- 仅讨论无符号数，即补数（补码=符号+补数）

- 补数的运算规则： $N_{\text{补}} = 2^n - N_{\text{原}} = N_{\text{反}} + 1$ → $-N_{\text{原}} = N_{\text{补}} - 2^n = N_{\text{反}} + 1 - 2^n$

- $A - B = A + B_{\text{补}} - 2^n = A + B_{\text{反}} + 1 - 2^n$

- 采用全加器实现 $A + B_{\text{反}} + 1$

- 再考虑借位信号

- 当加法器有进位信号时，即 2^n ，与上式中的 2^n 相抵消，不需要借位 → 借位信号为0
 - 当加法器没有进位信号时，最高位的进位信号为0，它与 2^n 相减差为1，同时还要给一个借位信号
 - 方法：只要将加法器最高位的进位信号取反，就实现了减 2^n 的运算，反相器输出为1表示有借位。

	正数	负数
补码变换操作	原码、反码和补码是一样的	原码符号位不变，将其余各位取反 → 反码 反码末位加1 → 补码

$N_{\text{反}}$ 每一位都取反，
没有符号位

两个4位无符号二进制数减法逻辑图

在计算机中处理加、减运算，只需要**加法器**和**求补电路**即可。
两个补码相加结果仍然是补码。

右上图实现下式：

$$A - B = A + B_{\text{补}} - 2^n = A + B_{\text{反}} + 1 - 2^n$$

A（被减数），B（减数） 为**两无符号数**。

借位信号V

$V = 0, A \geq B, A - B \geq 0$

$V = 1, A < B, A - B < 0$

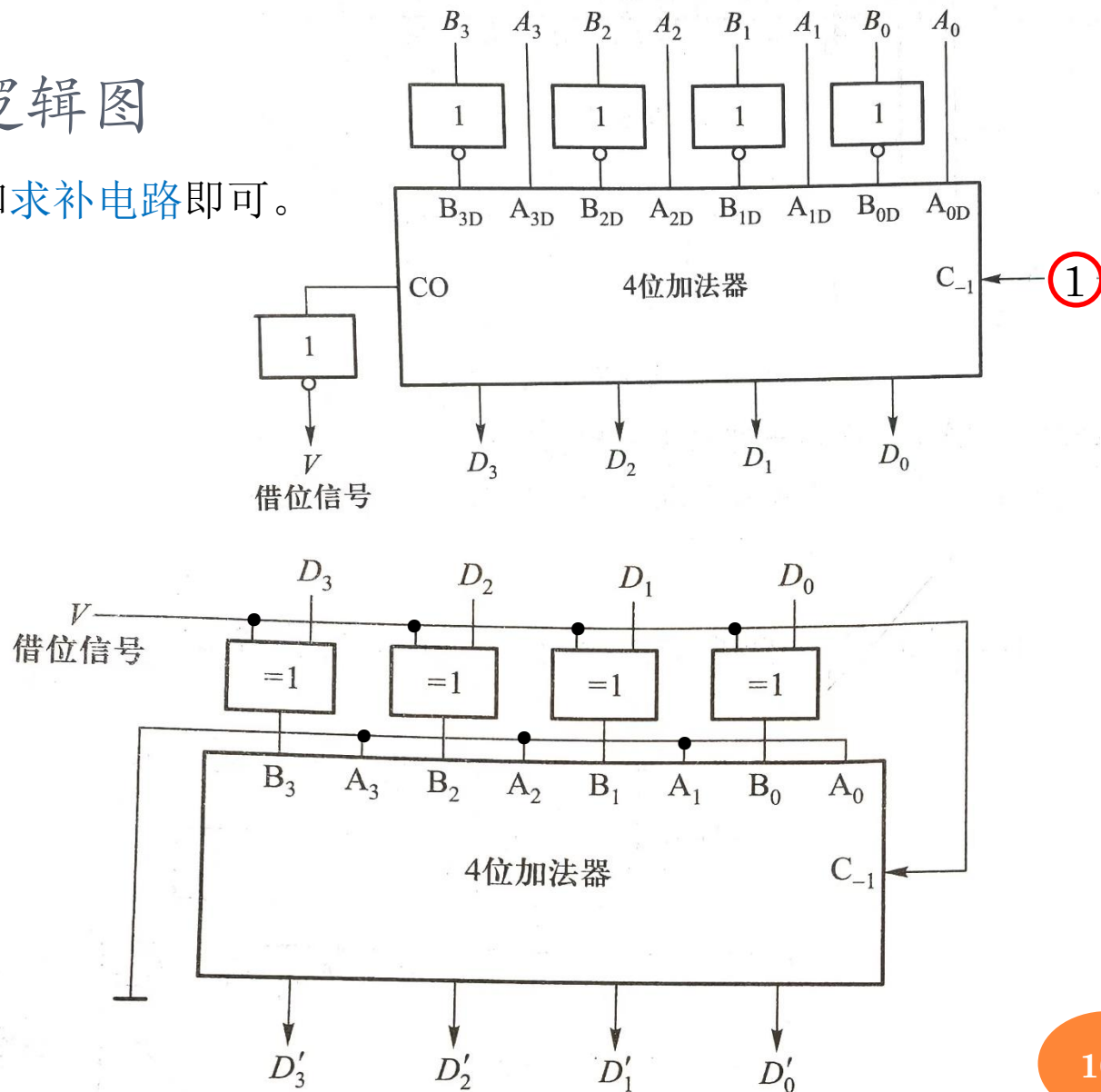
可以由**借位信号V**为0还是1（相减结果是正数还是负数），来决定是否对所得到的结果**求补**。

异或门的性质

$$D \oplus 0 = D$$

$$D \oplus 1 = \bar{D}$$

求补逻辑图设计，
如右下图所示：

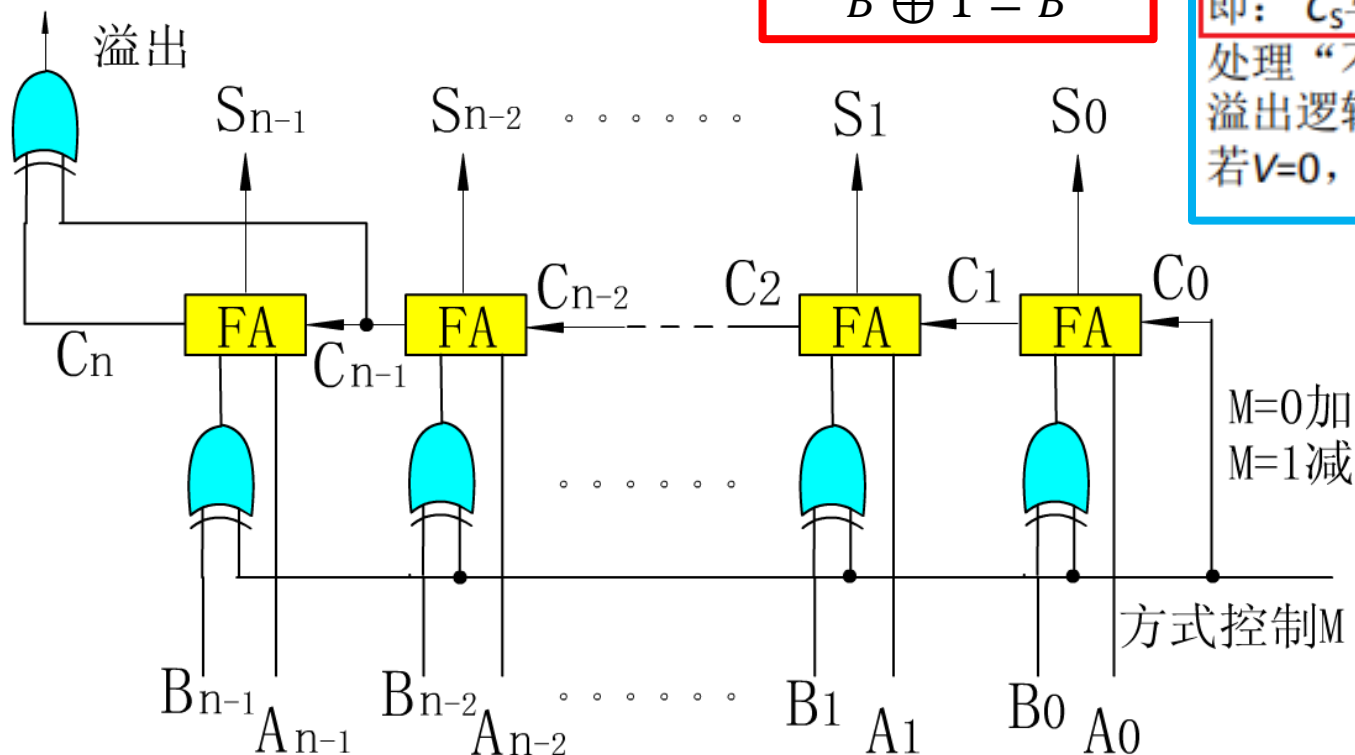


N位行波进位加减器

异或门的性质

$$B \oplus 0 = B$$

$$B \oplus 1 = \bar{B}$$



符号位

n个一位的全加器（FA）可级联成一个n位的行波进位加减器，M为方式控制输入线，当M=0时，做加法（A+B）运算；当M=1时，做减法运算；A—B运算转化成[A]补+[B]补运算，求补过程由B+1实现。

符号位的进位 C_s 最高数值位的进位 C_1

上溢	0	1
下溢	1	0

即： C_s 与 C_1 不同时溢出

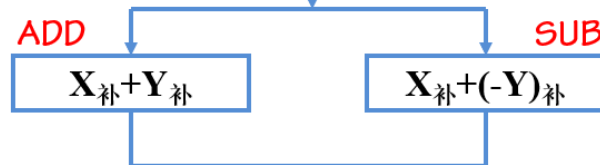
处理“不同”的逻辑符号：异或 $\oplus$

溢出逻辑判断表达式为 $V=C_s \oplus C_1$

若 $V=0$ ，表示无溢出； $V=1$ ，表示有溢出。

补码运算采用单符号位法来判断是否溢出

操作数是用补码表示的，符号位将会参与到运算中

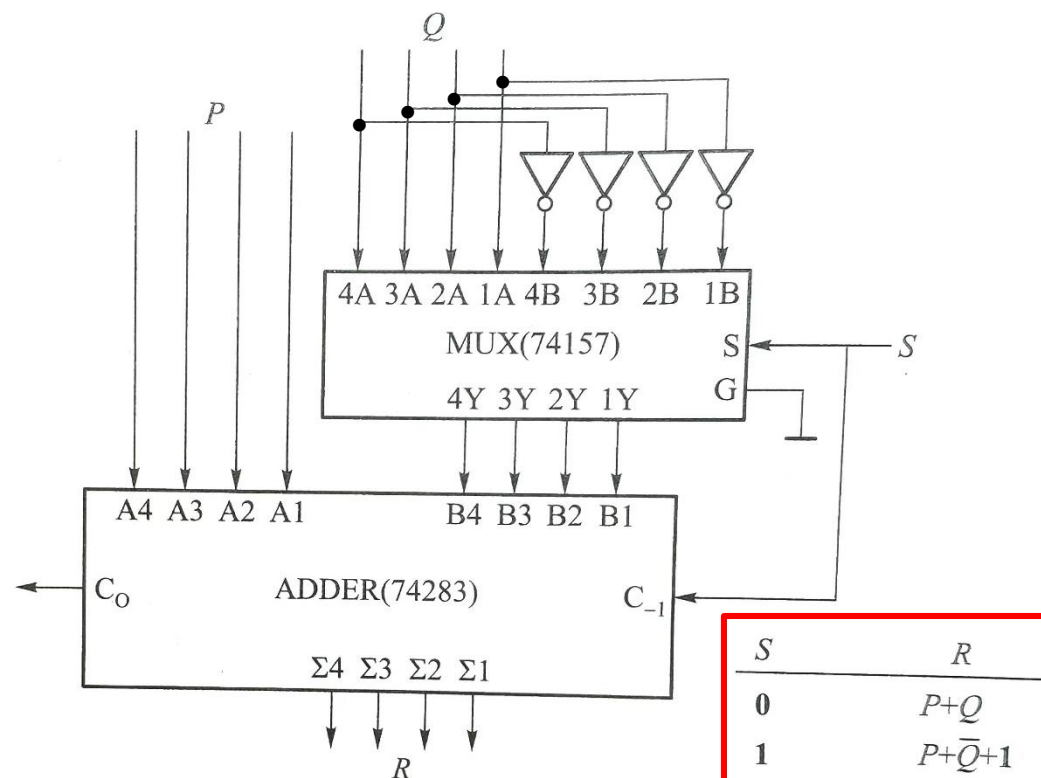
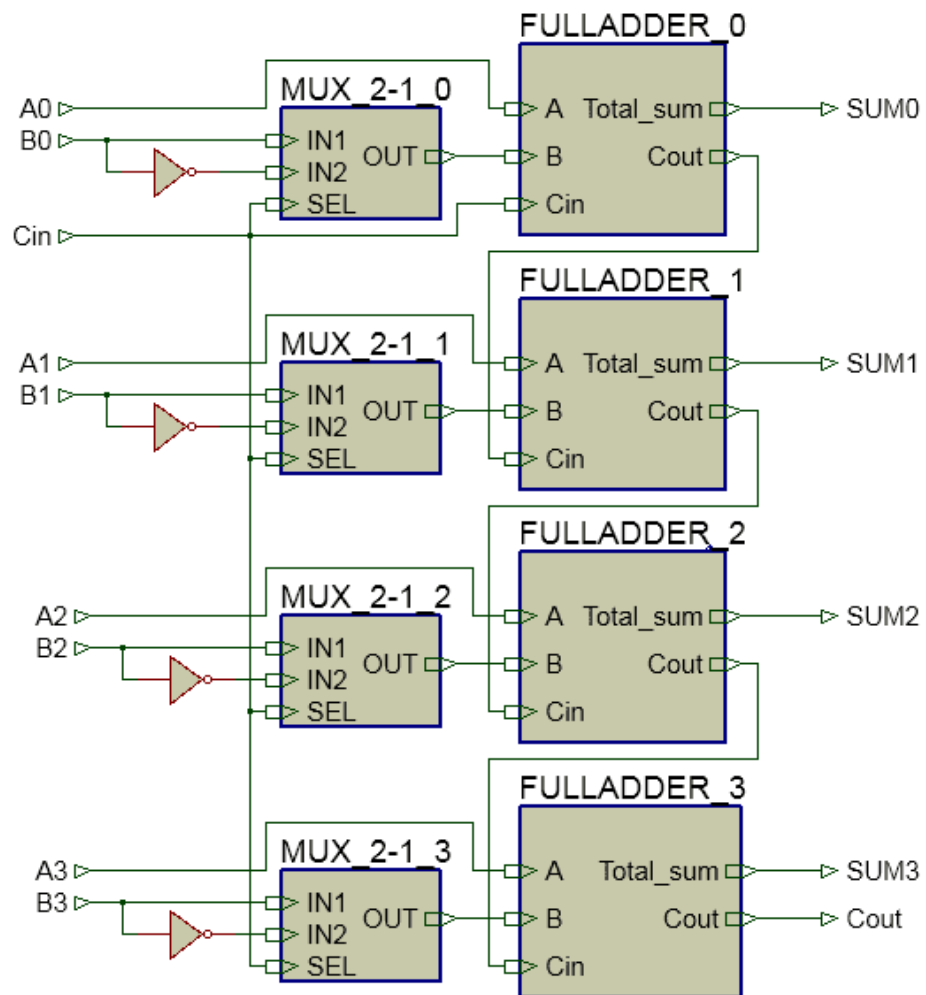
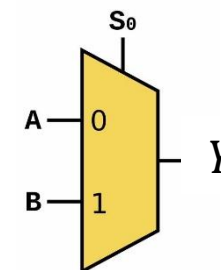


结果是用补码表示的，符号位能够指示结果的正负

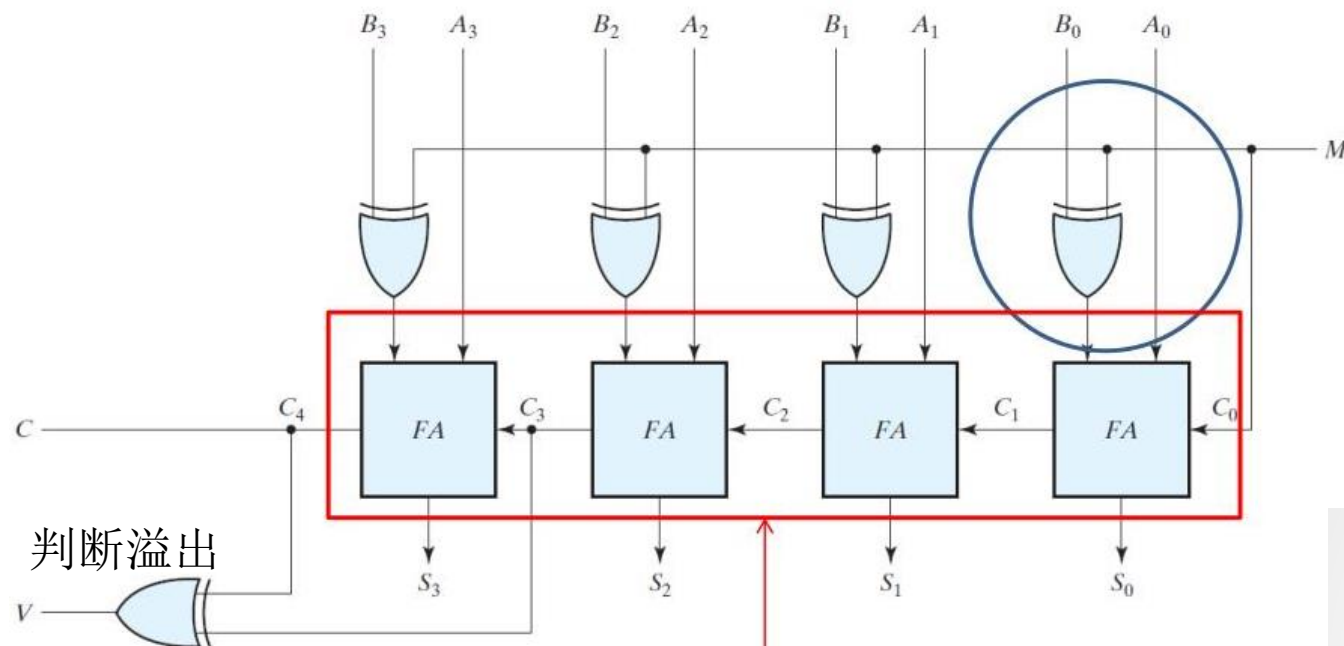
采用4位全加器电路74283和4个二选一数据选择器74157
(PPT第44页)实现这种补码加减法电路。

补码的加减法电路

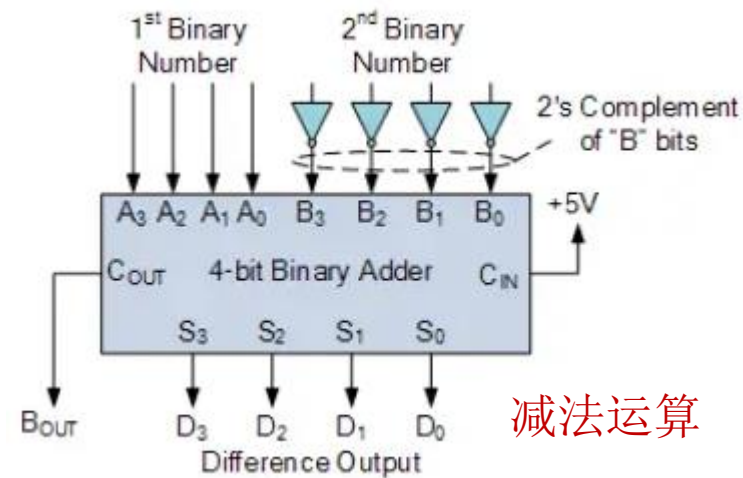
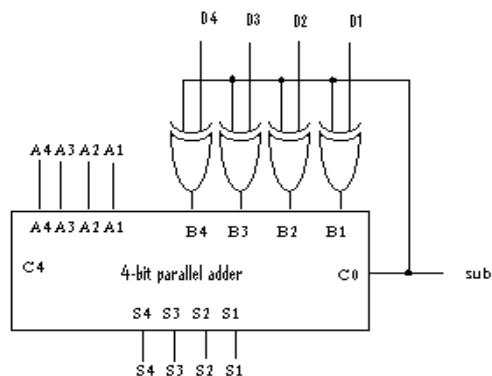
$$Y = A\bar{S} + BS$$



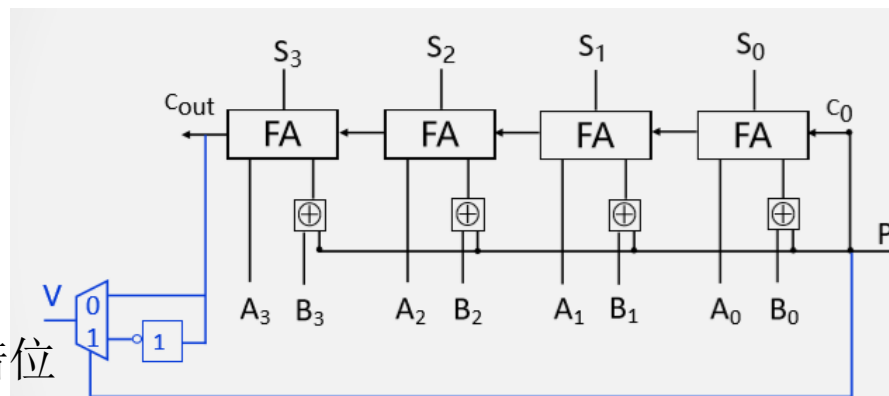
Four-Bit Adder-Subtractor



(4 bit ripple adder)



减法运算



进位/借位

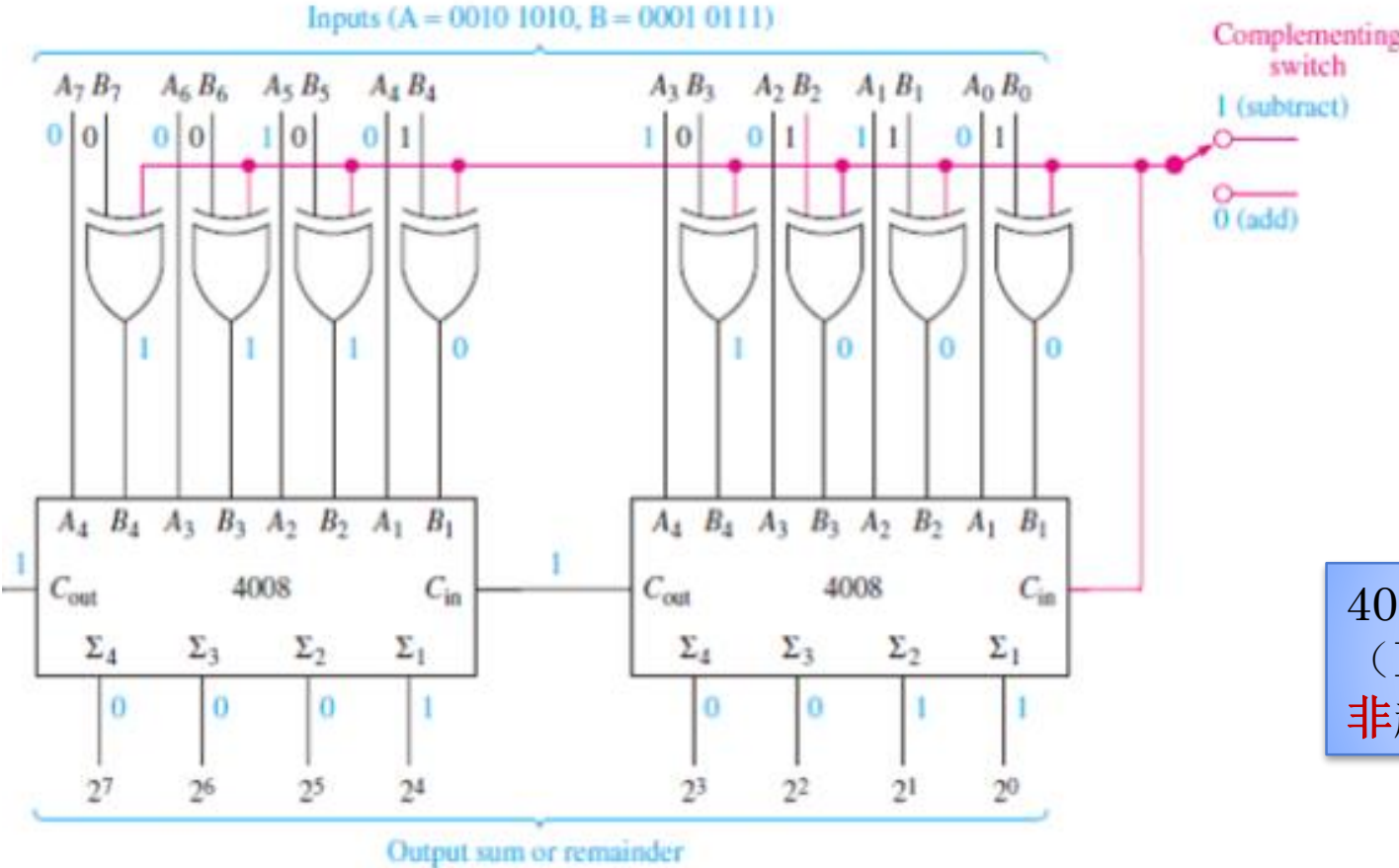
P=1 选择无符号数减法溢出 (借位)

P=0时, 选择无符号加法溢出 (进位)

8-BIT ADDER/SUBTRACTOR BUILT WITH ADDER ICS AND XOR GATES

$[B]_{\text{原}} = 00010111 \Rightarrow [-B]_{\text{补}} = 11101001$

	C_s	C_1									
			0	0	1	0	1	0	1	0	$[A]_{\text{补}}$ 42
+			1	1	1	0	1	0	0	1	$[-B]_{\text{补}}$ - 23
舍弃 $\rightarrow$	1		0	0	0	1	0	0	1	1	19



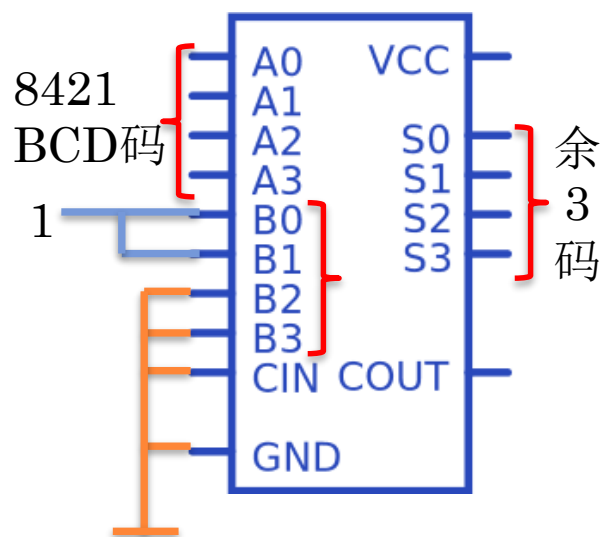
$V = C_s \oplus C_1 = 1 \oplus 1 = 0$
计算结果无溢出

4008是4-bit全加器
(FA, Full Adder)
非超前进位全加器

用加法器设计组合逻辑电路

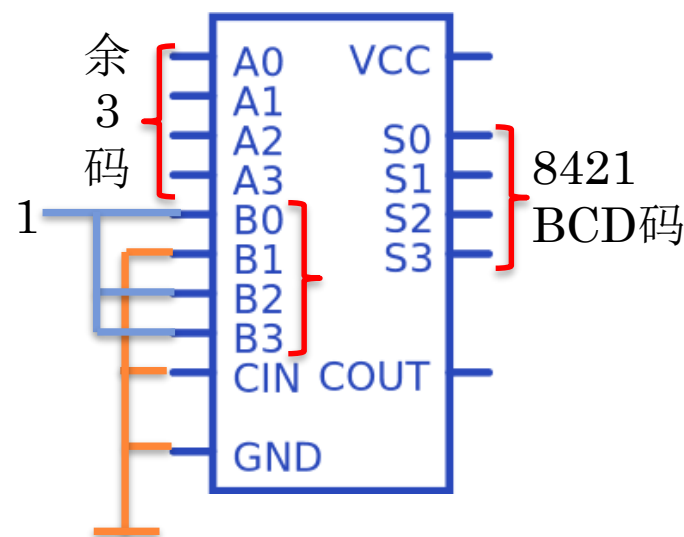
- 用4位二进制加法器74283实现8421BCD码到余3码的转换
- 编码规律:

$$\text{余3码} = 8421\text{BCD} + 0011$$



- 用4位二进制加法器74283实现余3码到8421BCD码的转换
- 编码规律:

$$8421\text{BCD} = \text{余3码} + 1101$$



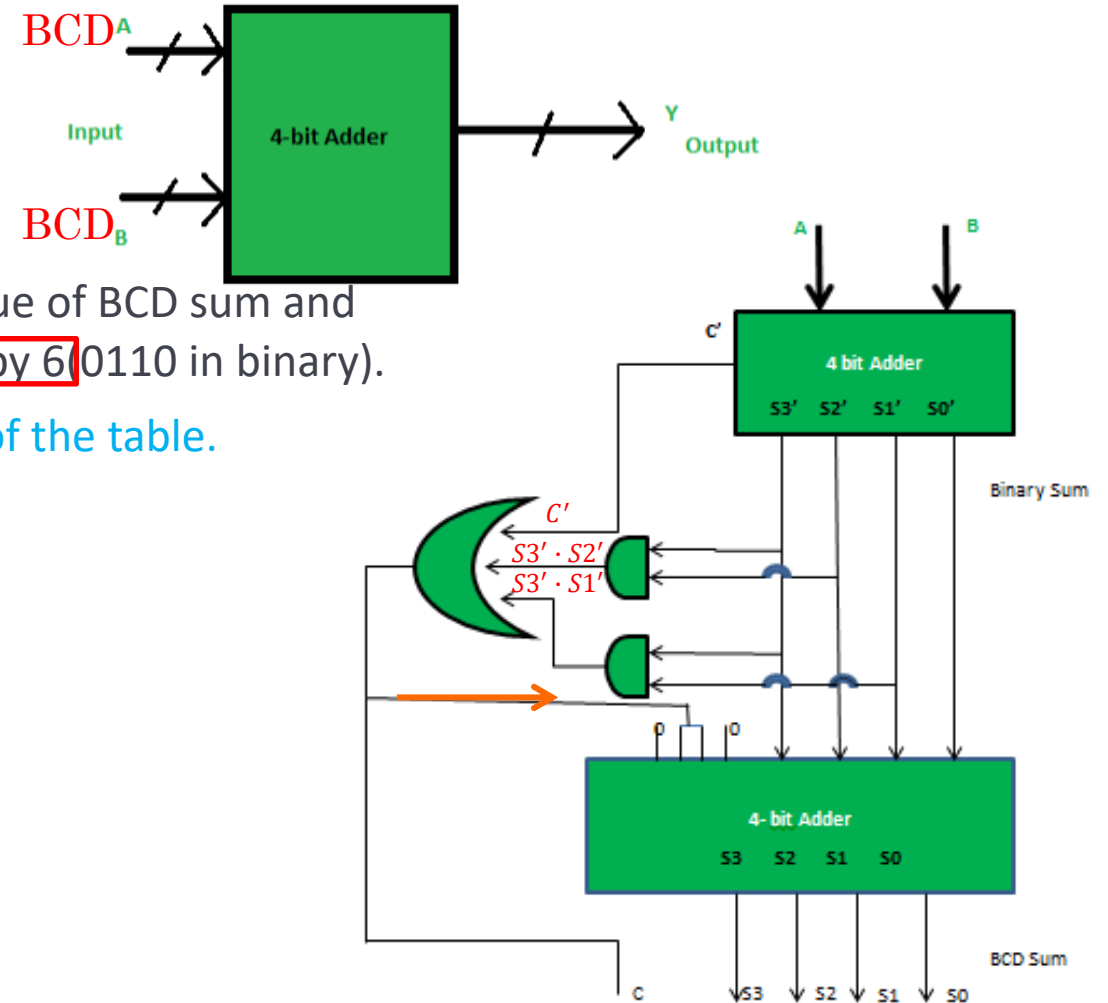
1-DIGIT BCD ADDER

BCD – Binary Coded Decimal

Note – If the **sum** of two number ≤ 9 , then the value of BCD sum and binary sum will be same, otherwise they will **differ by 6** (0110 in binary).

We are adding “0110” (=6) only to the second half of the table.

Decimal	Binary Sum					BCD Sum				
	C'	S3'	S2'	S1'	S0'	C	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

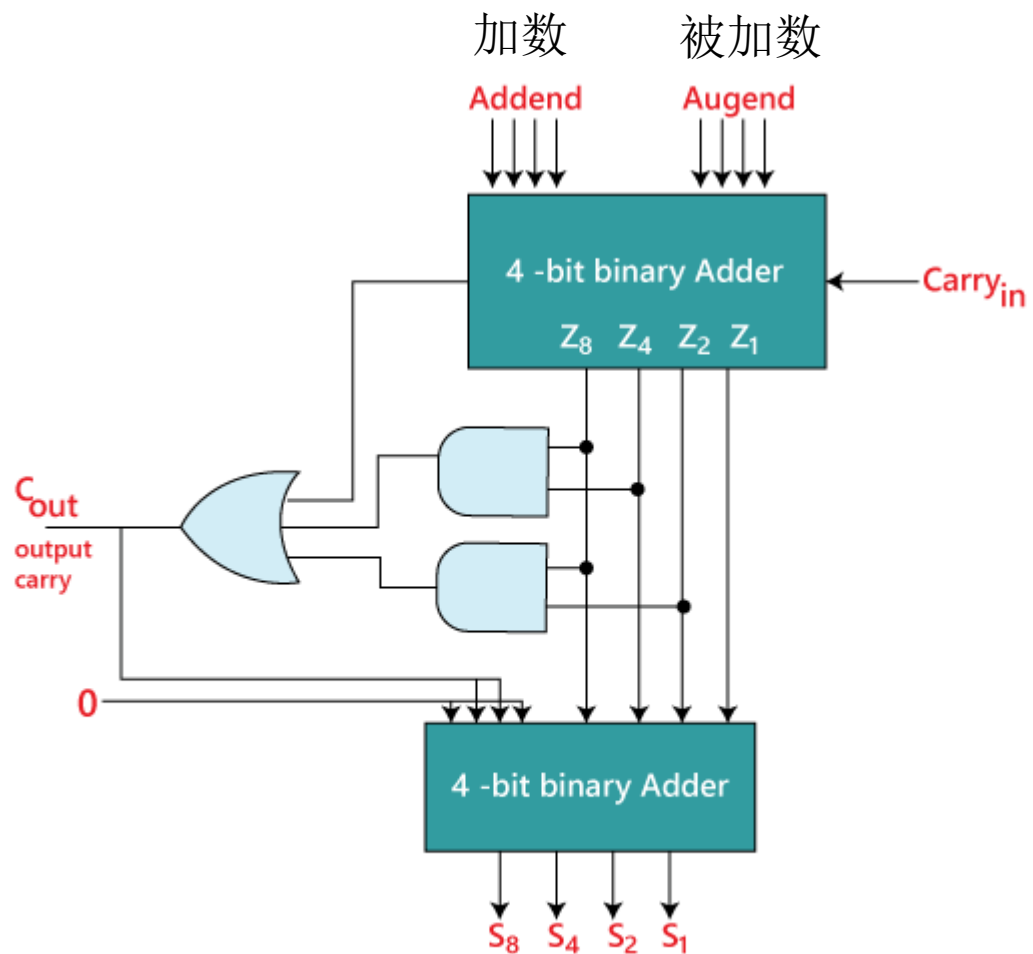


The conditions are:

- 1.If $C' = 1$ (Satisfies 16-19)
 - 2.If $S3' \cdot S2' = 1$ (Satisfies 12-15)
 - 3.If $S3' \cdot S1' = 1$ (Satisfies 10 and 11)
- So, our logic is

$$C' + S3' \cdot S2' + S3' \cdot S1' = 1$$

$$C = S'_3S'_1 + S'_3S'_2 + C'$$

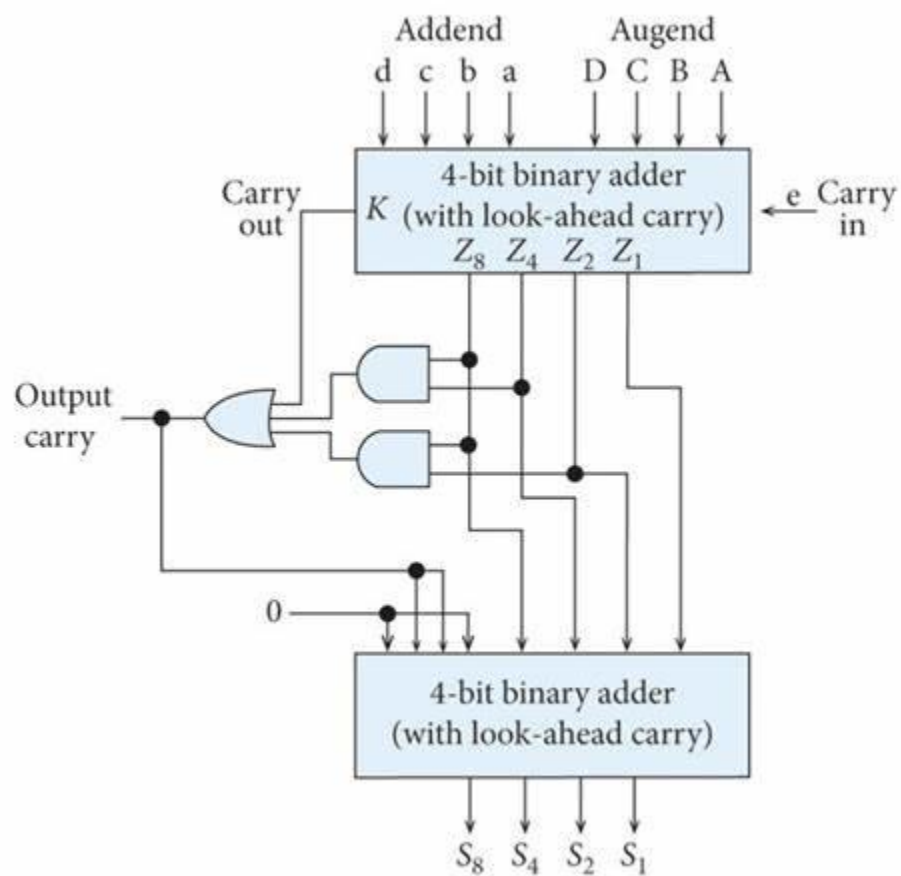


Block diagram of a BCD adder

$S'_2S'_1S'_0$		$S'_3S'_1$							
$C'S'_3$		000	001	011	010	110	111	101	100
00		0 ₀	0 ₁	0 ₃	0 ₂	0 ₆	0 ₇	0 ₅	0 ₄
01		0 ₈	0 ₉	1 ₁₁	1 ₁₀	1 ₁₄	1 ₁₅	1 ₁₃	1 ₁₂
11		x ₂₄	x ₂₅	x ₂₇	x ₂₆	x ₃₀	x ₃₁	x ₂₉	x ₂₈
10		1 ₁₆	1 ₁₇	1 ₁₉	1 ₁₈	x ₂₂	x ₂₃	x ₂₁	x ₂₀

C'

BCD ADDER

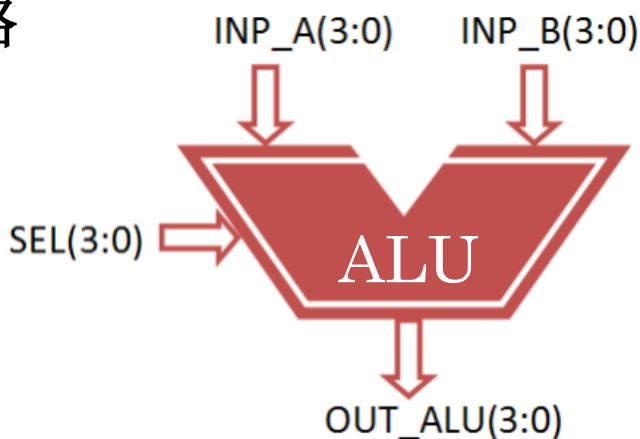
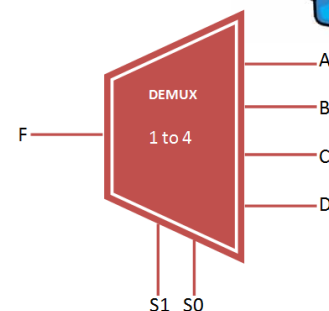
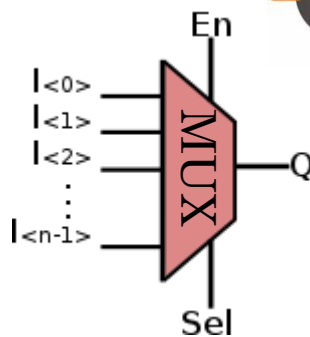
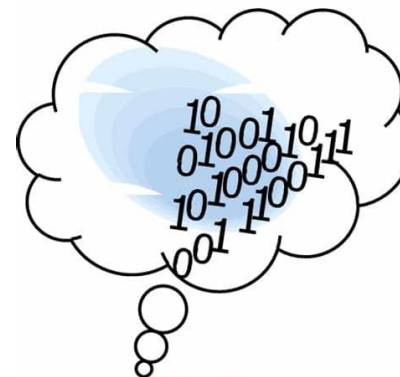
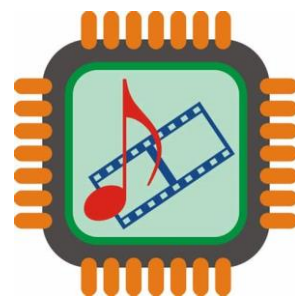
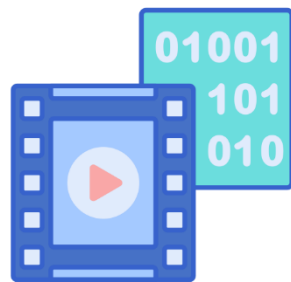
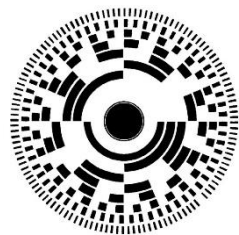


Binary

Cin	A				B				Sum				Carry
	A3	A2	A1	A0	B3	B2	B1	B0	S3	S2	S1	S0	Cout
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1	0	0	1	0	0
0	0	0	1	0	0	0	1	0	0	1	0	0	0
0	0	0	1	1	0	0	1	1	0	1	1	0	0
0	0	1	0	0	0	1	0	0	1	0	0	0	0
0	0	1	0	1	0	1	0	1	1	0	1	0	0
0	0	1	1	0	0	1	1	0	1	1	0	0	0
0	0	1	1	1	0	1	1	1	1	1	1	0	0
0	1	0	0	0	1	0	0	0	0	0	0	0	1
0	1	0	0	1	1	0	0	1	0	0	1	0	1
0	1	0	1	0	1	0	1	0	0	1	0	0	1
0	1	0	1	1	1	0	1	1	0	1	1	0	1
0	1	1	0	0	1	1	0	0	1	0	0	0	1
0	1	1	0	1	1	1	0	1	1	0	1	0	1
0	1	1	1	0	1	1	1	0	1	1	0	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

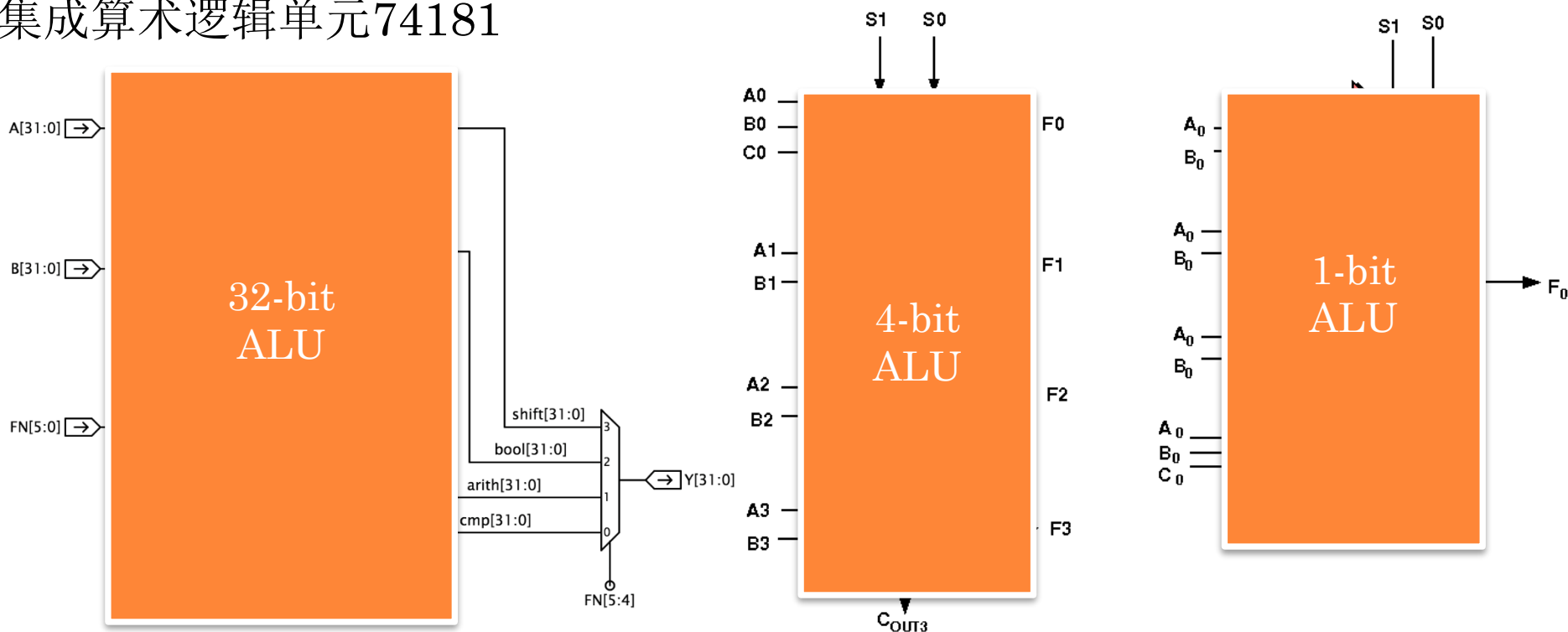
内容

- 6.1 编码器
- 6.2 译码器
- 6.3 数据选择器
- 6.4 数据分配器
- 6.5 数值比较器
- 6.6 奇偶校验位产生与校验电路
- 6.7 算术运算电路
- 6.8 算术逻辑单元ALU
- 本章小结

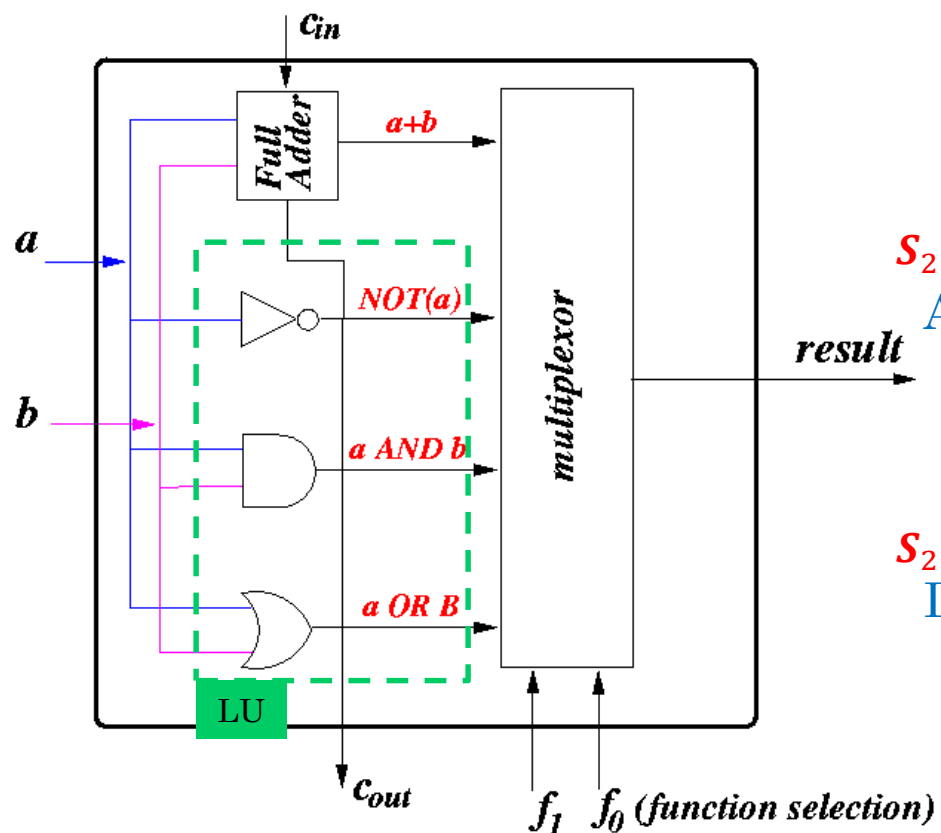


算术逻辑单元 (ALU) ARITHMETIC LOGIC UNIT

- 算术逻辑单元是指能够完成一系列**算术运算**和**逻辑运算**的电路，是构成计算机的核心部件。
 - 1位算术逻辑单元的设计
 - 集成算术逻辑单元74181



1位算术逻辑单元 (ALU) 的设计



Functional Description of Arithmetic Logic Unit (ALU)

Selection Input			Operation Performed
S_2	S_1	S_0	
0	0	0	$A + B$
0	0	1	$A - B$
0	1	0	$A - 1$
0	1	1	$A + 1$
1	0	0	$A \text{ and } B$
1	0	1	$A \text{ or } B$
1	1	0	not A
1	1	1	$A \text{ xor } B$



本章小结

- 本章介绍了常用的组合逻辑功能器件及其应用。
- 介绍了各种组合逻辑功能模块的概念和基本功能，
 - 从设计或者分析的角度出发，
 - 阐述了器件模块的设计或者分析方法
 - 并给出了器件模块的功能表和逻辑符号
- 结合ALU电路，介绍了层次化的组合逻辑电路设计方法
 - 这是大规模数字系统设计的常用方法，为数字系统的设计奠定基础。

教材勘误

页码	错误位置	错误描述
147	图6.5	图中的连线不清楚，没有标出交点
148	公式（6-4）（6-5）	漏了 I_0'
149	页面中间	" $I_{15}'-I_8$ ",其中 I_8 应为 I_8'
149	表格上方	“二-十进制编码器” → “十-二进制编码器”，即10线-4线编码器
151	公式（6-6）（6-7）	两个公式中间空格分开得不大，容易看错。
171	图6.37	图中的连线不清楚，没有标出交点
174	表6.17	删除表格中的Fodd和Feven，这两个是奇偶判定函数，不是奇偶校验位。
176	图6.41	公式（6-41）与图Feven逻辑表达式对应不起来，图画错了。
179	6.7.3节	没有解释X,G的代表含义，突然开始使用。建议看清华课本解释
182	表6.21	没有解释A,B,C(-1),F, Co的含义。
185	中间一段话的第二行	74157是4个二选一数据选择器，不是4个双二选一.....
201	6.26题	“4选1”改成“8选1”，倒数第二项去掉中间的F，还需要或门